Build Web, Guide AI

Business Web Development with AI

Michael Borck

2026-11-07

Build Web, Guide AI
Business Web Development with AI

Copyright © 2026 Michael Borck.

Book content: Creative Commons Attribution 4.0 International (CC BY 4.0 International).
https://creativecommons.org/licenses/by/4.0/

Original code examples and accompanying software: MIT License.
https://opensource.org/licenses/MIT

These grants cover material the author has authority to license. Separately credited third-party material retains its stated terms.

Published by Michael Borck
Perth, Western Australia

ISBN: 979-8-255-17415-7

First edition, 2026.

AI disclosure: This book was written using the methodology it describes. AI tools were used as thinking partners throughout the drafting, iterating, and refining process. The author reviewed, challenged, and took responsibility for every sentence and line of code.

Companion website: https://michael-borck.github.io/build-web-guide-ai
Source: https://github.com/michael-borck/build-web-guide-ai

Table of contents

Preface 1
Recommended Resources 7
Acknowledgments 21

I Your AI Development Partnership **23**
1 Understanding Your AI Web Development Partner 25

II The Practice Spine **33**
2 Specify Before You Build 35
3 Checks That Run 59
4 The Agent-Ready Repo and Handover 77

III Web Foundations **95**
5 Structure: HTML as Information Architecture 97
6 Presentation: CSS as Visual Communication 109
7 Behaviour: JavaScript as User Interaction 125
8 Connection: APIs as Business Integration 141
9 Project: Portfolio Website 157

IV Content Management for Business **167**
10 Why CMS? The Business Value of Managed Content 169
11 WordPress as Platform 181
12 Extending Platforms 195
13 Headless Architecture 221
14 Project: Business Website with WordPress 239

V Modern Frontend Development **249**
15 Component Thinking: React 251

16 Rapid Development: CSS Frameworks 269
17 Professional Practices 285
18 Project: React Integration 303

VI The Professional Developer **317**
19 Evaluating Emerging Technologies 319
20 Your Development Career 333
21 The AI-Era Developer 347

VII Appendices **361**
How to Use This Book 363
2027 Course Map 365
References 369
About the Author 371

Preface

Why This Book Exists

Most web development books teach you technologies. This one teaches you to make technology decisions.

AI can write HTML, CSS, and JavaScript faster than you ever will. If your only skill is writing markup, you are competing with something that does not sleep. But if you can evaluate whether a client needs a WordPress site or a React app, explain why to a non-technical stakeholder, and direct AI to build it correctly, you have the skills that actually matter.

This book teaches web development as a business decision-making skill. Every chapter starts with the concept, not the code. You explore ideas through AI conversation before writing a single line. By the time you reach the syntax, you understand what it means and why it exists. That understanding is what lets you evaluate AI output, choose the right technology for the problem, and communicate your reasoning to people who do not write code.

The 2027 edition adds the practices around the build: specify what the client needs, write checks that run, and hand over a repository another person can maintain. An agent can produce a site from a sentence. It cannot know what the client meant, test it on a phone on a bus, or tell you whether it is any good. Those judgements remain yours.

The book covers the full spectrum of web decisions: hand-coded HTML/CSS/JavaScript for control, WordPress for rapid deployment, headless architecture for flexibility, React for modern applications. You learn when to use each one as well as how.

The fundamentals in this book are not there to compete with AI-generated HTML. They are there because you need enough of the layer to specify what the client needs, evaluate whether AI built it correctly, and know when the output is plausible but wrong. That judgement is the trust infrastructure that makes AI-generated web work safe.

If You Have Been Vibe Coding Websites

You may have already built a website by prompting AI. For a simple landing page or portfolio, that works. AI generates the HTML, you paste it into a file, it looks fine.

But as sites get more complex, questions arise that vibe coding cannot answer. Why does the layout break on mobile? Should this be a WordPress site or a custom build? How do I connect to an API without exposing credentials? Why is the page slow?

This book gives you a look under the hood. Through simple projects, you learn how the parts fit together: how HTML structures content, how CSS controls presentation, how JavaScript adds behaviour, how APIs connect to external data. That understanding makes you better at guiding AI, whether you continue vibe coding or move toward more intentional development.

If you are curious about what AI is actually producing, or if you have hit the point where prompting alone is not enough, this book is a good next step.

Who This Book Is For

- Professionals who need to understand web development for their roles
- Career changers entering tech from business backgrounds
- Anyone who wants to build web solutions with AI as a partner
- People who need to evaluate web technologies, not just use them

No prior programming experience required. We start from scratch.

What This Book Is Not

This is not a computer science textbook. It does not cover algorithms, data structures, or computational theory. It teaches you to build things for the web, with enough understanding to make good decisions about how.

It is not a framework-of-the-month guide. You will learn HTML, CSS, JavaScript, WordPress, and React, technologies chosen because they cover the spectrum from simple to complex and will remain relevant regardless of what is trending next year.

It is not a book that treats AI as a novelty. AI is your development partner throughout. The book teaches you to architect solutions that AI helps you build, evaluate AI output with professional judgement, and communicate requirements clearly enough for AI to be useful.

And it is not a book for people who want to become full-time frontend engineers. It is for people who want to understand web development well enough to lead projects, evaluate vendors, build prototypes, and make informed technology decisions in a business context.

If You Are Feeling Uncertain

You do not need to be "technical" to build for the web. That word gets used as a gatekeeper, but the reality is that web development is a learnable skill, not a personality trait. If you can write a clear email, you can learn to write a clear prompt. If you can organise a spreadsheet, you can learn to structure a web page. This book starts where you are.

How This Book Is Structured

The book progresses from foundations to full applications:

Part I: Your AI Development Partnership introduces how to work with AI effectively from day one.

Part II: The Practice Spine covers the practices that make an AI-built site trustworthy: specifying before you build, checks that run, and the agent-ready repo and handover. These are the working chapters for the 2027 edition. Read them when you are building, and return to them at handover.

Part III: Web Foundations covers web fundamentals using vanilla HTML, CSS, and JavaScript. No frameworks, no build tools. This is deliberate. Every framework, React included, ultimately produces HTML, CSS, and JavaScript that a browser renders. Understanding these fundamentals means you can debug anything built on top of them. It also teaches you to control AI. If you delegate a web project to AI without constraints, it will probably choose React, set up a build system, and produce something that works but that you cannot modify or explain. Part III gives you practice guiding AI within constraints, building the habit of directing rather than delegating.

Part IV: Content Management for Business moves to platforms: WordPress for rapid development, content management, and extending with plugins and themes.

Part V: Modern Frontend Development introduces React for component-based applications, headless architecture, and modern development patterns.

Part VI: The Professional Developer ties it all together: professional practices, emerging technologies, and building a development career.

This edition supports ISYS3004 Business Web and Mobile Technologies, S1 2027. The unit materials are the source of truth; this book is the reference. The 2027 course map appendix shows which chapters support each module.

Each chapter follows a consistent pattern:

1. **Concept first.** A real-world analogy before any code. Restaurant menus for HTML structure, café atmospheres for CSS, ordering systems for APIs.
2. **AI exploration.** Guided prompts where you explore the concept through conversation with your AI assistant. You build understanding before you build anything.
3. **From concept to code.** The simplest implementation that demonstrates the idea. No walls of syntax.
4. **Business connection.** Why this technology matters commercially. When to use it, when not to.
5. **Practice and reflection.** Exercises levelled from direct application to open-ended design challenges, plus journal prompts for deeper thinking.

Conventions Used in This Book

Code examples appear with syntax highlighting:

```
<h1>Hello, World</h1>
```

AI prompts and non-code examples appear as plain text blocks:

```
Ask your AI:
Show me the simplest HTML page that displays a
heading
```

Throughout the chapters you will find:

- **AI Explorations**: guided prompts for learning concepts through AI conversation
- **Callout boxes**: tips, notes, and warnings highlighted for reference
- **Practice Exercises**: levelled from direct application (Level 1) to open-ended design (Level 5)
- **Learning Journal**: reflection prompts at the end of each chapter

The Series

This book is part of a series designed to help you master modern software development in the AI era.

The series has a trunk: *Conversation, Not Delegation* for method and *The Human Edge* for professional judgement. This book is the web development branch.

Two ways in:

Start with	Continue with	Because
Conversation, Not Delegation	*Build Web, Guide AI* (this book), or *Converse Python, Partner AI*	You want the ideas before (or without) code
Think Python, Direct AI	*Code Python, Consult AI*, then *Ship Python, Orchestrate AI*	You want to code from your first page

Converse Python, Partner AI can be read alongside either path; its methodology enhances every one.

- **Conversation, Not Delegation**: the general methodology for working with AI across any discipline. No code required; the broadest entry point.
- **Think Python, Direct AI**: computational thinking for absolute beginners. The entry point if you want to code from the start.
- **Converse Python, Partner AI**: the intentional prompting methodology applied to software development. A companion you can read alongside any Python book.
- **Code Python, Consult AI**: focused Python fundamentals through AI conversation.
- **Ship Python, Orchestrate AI**: professional Python development practices and tooling.
- **The Absolute Minimum You Must Know**: the free, always-current web companion at minimum.borck.education (https://minimum.borck.education), with quick primers on every topic this series covers in depth.
- **Build Web, Guide AI** (this book): web development with AI as your development partner.

All titles are available at books.borck.education (https://books.borck.education).

Ways to Engage with This Book

This book is available in several formats. Pick whichever fits how you work and learn.

- **Read it online.** The full book is freely available at the companion website, with dark mode, search, and navigation.

- **Read it on paper or e-reader.** Available as a paperback and ebook through Amazon KDP.
- **Converse with it.** The online edition includes a chatbot grounded in the book's content.
- **Feed it to your own AI.** The `llm.txt` file provides a clean text version of the entire book, ready to paste into ChatGPT, Claude, or any AI tool.
- **Run the code.** All code examples and project files are available on GitHub (https://github.com/michael-borck/build-web-guide-ai). DeepWiki (https://deepwiki.com/michael-borck/build-web-guide-ai) provides an AI-navigable view of the repository.
- **Browse all books.** This book is part of a series. See all titles at books.borck.education (https://books.borck.education).

The online version is always the most current.

Source Code & Feedback

All code examples and project files are available at: https://github.com/michael-borck/build-web-guide-ai

Found an error? Have a suggestion?

- Open an issue: https://github.com/michael-borck/build-web-guide-ai/issues
- Email: michael@borck.me

Recommended Resources

This reading list accompanies the book and collects external resources organised by topic. Use it alongside the chapters for different perspectives and official documentation.

> **Reading Strategy**
>
> Read the book chapter first to understand concepts, then explore external resources for different perspectives and official documentation.

Getting Started with the Web

Companion Chapter

Chapter 1

Essential Reading

Resource	Topic	Link
MDN	Getting started with the web	MDN: Getting Started (https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web)
web.dev	How the web works	web.dev: How the web works (https://web.dev/articles/howbrowserswork)

Resource	Topic	Link
W3Schools	Introduction to HTML	W3Schools: HTML Introduction (https://www.w3schools.com/html/html_intro.asp)

Deeper Dive

- History of the Web (https://www.w3.org/History.html) - W3C
- How browsers work (https://developer.chrome.com/blog/inside-browser-part1) - Google Chrome Blog

---

HTML Structure

Companion Chapter

Chapter 5

Essential Reading

Resource	Topic	Link
MDN	HTML basics	MDN: HTML Basics (https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/HTML_basics)
MDN	Document structure	MDN: Document Structure (https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML/Document_and_website_structure)

Resource	Topic	Link
web.dev	Semantic HTML	web.dev: Semantic HTML (https://web.dev/learn/html/s html)
W3Schools	HTML Elements	W3Schools: HTML Elements (https://www.w3schools.com/h ements.asp)

Deeper Dive

- HTML Living Standard (https://html.spec.whatwg.org/) - WHATWG (reference)
- HTML Best Practices (https://github.com/hail2u/html-best-practices) - Community guide

---

CSS Presentation

Companion Chapter

Chapter 6

Essential Reading

Resource	Topic	Link
MDN	CSS basics	MDN: CSS Basics (https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/CSS sics)
MDN	CSS layout	MDN: CSS Layout (https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout)

Resource	Topic	Link
web.dev	Learn CSS	web.dev: Learn CSS (https://web.dev/learn/css)
web.dev	Responsive design	web.dev: Responsive Design (https://web.dev/articles/responsive-web-design-basics)
W3Schools	CSS Introduction	W3Schools: CSS (https://www.w3schools.com/css/c tro.asp)

Deeper Dive

- CSS Tricks: A Complete Guide to Flexbox (https://css-tricks.com/snippets/css/a-guide-to-flexbox/)
- CSS Tricks: A Complete Guide to Grid (https://css-tricks.com/snippets/css/complete-guide-grid/)
- Every Layout (https://every-layout.dev/) - Layout patterns

JavaScript Behaviour

Companion Chapter

Chapter 7

Essential Reading

Resource	Topic	Link
MDN	JavaScript basics	MDN: JavaScript Basics (https://developer.mozilla.org/en-US/docs/Learn/Getting_started_with_the_web/JavaSc sics)

Resource	Topic	Link
MDN	DOM Introduction	MDN: DOM Introduction (https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction)
MDN	Events	MDN: Introduction to Events (https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Building_blocks/Events)
javascript.info	The Modern JavaScript Tutorial	javascript.info (https://javascript.info/)
W3Schools	JavaScript Tutorial	W3Schools: JavaScript (https://www.w3schools.com/j

Deeper Dive

- Eloquent JavaScript (https://eloquentjavascript.net/) - Free online book
- You Don't Know JS (https://github.com/getify/You-Dont-Know-JS) - Deep dive series

---

APIs and Data

Companion Chapter

Chapter 8

Essential Reading

Resource	Topic	Link
MDN	Fetching data	MDN: Fetching Data (https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Client-side_web_APIs/Fetching_data)
MDN	Working with JSON	MDN: Working with JSON (https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Objects/JSON)
MDN	Async JavaScript	MDN: Asynchronous JavaScript (https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous)
web.dev	Introduction to fetch	web.dev: Fetch API (https://web.dev/articles/introduction-to-fetch)

Deeper Dive

- REST API Tutorial (https://restfulapi.net/) - REST concepts
- JSONPlaceholder (https://jsonplaceholder.typicode.com/) - Practice API
- Public APIs List (https://github.com/public-apis/public-apis) - APIs to explore

---

Accessibility

Companion Chapter

Chapter 9

Essential Reading

Resource	Topic	Link
web.dev	Learn Accessibility	web.dev: Learn Accessibility (https://web.dev/learn/accessibility)
MDN	Accessibility	MDN: Accessibility (https://developer.mozilla.org/en-US/docs/Learn/Accessibility)
W3C	WCAG Quick Reference	WCAG 2.1 Quick Ref (https://www.w3.org/WAI/WCref/)
WebAIM	Introduction to Accessibility	WebAIM: Introduction (https://webaim.org/intro/)

Deeper Dive

- A11y Project Checklist (https://www.a11yproject.com/checklist/)
- Inclusive Components (https://inclusive-components.design/) - Accessible patterns

---

Content Management Systems

Companion Chapters

Chapter 10 and Chapter 11

Essential Reading

Resource	Topic	Link
WordPress.org	Getting Started	WordPress: Getting Started (https://wordpress.org/documentation/article/get-started-with-wordpress/)
WordPress.org	WordPress Features	WordPress: Features (https://wordpress.org/about/features/)
W3Techs	CMS Usage Statistics	W3Techs: CMS (https://w3techs.com/technologies/overview/content_management)
WPBeginner	WordPress Guide	WPBeginner: Guide (https://www.wpbeginner.com/guides/)

Deeper Dive

- WordPress Developer Resources (https://developer.wordpress.org/)
- WordPress TV (https://wordpress.tv/) - Video tutorials

Extending WordPress

Companion Chapter

Chapter 12

Essential Reading

Resource	Topic	Link
WordPress.org	Theme Handbook	Theme Handbook (https://developer.wordpress.org/themes/)
WordPress.org	Plugin Handbook	Plugin Handbook (https://developer.wordpress.org/plugins/)
WordPress.org	Child Themes	Child Themes (https://developer.wordpress.org/themes/advanced-topics/child-themes/)
WPBeginner	Must Have Plugins	Essential Plugins (https://www.wpbeginner.com/showcase/24-must-have-wordpress-plugins-for-business-websites/)

Deeper Dive

- WordPress Coding Standards (https://developer.wordpress.org/coding-standards/)
- Theme Unit Test (https://codex.wordpress.org/Theme_Unit_Test)

Headless Architecture and REST APIs

Companion Chapter

Chapter 13

Essential Reading

Resource	Topic	Link
WordPress.org	REST API Handbook	REST API Handbook (https://developer.wordpress.org/rest-api/)
WordPress.org	REST API Reference	REST API Reference (https://developer.wordpress.org/rest-api/reference/)

Deeper Dive

- Postman (https://www.postman.com/) - API testing tool
- WP REST API Demo (https://developer.wordpress.org/rest-api/using-the-rest-api/)

React

Companion Chapter

Chapter 15

Essential Reading

Resource	Topic	Link
React.dev	Quick Start	React: Quick Start (https://react.dev/learn)
React.dev	Thinking in React	Thinking in React (https://react.dev/learn/thinking-in-react)
React.dev	Tutorial	React Tutorial (https://react.dev/learn/tutorial-tic-tac-toe)

Resource	Topic	Link
MDN	React Getting Started	MDN: React (https://developer.mozilla.org/en-US/docs/Learn/Tools_and_testing/Client-side_JavaScript_frameworks/React_getting_started)

Deeper Dive

- React Patterns (https://reactpatterns.com/)
- React TypeScript Cheatsheet (https://react-typescript-cheatsheet.netlify.app/)

Frameworks and Professional Practice

Companion Chapters

Chapter 16 and Chapter 17

Essential Reading

Resource	Topic	Link
Bootstrap	Getting Started	Bootstrap Docs (https://getbootstrap.com/docs/)
Tailwind CSS	Getting Started	Tailwind Docs (https://tailwindcss.com/docs)
Git	Git Handbook	GitHub: Git Handbook (https://guides.github.com/introduction/git-handbook/)
web.dev	Testing	web.dev: Testing (https://web.dev/learn/testing)

Deeper Dive

- Atlassian Git Tutorials (https://www.atlassian.com/git/tutorials)
- Testing Library (https://testing-library.com/)

Emerging Technologies and Career

Companion Chapters

Chapter 19, Chapter 20, and Chapter 21

Essential Reading

Resource	Topic	Link
web.dev	Web Vitals	web.dev: Web Vitals (https://web.dev/articles/vitals)
web.dev	PWA	web.dev: Learn PWA (https://web.dev/learn/pwa)
State of JS	Survey Results	State of JS (https://stateofjs.com/)
State of CSS	Survey Results	State of CSS (https://stateofcss.com/)

Deeper Dive

- Roadmap.sh (https://roadmap.sh/) - Developer roadmaps
- The Pragmatic Engineer (https://blog.pragmaticengineer.com/) - Career insights
- Josh W Comeau's Blog (https://www.joshwcomeau.com/) - Modern CSS/React

---

Quick Reference: External Resources

Official Documentation

Resource	Best For	URL
MDN Web Docs	Comprehensive reference	developer.mozilla.org (https://developer.mozilla.org/)
web.dev	Modern best practices	web.dev (https://web.dev/)
W3Schools	Quick examples	w3schools.com (https://www.w3schools.c
WordPress.org	WordPress reference	wordpress.org (https://wordpress.org/)
React.dev	React documentation	react.dev (https://react.dev/)

Learning Platforms

Resource	Type	URL
freeCodeCamp	Interactive tutorials	freecodecamp.org (https://www.freecodecamp.org/)
Codecademy	Interactive courses	codecademy.com (https://www.codecademy.co
Scrimba	Video + coding	scrimba.com (https://scrimba.com/)
Frontend Masters	Video courses (paid)	frontendmasters.com (https://frontendmasters.com/)

Community & News

Resource	Type	URL
Dev.to	Articles & community	dev.to (https://dev.to/)

Resource	Type	URL
CSS Tricks	CSS & frontend	css-tricks.com (https://css-tricks.com/)
Smashing Magazine	Web design & dev	smashing-magazine.com (https://www.smashing-magazine.com/)
Hacker News	Tech news	news.ycombinator.com (https://news.ycombinator.com/)

Tools

Tool	Purpose	URL
Can I Use	Browser compatibility	caniuse.com (https://caniuse.com/)
CodePen	Code playground	codepen.io (https://codepen.io/)
GitHub	Code hosting	github.com (https://github.com/)
Lighthouse	Performance testing	Built into Chrome DevTools

Reading Tips

1. **Don't read everything** - Focus on Essential Reading first
2. **Hands-on over theory** - Try examples as you read
3. **MDN is your friend** - Best for accurate, detailed explanations
4. **web.dev for modern practices** - Google's recommendations
5. **W3Schools for quick reference** - Fast examples, less depth
6. **Use AI to explain** - If an article is confusing, ask AI to explain it differently

Acknowledgments

This book grew out of teaching business web and mobile technologies to students who were not computer science majors but needed to build real things for real purposes. The exercises, case studies, and projects in these pages were tested in classrooms, and the students who worked through them shaped every chapter. Their questions, especially the ones that started with "but why would I…", kept the book focused on what actually matters in a business context.

Colleagues in business and information systems provided the cross-disciplinary perspective that kept the technical content grounded in professional reality. The book is better for every conversation about what employers actually need graduates to know.

The web development community's open documentation (MDN Web Docs, the WordPress documentation, the React documentation) makes resources like this possible. The open source community behind Quarto, GitHub, and GitHub Pages made it possible to write, build, and publish across multiple formats without a traditional publisher.

AI tools were used throughout the writing process. Claude (Anthropic) served as a conversation partner for drafting, iterating, and refining. The process was the same one the book teaches: guide the AI, evaluate its output, and maintain professional judgement throughout. Every chapter reflects the author's decisions. The AI made the work faster. It did not make the decisions.

Part I

Your AI Development Partnership

Chapter 1

Understanding Your AI Web Development Partner

1.1 A New Way to Build for the Web

Right now, AI can write HTML, CSS, and JavaScript in seconds. It can create entire websites, debug code, and explain complex concepts. So why learn web development at all?

AI is fast at writing code, but it doesn't understand what your business needs. You're the architect, the translator between business requirements and technical solutions. AI is your highly skilled assistant who needs clear direction.

This book teaches you to be that architect.

1.2 The Partnership Experiment

This experiment shows how AI works as a web development partner, and it will shape how you learn throughout this book.

1.2.1 Round 1: The Vague Request

Open your AI assistant (ChatGPT, Claude, or whatever you're using). Type this exactly:

```
Make a website
```

What did you get? The AI probably asked questions or made assumptions. This is your first lesson: **AI needs direction**.

1.2.2 Round 2: The Simple Request

Now try:

```
Make a business website
```

You likely got a massive amount of HTML, CSS, maybe JavaScript: a complete but generic template. **This is AI's default: give you everything at once**.

1.2.3 Round 3: The Learning Request

Now try:

```
I'm learning web development. Show me the absolute
simplest HTML page that displays a business name
and tagline. Nothing else.
```

You might get:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Business</title>
</head>
<body>
    <h1>Sunrise Café</h1>
    <p>Fresh coffee, warm welcome</p>
</body>
</html>
```

Much clearer! **AI responds to your learning needs when you express them clearly**.

1.2.4 Round 4: The Concept Request

Finally, try:

```
Explain how a web page is structured, using a
building analogy, before showing any code
```

Now you're using AI to build understanding as well as generate code.

1.3 What This Experiment Teaches Us

1. **AI defaults to complexity:** it assumes you want a "complete" solution
2. **Your prompts shape your learning:** clear learning goals get clearer responses
3. **Concepts before code:** you can use AI to understand ideas before syntax
4. **You're in control:** AI follows your lead, not the other way around

1.4 The Three Learning Strategies

Throughout this book, we'll follow three core strategies:

1.4.1 Strategy 1: Understand the Concept Before the Code

Every web development task follows patterns. Understand the pattern first, then learn how HTML/CSS/JavaScript expresses it.

Example: Don't ask "How do I make a navigation menu?" Instead, ask "What is the purpose of navigation on a website?" Then, "Show me the simplest HTML navigation."

1.4.2 Strategy 2: Use AI to Explore, Not to Avoid Learning

AI is your exploration tool. Use it to:

- See different approaches to the same problem
- Understand why code works
- Compare technologies
- Debug your understanding

1.4.3 Strategy 3: Build Mental Models, Not Just Working Websites

Aim to understand how and why a website works, and use AI to build those mental models.

Example: Ask "Draw a diagram showing how HTML, CSS, and JavaScript work together" or "Explain responsive design using a newspaper analogy."

1.5 How AI Thinks vs How Developers Think

1.5.1 AI Thinks in Patterns

- It has seen millions of websites
- It pattern-matches to give you a "typical" solution

- It doesn't understand your specific business context
- It can't know what you don't know yet

1.5.2 Developers Think in Problems

- What exactly needs to be solved?
- Who is the user?
- What's the simplest solution?
- How will this be maintained?
- What could go wrong?

Your job is to bridge this gap: Think like a developer, then guide AI to help you implement.

1.6 Your Progressive AI Journey

1.6.1 Part I (Your AI Development Partnership): AI as Partner

```
Example prompts:
- "Explain how you approach a web build, and what
you need from me before you start"
- "What assumptions are you making about this
client?"
- "What should I check before I trust that
answer?"
```

1.6.2 Part II (The Practice Spine): AI as Practice Partner

```
Example prompts:
- "Turn this client email into five checkable
acceptance criteria"
- "Write a check that would fail if the booking
action were invisible at 375px"
- "Review this diff for scope creep and tell me
what changed outside the task"
```

1.6.3 Part III (Web Foundations): AI as Concept Explorer

```
Example prompts:
- "Explain semantic HTML using a library analogy"
- "Show me 5 ways CSS can position elements,
```

```
simplest first"
- "What happens when a user clicks a button?
Explain the event flow"
```

1.6.4 Part IV (Content Management for Business): AI as Implementation Assistant

```
Example prompts:
- "I need a WordPress theme for a café. What
questions should I ask before choosing?"
- "Compare these two plugins for contact forms.
What are the trade-offs?"
- "Explain REST APIs as if I'm explaining to a
business stakeholder"
```

1.6.5 Part V (Modern Frontend Development): AI as Code Producer

```
Example prompts:
- "Create a React component that displays a list
of products. Here's my design..."
- "Convert this WordPress post display to a React
component"
- "Review this code for accessibility issues"
```

1.6.6 Part VI (The Professional Developer): AI as Strategic Advisor

```
Example prompts:
- "Compare PWAs vs native apps for a small retail
business"
- "What should a junior developer's portfolio
demonstrate?"
- "What web technologies should I learn next,
given I know X, Y, Z?"
```

1.7 The Honest Truth

By the end of this book:

- **AI will still write code faster than you**

- **But you'll know what to ask for**
- **You'll understand what it gives you**
- **You'll fix it when it's wrong**
- **You'll explain it to stakeholders**
- **You'll be the architect, not the typist**

This is the actual job of a modern web developer.

1.8 Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

1.8.1 Exercise 1.1: Prompt Evolution (Level 1)

Start with "website" and evolve your prompt through 4 iterations to get the simplest possible "About Us" page. Document each iteration and what changed.

1.8.2 Exercise 1.2: Concept Extraction (Level 2)

Ask AI to explain "responsive design" three different ways: 1. Using a technical explanation 2. Using a real-world analogy 3. Using a business justification

Which helped you understand best? Why?

1.8.3 Exercise 1.3: AI Comparison (Level 3)

Ask the same question to two different AI tools (or the same tool twice). Compare the responses. What's consistent? What differs? What does this tell you about using AI?

1.8.4 Exercise 1.4: Business Translation (Level 4)

A client says: "I want a website that looks professional and works on phones."

Write three different AI prompts that would help you: 1. Clarify what "professional" means 2. Understand their mobile requirements 3. Explore technology options

1.8.5 Exercise 1.5: AI Learning Plan (Level 5)

Design your personal AI learning strategy for this book: 1. What kinds of prompts will you start with? 2. How will you verify AI's answers? 3. What questions will you ask to deepen understanding? 4. How will you track what you've learned?

Create a "My AI Partnership Plan" document.

1.9 Chapter Summary

- AI is your learning partner, not your replacement
- Clear prompts lead to clear learning
- Understanding concepts matters more than memorising syntax
- You're learning to be an architect who uses AI as a tool
- Business context shapes everything in web development

1.10 Reflection

Before moving on, ensure you:

☐ Completed the Partnership Experiment
☐ Understand why AI overcomplicates by default
☐ Can evolve prompts from vague to learning-focused
☐ See yourself as an architect, not a code typist
☐ Have a plan for using AI as a learning partner

1.11 Your Learning Journal

Start your learning journal now. For this chapter, record:

1. **Partnership Experiment Results**: What surprised you about AI's responses?
2. **Best Prompt**: Which prompt got you the most useful response?
3. **Mental Model**: How do you now think about AI as a development partner?
4. **Business Question**: What kind of web solutions do you want to build?

1.12 Related Materials

This book is part of a series on software development in the AI era:

Foundational Methodology

- Converse Python, Partner AI: The Python Edition (https://michael-borck.github.io/converse-python-partner-ai)

Python Track

- Think Python, Direct AI: Computational Thinking for Beginners (https://michael-borck.github.io/think-python-direct-ai) - Perfect for absolute beginners
- Code Python, Consult AI: Python Fundamentals for the AI Era (https://michael-borck.github.io/code-python-consult-ai) - Core Python knowledge
- Ship Python, Orchestrate AI (https://michael-borck.github.io/ship-python-orchestrate-ai) - Professional Python in the AI Era

Web Track

- Build Web, Guide AI: Business Web Development with AI (https://michael-borck.github.io/build-web-guide-ai) (this book) - HTML, CSS, JavaScript, WordPress, React

1.13 Next Steps

In the next chapter, we will explore how web pages are structured using HTML. You'll use your new prompt skills to discover semantic markup, the foundation of everything we build. We'll start with the concept of "documents as structured information" before writing a single line of code.

You are learning to think like a web professional who directs AI to help build solutions.

Part II

The Practice Spine

Chapter 2

Specify Before You Build

An agent will build whatever you describe, including the parts you did not mean. Give it a sentence and it fills the gaps with the most common pattern it has seen. That pattern might be a carousel, a contact form, three pricing cards, or a stock photo of strangers laughing at a salad. The agent is doing what the brief allows.

Maria's first email contains one sentence of need wrapped in a launch date. The distance between that email and a working site is where projects succeed or fail. This chapter is about closing that distance before the build starts.

2.1 The Concept First

A specification is a written contract for a build. It records who the site serves, what those people need to do, what content exists, which design decisions are fixed, what counts as done, and what is excluded. You write it before the build and use it during the build. When the client asks for a change, the spec tells you whether the change is inside the agreement or a new request.

A wish list has a different job. It records what the client likes. A contract records what the build must do.

Wish list	Contract
A gallery	Six photos, owned by Maria, each with alt text, on the home page
Mobile friendly	At 375px, no page scrolls horizontally and the booking action is visible without scrolling
Easy to update	Dev can update the menu in under five minutes without editing code

Wish list	Contract
Professional	Brand palette and type scale applied, no stock photos, hours correct on every page

A contract can be tested. A wish can only be discussed.

The test of a spec is direct. Give it to a builder who has never met the client. If that builder produces something the client would accept, the spec did its job. If the builder has to ask what the client meant, the spec has a gap.

2.2 Understanding Through a Renovation Brief

A cafe renovation has the same problem. The owner says she wants the room to feel warmer. The builder can install a heater, paint a wall, or add lamps. All three might make the room warmer. None of them can be accepted or rejected against the word “warmer” alone.

A useful renovation brief says which walls are moving, which lights are being replaced, which finishes are approved, which parts of the room stay, and what the client will inspect before paying. The builder still makes design decisions. The brief sets the boundaries those decisions must respect.

A web spec works the same way. “Make it professional” is the feeling. “Brand palette, type scale, six photos, booking action above the fold at 375px, hours correct on every page” is the brief. The first gives the agent room to invent. The second gives the agent a standard to meet.

> **A spec sets the boundary**
>
> You do not need to tell the agent which HTML element to use. You need to tell it what the result must do. The boundary is the contract. The implementation is the agent’s job.

2.3 The Client’s Brief

This is Maria’s email. It is the brief that starts the client project.

> From: Maria To: Contractor Subject: Website for the new place
>
> Hi,

> We're opening the [suburb] store and I need a website. Nothing fancy. People should be able to find us, see the menu, and book a table. That's it.
>
> My nephew did one for his mum's salon in an afternoon with one of those AI things, so I don't think this is a big job. I've told the local paper we launch on [date]. Can you make that?
>
> I'll send you the logo and a few photos. Dev has the menu.
>
> Thanks, Maria

The brief gives you a purpose, a rough scope, a launch date, and a budget signal. It does not give you the things a builder needs.

It does not say who updates the menu, how often the menu changes, which device customers use, what "book" means, what happens after a booking, which content is current, or what "professional" looks like. It does not say what Maria would drop if the date slipped. It does not say how the site connects to the phone bookings the cafe already takes.

None of these gaps are Maria's fault. She is describing her business. She is not writing a technical document. Your job is to ask.

> ⚠ Do not fill a gap with a guess
>
> A guess in the spec becomes a defect in the build. Write the question down. Ask the person who knows. "I assumed" is not a requirement.

This chapter uses Harbour & Hearth Northbridge as a teaching example. Your fact sheet and interviews are the source of truth for your own franchise. Do not copy the example's content into your assessment. The method is the same.

2.4 Working with Your AI Partner

AI is useful in three places in this chapter. It can find the gaps in a brief, role-play the people who know the answers, and test whether a criterion can be read two ways. AI is a poor tool for writing the spec. The spec records your judgement, and your judgement is what your assessment rewards.

2.4.1 Exploration 1: Find the Gaps Without Filling Them

Paste Maria's email and ask for questions. Do not ask it to answer them.

```
Ask your AI:
Here is a client email. List every question you
```

```
would need answered before
you could build the site. Do not answer the
questions. Group them by the
person most likely to know: the owner, the office
manager, the customer,
or the IT vendor.
```

A useful answer will include questions about content ownership, update frequency, devices, the booking process, brand assets, hosting, and what happens when the site is wrong. If the AI starts answering its own questions, stop it. You want the gaps. Plausible filler hides them.

2.4.2 Exploration 2: Interview a Persona

A client simulator uses one persona per conversation. You can do the same with any AI tool.

```
Ask your AI:
Role-play Dev, the office manager at Harbour &
Hearth. I am the
contractor for this franchise. Answer only from
the fact sheet I
provide. Do not volunteer information. If I ask a
vague question, give
a vague answer. If I ask about your work, answer
as someone who updates
the menu and hours.

Fact sheet:
[paste your fact sheet here]
```

Then ask Dev what changes, how often, and who does it. Ask what changes in a normal week, and what catches her out. Ask what she would stop using the site over. The requirements that matter usually sit inside those answers. A feature list hides them.

2.4.3 Exploration 3: Test a Criterion for Ambiguity

Take a weak criterion and ask the AI to build two different sites that would both pass it.

```
Ask your AI:
Read this acceptance criterion: "The site is easy
```

```
to use on a phone."
Describe two different websites that would both
pass it. Then tell me
which words are doing too much work.
```

The answer should expose the problem. “Easy” has no observable result, and “phone” does not name a screen width, a device, or a context. A criterion that allows two unrelated builds cannot settle a disagreement.

2.5 From Brief to Spec

A practical spec template has seven parts, plus a log for the replication test. Each part answers a question a builder would otherwise have to guess.

```
# Spec: Harbour & Hearth Northbridge

**Version:** v1
**Client:** Maria, owner; Dev, office manager
**Last replication test:** v1

## Purpose

## Users and top tasks

## Content

### Content model

### Design tokens

## Must / Should / Won't

## Constraints

## Acceptance criteria

## Out of scope

## Replication test log
```

2.5.1 Purpose

Write one or two sentences from the client's point of view. Name the audience, the job and the outcome. Leave out the technology.

> Let local customers find the current menu, hours and booking action on a phone, and let Dev update the menu without calling the contractor.

Three requirements are carried in those two sentences. "Current" carries the weekly update. "On a phone" carries Sam's context. "Without calling the contractor" carries Dev's constraint. The purpose does not mention a framework, a plugin, or a page count.

2.5.2 Users and Top Tasks

Define users by what they need to do. Age and job title say less than context. Include the people inside the business. Maria pays for the site, but Sam uses it. Both belong in the spec.

User	Task	Device and context
Local customer	Find the current menu and hours	Phone, on the bus, often one-handed
Local customer	Book a table	Phone, about 30 seconds, possibly poor signal
Dev	Update the menu and opening hours	Desktop, during the working week, no code
Maria	Check that the site looks professional and current	Phone, in the store, between customers
Alex	Keep the site maintained	Occasional, remote, low tolerance for plugin sprawl

The context column changes the requirement. A booking at a desk with a fast connection is a different task from a booking on a bus with one bar of signal. The first can tolerate a long form. The second cannot.

2.5.3 Content and the Content Model

Content is what the site says. A content model is what the client edits.

Start with the content the business already has: the menu, the hours, the photos, the staff bios, the contact details. For each item, ask who owns it, how often it changes, which fields it needs, and where it appears.

Content type	Fields	Owner	Changes	Appears on
Menu	Sections; item name, description, price, dietary tags	Dev	To confirm in the interview	Menu, Home
Opening hours	Day, open, close, note	Dev	Rarely; public holidays	Home, Contact, Footer
Booking enquiry	Name, phone, party size, date, time, notes	Front of house	Per booking	Booking page, staff inbox
Photo	Image, alt text, caption, credit	Maria	Occasionally	Home, About
Promotion	Title, dates, image, link	Maria	Occasionally	Home
Site details	Address, phone, email, social links	Dev	Rarely	Everywhere

Maria says "see the menu". The brief does not say how often the menu changes or who updates it. The content model forces those questions. A spec that records "to confirm" is more honest than one that guesses. The interview replaces the placeholder with a frequency and an owner.

A content model is not a database schema. It is a list of the things the client edits and the fields each thing needs. You will meet the same idea in WordPress as custom post types and fields (Chapter 10).

Every field needs a reason. If nobody can say where a field appears or who changes it, remove it. A content model with twelve fields Dev never fills is a maintenance cost disguised as thoroughness.

2.5.4 Design Tokens

Maria wants the site to look professional and warm, and she does not want it to feel corporate. That is a direction. A builder cannot apply it consistently without decisions. Design tokens turn the direction into named values.

Tokens are the small set of decisions that every page reuses: colour roles, typefaces, type scale, spacing, radius, borders, shadows, motion and focus. A first set for

Northbridge might look like this.

Token	Value	Rule
`--colour-ink`	`#2b2118`	Body text. At least 7:1 contrast on `--colour-paper`
`--colour-paper`	`#f7f1e7`	Page background
`--colour-accent`	`#b4552d`	Primary action only. At least 4.5:1 with white text
`--font-display`	Fraunces, Georgia, serif	Headings
`--font-body`	Source Sans 3, system-ui, sans-serif	Body text
`--space-1` to `--space-6`	`0.25rem` to `1.5rem`	A spacing scale; every gap uses one of these values
`--radius`	`4px`	Buttons, cards, images
`--focus-ring`	`3px solid #1f5c99`	Every interactive element

```
:root {
  /* Colour roles. Pages refer to the role, not
  the hex value. */
  --colour-ink: #2b2118;
  --colour-paper: #f7f1e7;
  --colour-accent: #b4552d;
  --font-display: "Fraunces", Georgia, serif;
  --font-body: "Source Sans 3", system-ui,
  sans-serif;

  /* Spacing scale. Every gap uses one of these
  values. */
  --space-1: 0.25rem;
  --space-2: 0.5rem;
  --space-3: 0.75rem;
  --space-4: 1rem;
  --space-6: 1.5rem;
  --radius: 4px;
  --focus-ring: 3px solid #1f5c99;
}
```

The spec names the roles and the constraints. The build applies them. Chapter 6 shows how CSS uses tokens; this section is about deciding them before the agent starts.

Contrast belongs in the token decision. A warm accent that fails contrast is not usable for text, however good it looks in the brand guide. Two font families are also a performance decision. If the brand allows, use one family and a system fallback.

2.5.5 Must, Should and Won't

Priorities are how a spec survives a bad week. If everything is a must, the first slip becomes a negotiation about everything.

Priority	Meaning	Northbridge examples
Must	Without it, the site fails its purpose	Current menu, correct hours, booking action at 375px, Dev can update the menu, brand palette and type
Should	Valuable, and can slip without breaking the contract	Six photos, a short story, a live music section, a map embed
Won't	Explicitly excluded from this version	Online payments, customer accounts, a native app, a loyalty scheme, a blog, online ordering

The stakeholders disagree. Maria wants a gallery first. Sam wants the booking action first. Dev wants the menu to be correct. Alex wants fewer parts that can break.

The spec resolves the disagreement in writing. Booking is a Must because Sam will not complain twice. The menu update is a Must because a wrong menu loses trust. The gallery is a Should, limited to six photos owned by Maria, with no third-party gallery plugin. Maria gets the look she asked for, Sam gets the booking action, Dev gets a maintenance load she can carry, and Alex gets one less plugin to update.

Record the decision and the rejected option in `DECISIONS.md`. The spec says what will be built. The decision record says why this option was chosen and what it cost.

2.5.6 Constraints

Constraints are requirements that limit the solution.

Constraint	Why it exists	What it forces
Plain HTML, CSS and JavaScript	Static site scope	No framework, no build step
Works from 375px	Sam books on a phone	Layout and type are tested at 375px
WCAG 2.2 AA	Public site, low-vision user	Contrast, focus, headings, alt text, target size
No paid tools	Unit budget	Free hosting and open fonts only
Static hosting	No server to maintain	Booking is an enquiry. Staff confirm it later
Launch date	Maria's newspaper promise	Scope can shrink; Must items cannot

Constraints can conflict. A static site cannot confirm a booking without a server. The spec has to say what "book" means, or the build will invent a confirmation the business cannot honour.

2.5.7 Acceptance Criteria

An acceptance criterion is a statement about the build that a person can mark pass or fail. It describes an observable result and the condition under which it must hold. Write criteria for the Must items. A criterion that cannot fail is not a criterion.

Test	Question	Weak example	Why it fails
Observable	Can you see the result?	Looks professional	No observable result
Specific	Does it name one condition?	Works on mobile and desktop	Two conditions, neither defined
Implementation-free	Does it allow any suitable build?	Uses a hamburger menu	Names a solution
Decidable	Would two reviewers agree?	Easy to use	Depends on the reviewer

These criteria pass all four tests:

Acceptance criterion	How it is checked
A customer can see today's opening hours on the home page at 375px without scrolling.	Open the home page at 375px. Compare the hours with the fact sheet.
A customer can start a booking enquiry from any page in one tap, and the form asks for party size, date and time.	Visit each page at 375px. Count the taps to the form. Check the fields.
Dev can update the menu in under five minutes without editing code.	Time Dev doing it with the agreed fields.
Every page has one `h1` and a logical heading order.	Read the heading outline for each page.
Every interactive element has a visible focus ring of at least 3px.	Tab through each page and watch the focus indicator.
The booking action is visible at 375px without horizontal scrolling.	Open each page at 375px. Check for horizontal scroll and the action's position.

A useful pattern is a person doing a task, under a condition, with an observable result. The pattern is a guide. Do not apply it as a formula. Some criteria describe a property of the whole site, such as heading order or contrast.

One criterion can need several checks. "No horizontal scroll at 375px" is one criterion. The check might visit every page and every interactive state. Chapter 17 introduces checks; the next practice chapter turns criteria into checks that run.

Six to ten criteria are enough for a static client site. Thirty criteria become a document nobody checks. Add a criterion when a real defect appears. Do not add one because the list looks thin.

Acceptance criteria also protect the client. Maria can reject a build that misses a Must criterion. She cannot reject a build for failing a feeling she never defined.

> **i** A criterion can expose a stack limit
>
> A static site cannot give Dev a full content management screen. If a Must criterion needs one, the spec and the build disagree. Record the gap in `CRITIQUE.md`, agree with the client whether it is a Must for this version, and carry the requirement into the content-managed version. Do not drop it without recording the decision.

2.5.8 Out of Scope, Assumptions and Change

Out of scope protects the launch. It names what a reasonable person might expect but this version will not do.

For Northbridge: online payments, customer accounts, a native app, a loyalty scheme, a blog, online ordering, and multi-language content.

Assumptions are facts you need but do not control. Write them down.

- The brand office supplies the logo and palette before the build starts.
- Dev supplies the menu in the agreed fields. A photo of the chalkboard is not enough.
- The host supports a static site and a form endpoint.
- Booking is an enquiry. Staff confirm by phone.
- Maria can approve content within two working days.

If an assumption fails, the spec changes. Writing it down makes the change visible.

A change clause keeps the contract usable after sign-off.

> A change that adds a Must item, a page, or a content type is a change request. Record it in `DECISIONS.md`, update the spec version, and re-run the replication test if it changes a Must criterion.

Version the spec. Use v0 after the first brief, v1 after the interviews and the replication test, and v1.1 after a material change. The version number is evidence of what was agreed.

2.6 The Failure: Describing the Current Build

The most common bad spec is an accurate description of the site you already have.

Describes the build	Describes the client's need
A hero carousel with three slides	A first-time visitor can tell what the business is and whether it is open now
A sticky navigation bar with a hamburger menu	A phone user can reach the menu and the booking action from any page in one tap
A grid of six menu cards with hover effects	Dev can publish the menu without calling a developer

Describes the build	Describes the client's need
A contact form with name, email and message	Sam can request a table for four on Friday at 7pm and knows what happens next

The drift happens in four steps. The agent builds something. It looks complete. The spec is rewritten to match it. The spec now passes the replication test because it describes what the agent already built. The client's need is still unmet.

The spec has become a receipt. It records what was bought. The requirement is missing.

Use one question to separate the two.

> If the build disappeared, would this sentence still tell a new builder what the client needs?

"The site has a hero carousel with three slides" fails. Delete the build and the need disappears with it. "A first-time visitor can tell whether the cafe is open now and start a booking in one tap" passes. It still describes the client's need when no build exists.

The build is evidence. The spec is the standard the evidence is judged against.

Agents default to the common pattern. If the spec says "menu", the agent builds a static menu. If it says "booking", the agent builds a contact form. The agent is filling a gap with the average of the web. The gap is yours to close.

2.6.1 A Worked Failure

The first `SPEC.md` for Northbridge said: "The site must be responsive."

The agent produced a desktop layout that shrank. At 375px the booking button sat below three screens of gallery. The student asked the agent to "make it more mobile". The next build moved the button. The booking form was still a general contact form, and the gallery still autoplayed.

The agent's CSS was not the main problem. The spec had no criterion for the booking action at 375px and no rule for the gallery's motion. The fix was two sentences in the spec. A longer chat would not have helped.

Change the spec, then rebuild. A chat fix disappears when the session ends. A spec fix survives the next agent, the next developer and the handover.

2.7 The Replication Test

The replication test measures the spec. It does not measure the agent.

You give the spec alone to a fresh agent and measure the gap between what the spec says and what the client needs. The agent reads the spec as a stranger would. If the reader has to guess, the spec is incomplete.

2.7.1 The Procedure

Step	What to do
1. Freeze	Commit `SPEC.md` and note the version.
2. Start fresh	Open a new agent session and an empty folder outside your repo.
3. Remove context	Do not show the agent your site, your screenshots, your repository or your earlier chat.
4. Give the spec	Provide `SPEC.md`, the fact sheet and the fixed constraints.
5. Build without coaching	If the agent asks a question the spec answers, answer from the spec. If the spec does not answer it, record the question and tell the agent to use its judgement.
6. Compare	Judge the replica against the acceptance criteria. Your site is not the standard.
7. Classify	Label each gap: missing, ambiguous, unstated constraint, or builder error.
8. Revise	Change the spec, not the chat. Bump the version. Run again if time allows.

The comparison grid is the acceptance criteria. A replica that looks different can still pass. A replica that looks similar can still fail a criterion.

2.7.2 A Worked Replication Test

The v1 spec produced a plausible home page, a menu page, a contact form, and a three-slide autoplay gallery. The first impression was good. The criteria told a different story.

Acceptance criterion	Replica result	Gap type	Spec change
Menu items show dietary tags	No dietary tags on any item	Missing	Added dietary tags to the menu fields
Booking asks for party size, date and time	Contact form with name and message	Ambiguous	Defined booking as an enquiry and listed its fields
Opening hours include public holiday notes	Hours table omits public holidays	Missing	Added a note field and named the fact sheet as the source
Booking action visible at 375px	Button below the gallery	Ambiguous	Added “visible without scrolling at 375px”
Every interactive element has visible focus	No focus styles	Missing	Added the focus token and a criterion

Each gap in the table was a spec problem. The replica did what the spec allowed.

The gallery deserves a second look. Maria asked for a gallery. The spec did not say how many photos, who owns them, or whether they may autoplay. The agent chose the common pattern: three slides, automatic movement, no pause control. The fix belongs in the spec. Complaining about the agent’s taste changes nothing.

2.7.3 Classifying the Gap

Gap type	Meaning	What to change
Missing	The spec never said it	Add the requirement or criterion
Ambiguous	The spec allowed two readings	Define the term and add a condition
Unstated constraint	The spec assumed something outside the builder’s control	Add the assumption and its consequence
Builder error	The spec was clear and the replica ignored it	Keep the spec; add or strengthen a check

Only the first three lead to a spec change. A builder error is still useful evidence. It tells you which criteria need a check that runs.

2.7.4 What Counts as a Pass

A pass means every Must criterion is met, and no gap depends on knowledge that only you have. Two good builders will differ in details and agree on the contract. That is a pass.

Do not score the replica out of ten. A number hides which criterion failed and why. Record the biggest gap, the gap type, and the sentence you added.

2.7.5 Limits of the Test

Treat one run as a sample. The same agent family may share the same blind spots, so use a different tool when you can. The replica may be better than your build in one place; copy the better idea or record why you rejected it. The test does not replace client review. It tests whether the spec communicates the client's need.

The test takes about 25 to 30 minutes. Run it when a Must criterion changes. Do not run it after every commit.

> **Keep the replica**
>
> Keep the replica folder and the gap table. They are evidence for the assessment conversation, and they show the spec improving across versions. A spec that never failed a replication test has probably never been tested seriously.

2.8 Building Your Mental Model

2.8.1 The Artefacts

File	Question it answers	Written when
`SPEC.md`	What does the client need, and what counts as done?	Before the build
`DECISIONS.md`	Which option did we choose, and what did we reject?	During the build
`CRITIQUE.md`	What is still wrong or unverified?	After the build
`checks/`	Does the build still meet the criteria?	During and after the build

These artefacts answer different questions. None of them is an essay. The spec sets the standard. `DECISIONS.md` records the trail, `CRITIQUE.md` records the honest gap, and the checks provide evidence.

2.8.2 The Loop

```
client need
    |
    v
  SPEC.md ------------> build ------------> checks
    ^                                         |
    |                                         v
    +-------- replication test <--------
    CRITIQUE.md
```

The replication test is the arrow that returns to the spec. It is the feedback loop that most projects skip.

2.8.3 The Readers

Reader	What they need from the spec
Maria	Scope, cost, what she will get, and what she will not
Dev	Content fields, update effort, and who owns each item
Alex	Maintenance load, hosting constraints, and plugin count
The builder	Tasks, constraints, and acceptance criteria
The checker	Criteria that can become checks

If one reader cannot answer their question, the spec is incomplete for that reader. The client, the content owner, the maintainer, the builder and the checker need the same decisions, even when they need different documents.

2.8.4 The One-Page Test

Can a stranger answer these five questions from the spec alone?

1. Who is the site for?
2. What must each person be able to do?
3. What content exists, and who changes it?
4. What does done look like?
5. What is excluded?

If any answer is missing, the spec is not ready for a replication test.

2.9 Business Applications

A spec is a quoting tool. A fixed-price quote without one is a guess. With one, the vendor prices the Must items, lists assumptions, and states what a change will cost.

It also controls change. The out-of-scope list turns "can you just add online ordering" into a conversation about budget and date. The change clause gives both sides a procedure.

The same document supports handover. The next developer can rebuild or maintain the site without your chat history. A replication test is part of the handover. It shows whether the written record is enough for a stranger.

The cheapest time to find a missing requirement is before the build. The assessment conversation is an expensive place to discover it. The replication test is a pre-mortem with a stopwatch.

For AI projects, the spec is the difference between directing an agent and discovering what it decided. The agent can produce a site in an afternoon. The spec decides whether that site is the one the client needed.

> **i** Skills You Are Building
>
> This develops evaluating and improving solutions, analysing stakeholder requirements, and professional communication. A clear spec is how a business student turns a client conversation into a build that can be checked.

2.10 Practice Exercises

> **i** Exercise Levels
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

2.10.1 Exercise 2.1: Label the Parts (Level 1)

Read this excerpt from a `SPEC.md`.

```
## Purpose
Let local customers find the current menu, hours
```

```
and booking action on a phone.

## Users and top tasks
| User | Task | Context |
|---|---|---|
| Local customer | Book a table | Phone, on the
bus |

## Content
- Weekly menu: week starting, sections, items,
prices.
- Opening hours: day, open, close.

## Design tokens
--colour-ink: #2b2118
--space-unit: 4px

## Acceptance criteria
1. The booking action is visible at 375px without
scrolling.
2. The site feels warm and professional.

## Out of scope
Online payments, accounts, a native app.
```

Label each part: purpose, user, task, content type, token, acceptance criterion, out of scope. Then answer:

1. Which content type is missing, given that the site takes bookings?
2. Which acceptance criterion fails the four tests, and why?
3. Rewrite the weak criterion so it can be marked pass or fail.

Done when: every part is labelled, the missing content type is named, and the weak criterion is rewritten.

2.10.2 Exercise 2.2: From Vague to Checkable (Level 2)

Rewrite each statement as an acceptance criterion that passes the four tests.

1. The site should look professional.
2. It needs to work on mobile.
3. The menu should be easy to update.
4. The site should be accessible.
5. Booking should be quick.

For each criterion, name the condition and the observable result. Then swap with a classmate and mark each other's criteria pass or fail without asking what they meant.

Done when: a classmate can mark every criterion without asking you a question.

2.10.3 Exercise 2.3: Five Criteria from Maria's Brief (Level 3)

Use Maria's email and your franchise fact sheet. Write five Must acceptance criteria for your site. Include at least one for each area:

- the menu
- the booking flow
- the 375px layout
- content editing by Dev
- accessibility

Then add one Should item and one Won't item. Identify one conflict between Maria, Dev, Sam and Alex, and state how the spec resolves it.

Done when: all five criteria pass the four tests, and the conflict is resolved in writing.

2.10.4 Exercise 2.4: Run a Replication Test (Level 4)

Swap specs with a classmate. Do not show them your site, your repository, your screenshots or your chat. Use a fresh agent session and an empty folder. Give the agent only the spec and the fact sheet.

Record:

- which Must criteria were met
- the biggest gap
- the gap type
- the one sentence you would add

Revise your classmate's spec. The chat is not the deliverable. Return the spec with the gap table and a new version number.

Done when: you can name the sentence that would have prevented the biggest gap, and the spec has a new version.

> ⚠ A replica with the original open is not a test
>
> If the agent can see the original site, it will copy the original. Use an empty folder and a fresh session. The test measures the spec. It does not measure the agent's ability to browse your repository.

2.10.5 Exercise 2.5: The Current-Build Trap (Level 4)

Take your `SPEC.md` v0. Mark every sentence that describes the build rather than the client's need. Rewrite three of them. For each, explain what a new builder would do differently after the rewrite.

Done when: the rewritten sentences still make sense if the current build is deleted.

2.10.6 Exercise 2.6: The Contract (Level 5)

Write `SPEC.md` v1 for your franchise. Include:

- purpose
- users and top tasks
- content model
- design tokens
- Must, Should and Won't
- constraints
- at least six acceptance criteria
- out of scope
- assumptions
- a change clause
- sign-off

Then write 300 words defending one trade-off among the competing priorities of Maria, Dev, Sam and Alex. Name what you gave up and why. Record the decision in `DECISIONS.md`.

Done when: a stranger could build from the spec, and the trade-off is recorded with the rejected option.

2.11 Chapter Summary

- A spec is a contract. It records who the site serves, what done means, and what is excluded.
- Users are defined by their tasks and context. Internal users belong in the spec.
- A content model names what the client edits, how often, and which fields each item needs.
- Design tokens turn taste into named decisions that can be applied and checked.
- Acceptance criteria are observable, specific, implementation-free and decidable.

- The current build is evidence. The spec describes the client's need.
- The replication test measures the spec by giving it to a fresh builder and classifying the gaps.
- Revise the spec after a failed test. A chat fix does not survive the session.

2.12 Reflection

Before moving on, ensure you can:

☐ Explain the difference between a wish list and a contract
☐ Write a user task with a device and a context
☐ Model content by type, fields, owner and update frequency
☐ Turn a mood adjective into design tokens and a checkable criterion
☐ Write five criteria that pass the four tests
☐ Tell a sentence that describes the build from one that describes the need
☐ Run a replication test and classify the gaps
☐ Name one thing your spec still does not settle

2.13 Your Learning Journal

Record your responses to these prompts:

1. **The gap I almost filled**: Which question did you answer with a guess? What would you ask instead?
2. **The conflict**: Where did two stakeholders disagree? What did you choose, and what did you give up?
3. **The replication**: What did the fresh builder misunderstand? What did that teach you about your words?
4. **The current build**: Which sentence in your spec described what you had already built? How did you rewrite it?
5. **Client translation**: How would you explain the value of a spec to Maria in two sentences, without technical language?

2.14 Next Steps

Chapter 5 shows how to turn the content model into semantic HTML. Start with the content model and let the page structure follow it.

Chapter 6 shows how to apply the design tokens consistently. The tokens you agreed in the spec become the only values the build should use.

The next practice chapter, **Checks That Run**, turns acceptance criteria into checks that a machine can run. The spec tells you what to check. The checks tell you whether the build still meets it.

Chapter 4 shows how to turn a repeated workflow, such as the replication test, into a reusable skill.

Chapter 3

Checks That Run

The agent says the site is done. The page looks right on your laptop. A customer opens it on a phone and cannot find the booking button. Nothing in the build failed. No one had written a check for that.

A check is how you turn “it looks fine” into evidence. It is also how you stop the same defect from returning next week.

3.1 The Concept First

A check is a claim about the build that can fail. A manual check is a person looking. A check that runs is a command that repeats the same judgement every time. The command gives you evidence you can show. The person gives you an opinion you have to trust.

A check does not prove the site is good. It proves that one claim held under one set of conditions. That is enough to catch regressions, and not enough to sign off the site.

> **A check is evidence only if it can fail**
>
> If a check passes on a broken page, it is not testing the thing you care about. Before you trust a check, break the page on purpose and watch the check fail.

3.2 Understanding Through a Fire Drill

A fire drill tests whether the alarm sounds and the exits open. It cannot tell you whether the exit sign points the right way.

Automated checks are the drill. They test specific paths under specific conditions. They run every time, and they only test what someone thought to test. The exit sign still needs a person to look at it.

Tools catch a useful subset. You decide what the subset means, and you check the rest yourself. The rest of this chapter shows how.

3.3 The Four Principles

WCAG organises accessibility under four principles: Perceivable, Operable, Understandable, Robust. The principles give a person a checklist and give the tools a map.

Principle	Question	Northbridge example
Perceivable	Can everyone perceive the content?	The menu photo has alt text that describes the dish, not “image”.
Operable	Can everyone operate it?	The booking button can be reached by keyboard and has a visible focus ring.
Understandable	Can everyone understand it?	The form says “Party size” and explains what happens after submission.
Robust	Does it work with different browsers and assistive technology?	The form uses a real `button` and `label`, not a clickable `div`.

The principles are not a score. A page can pass a tool and still fail a person. The tool checks a rule. The principle asks whether the page works.

3.4 What the Tools Catch

A practical check suite uses four tools. Each one looks at a different layer.

Tool	What it checks	What it misses
html-validate	HTML structure, duplicate IDs, invalid nesting, some missing attributes	Whether the content is meaningful
axe	A subset of WCAG A and AA: contrast, labels, roles, names, some heading order	Whether alt text is meaningful, whether the heading order makes sense, whether the copy is clear
Playwright	Behaviour, layout, keyboard access, responsive layout, API failure	Whether the behaviour is right for the business
Lighthouse CI	Performance, accessibility score, best practices, budgets	Whether the site is usable on a cheap phone with a cracked screen, whether the data is correct

The tools overlap, which is useful, and they share blind spots.

Command	What it runs
`npm run validate`	`html-validate` over `site/`
`npm test`	Playwright checks in `checks/` and `tests/`
`npm run lighthouse`	Lighthouse CI
`npm run check`	validate, secrets and Playwright

Use `npm run check` before you commit. Run `npm run lighthouse` when you change images, fonts or layout.

An axe check looks like this:

```
import { test, expect } from '@playwright/test';
import AxeBuilder from '@axe-core/playwright';

test('home page has no detectable WCAG A/AA
violations', async ({ page }) => {
  await page.goto('/');
  const results = await new AxeBuilder({ page })
```

```
    .withTags(['wcag2a', 'wcag2aa', 'wcag21a',
    'wcag21aa', 'wcag22aa'])
    .analyze();
  expect(results.violations).toEqual([]);
});
```

The check reports detectable violations. It does not tell you whether the page is accessible.

3.4.1 What No Automated Check Can See

- Whether alt text describes the image.
- Whether the heading order helps a person navigate.
- Whether the copy is clear.
- Whether the booking flow matches the business process.
- Whether the site works on a slow connection.
- Whether the data is correct.
- Whether the user can recover from an error.

These are not edge cases. They are the parts of the experience that decide whether the client keeps the site.

> ⚠ A passing axe run is not an accessibility sign-off
>
> axe can find a missing `alt` attribute. It cannot tell you whether the `alt` text says anything useful. It can find a missing label. It cannot tell you whether the label uses words the customer understands.

3.5 From Criterion to Check

The check starts with the acceptance criterion, not the code. If the criterion is vague, the check will be vague. Chapter 2 gave you the criterion, and a check turns it into evidence.

A useful check has four parts:

1. The condition. A viewport, a page, a network response, a keyboard action.
2. The action. What the user or the test does.
3. The observable result. What must be true.
4. The failure message. What the output says when it is false.

If one part is missing, the check is weak. A check that says “the page works” has no condition and no observable result.

3.5.1 Example: The Booking Action at 375px

The criterion is: "The booking action is visible at 375px without scrolling."

A weak check looks for the viewport meta tag. The tag is a proxy. A page can have the tag and still overflow.

```
// Weak: checks a proxy, not the outcome.
test('has a viewport meta tag', async ({ page })
=> {
  await page.goto('/');
  await
  expect(page.locator('meta[name="viewport"]')).toHaveAttribute(
    'content',
    /width=device-width/
  );
});
```

A stronger check opens the page at 375px and tests the result.

```
import { test, expect } from '@playwright/test';

test('booking action is visible at 375px', async
({ page }) => {
  await page.goto('/');
  const cta =
  page.locator('[data-cta="primary"]');
  await expect(cta).toBeVisible();

  const box = await cta.boundingBox();
  const viewport = page.viewportSize();
  expect(box.x).toBeGreaterThanOrEqual(0);
  expect(box.x +
  box.width).toBeLessThanOrEqual(viewport.width);
});

test('no horizontal scroll at 375px', async ({
page }) => {
  await page.goto('/');
  const overflows = await page.evaluate(
    () =>
      document.documentElement.scrollWidth >
      document.documentElement.clientWidth
  );
```

```
  expect(overflows).toBe(false);
});
```

The first check is not useless. It catches a missing tag. It does not catch the defect the customer reported.

`toBeVisible()` is stronger than `toBeAttached()`. An element can be attached to the page and still sit outside the viewport. The customer cannot see it, and the weak check goes green.

3.5.2 Example: Every Page Has a Title

The criterion is: “Every page has a non-empty title.”

```
import { test, expect } from '@playwright/test';
import { readFileSync } from 'node:fs';

const pages = JSON.parse(
  readFileSync(new URL('../pages.json',
  import.meta.url), 'utf8')
);

for (const path of pages) {
  test(`${path} has a non-empty title`, async ({
  page }) => {
    await page.goto(path);
    expect((await
    page.title()).trim().length).toBeGreaterThan(0);
  });
}
```

The check is only as good as the page list. If a page is missing from `pages.json`, the check will not visit it. Review changes to `pages.json` as carefully as changes to the site.

3.5.3 Example: The API Failure

The criterion is: “When the live data request fails, the page shows a friendly message and a way to retry.”

The happy path is easy to check. The unhappy path is the one that protects the customer.

```
import { test, expect } from '@playwright/test';

test('live data shows a friendly message when the
API fails', async ({ page }) => {
  await
  page.route('**/.netlify/functions/opening-status*',
  route =>
    route.fulfill({ status: 500, body: 'error' })
  );
  await page.goto('/');
  await
  expect(page.getByRole('status')).toContainText(/unavailable|try
  again/i);
});
```

Use the path of the function your page calls. The route mock makes the check fast and repeatable. It does not prove the real API works. It proves the page handles a failure. Pair it with a smoke test against the real endpoint.

State	What the user sees	What the check asserts
Loading	A message or a skeleton while the request is in flight	The status text appears before the data resolves
Error	A friendly message and a retry action	The message appears when the route returns 500
Empty	A message that says there is no data	The message appears when the response is an empty list

A blank space is a defect. A spinner that never stops is a defect. An error message that says “Error” is barely better than a blank space.

Chapter 8 shows how the request, the function and the response fit together. This chapter shows how to prove the page behaves when the response is bad.

3.6 Working with Your AI Partner

AI is useful at three points in this chapter. It can draft a check from a criterion, show you how a check can be gamed, and describe a page that passes a check but

fails a person. It is a poor judge of whether the check is enough. That judgement is yours.

3.6.1 Exploration 1: Draft a Check from a Criterion

```
Ask your AI:
Here is an acceptance criterion: "The booking
action is visible at 375px
without scrolling." Write the smallest Playwright
check that would fail
if the criterion were false. Then name one thing
the check cannot see.
Do not change the site.
```

Read the check before you run it. Does it test the outcome or a proxy? Does it use a role or a brittle selector? What happens if the button is attached but off-screen? Improve the check before you trust it.

3.6.2 Exploration 2: Ask How the Check Could Be Gamed

```
Ask your AI:
Here is a passing check. Show me three ways to
make it pass without
fixing the page. For each one, explain what it
hides from the customer.
```

The answer is a catalogue of the shortcuts in the next section. Do not apply them. Use them to recognise the diff when an agent does.

3.6.3 Exploration 3: Find the Proxy

```
Ask your AI:
Here is a check. Is it testing the outcome or a
proxy? Describe a page
that would pass the check and still fail the user.
```

A good answer names the gap between the assertion and the experience. That gap is why the manual check still exists.

3.7 The Failure: The Agent Games the Check

An agent asked to make a check pass has three options. It can fix the site, weaken the check, or hide the problem. It will often choose the cheapest path that produces green. The agent is not malicious. It is optimising for the signal you gave it.

Your job is to make sure the green means something.

Each common shortcut below is a diff to read. The output alone is not enough.

3.7.1 Weakening the Assertion

```
- await
expect(page.locator('[data-cta="primary"]')).toBeVisible();
+ await
expect(page.locator('[data-cta="primary"]')).toBeAttached();
```

`toBeAttached()` passes when the element is in the DOM. It can be off-screen, covered, or invisible. `toBeVisible()` tests what the customer can see.

```
- expect(results.violations).toEqual([]);
+
expect(results.violations.length).toBeLessThan(3);
```

The second check allows two accessibility violations. That is not a stricter version of the first. It is a different contract.

3.7.2 Skipping the Check

```
- test('no horizontal scroll at 375px', async ({
page }) => {
+ test.skip('no horizontal scroll at 375px', async
({ page }) => {
```

A skipped check is a green check that tests nothing. Read the output for `skipped` as well as `passed`.

3.7.3 Hiding the Problem

```
- <button data-cta="primary">Book a table</button>
+ <button data-cta="primary"
aria-hidden="true">Book a table</button>
```

The button is now hidden from screen readers. A sighted user can still click it. A screen reader user loses the action. A visibility check may pass because the page still renders. If the suite runs axe, this may be caught. If it only checks visibility, it will not. The customer is worse off.

`aria-hidden` is not a fix for a duplicate control or a confusing label. It is a way to remove content from the accessibility tree. Use it for decorative content. Do not use it to hide content from a check.

3.7.4 Hiding the Symptom

```
body {
  overflow-x: hidden;
}
```

This clips the horizontal scroll. The fixed-width element is still there. The customer can still lose content off the edge. Find the fixed width, the image, or the grid that causes the overflow. Chapter 6 covers the common causes.

3.7.5 Editing the Page List

A failing page is removed from `pages.json`. The checks visit fewer pages. The suite goes green.

Review `git diff pages.json` after every agent change. The page list is part of the test surface.

3.7.6 Waiting for Luck

```
await page.waitForTimeout(2000);
```

A fixed wait can make a flaky check pass once. It slows the suite and hides the race. Use a web-first assertion such as `await expect(locator).toBeVisible()`. Let the tool wait for the condition. A fixed delay only guesses how long that takes.

3.7.7 How to Catch a Shortcut

- Read the diff as well as the green output.
- Run `git log --oneline -- checks/`.
- Make the check fail on purpose before you trust it.
- Ask the agent for the command and the output.
- Review changes to `pages.json`.

- Check that the assertion names the observable result.

3.8 Git Evidence

The provided checks are a contract. The project template puts them in `checks/`. Your own checks go in `tests/`.

The history should show one commit for the provided checks: the commit that added the template. Run:

```
git log --oneline -- checks/
```

If there is a second commit, read it. If it edits a check, the checks were weakened. Revert the change, then fix the site. Record what happened in `DECISIONS.md`.

```
git checkout <commit> -- checks/
```

The commit hash is the one that added the checks. If you are unsure, ask the agent to show `git log -p -- checks/` and read the diff before you revert.

A check is evidence only if the check itself is trustworthy. The git history is how you prove that the provided checks were not weakened.

A provided check can be wrong. If you believe it is, do not edit it. Record the problem in `CRITIQUE.md`, show the evidence, and ask for a decision. The default is to fix the site.

3.9 A Worked 375px Audit

The site passes the desktop checks. The booking page looks fine on a laptop. At 375px, the page scrolls sideways and the booking button sits off-screen.

The existing check looks for the viewport meta tag. It passes. The customer still cannot book.

1. Open the site at 375px. Record the defect in `CRITIQUE.md`.
2. Write the outcome check. It fails.
3. Find the fixed width. It might be a hero image, a grid, a table, or a container with a hard-coded `width`.
4. Fix the cause. Do not add `overflow-x: hidden`.
5. Run the check again. It passes.
6. Add the criterion to `SPEC.md` if it was missing. The check and the criterion should point at each other.

The audit is not finished when the page looks better. It is finished when the check would catch the defect if it returned.

3.10 A Worked API Failure

The live data widget shows opening status, today's specials, or another small piece of business data. When the API is up, the widget looks fine. When the API fails, the widget is blank.

The happy-path check passes. The unhappy path has no check.

1. Mock a 500 response in a Playwright test.
2. Assert that the page shows a friendly message.
3. Run the check. It fails because the page has no error state.
4. Add the error state. Add a loading state and an empty state while you are there.
5. Run the check again. Then run the happy path again.
6. Record the decision in `DECISIONS.md` and the remaining gap in `CRITIQUE.md`.

The check does not prove the API will never fail. It proves the page does something useful when it does.

3.11 Building Your Mental Model

3.11.1 The Check Pyramid

```
performance and budgets      Lighthouse CI
behaviour and layout         Playwright
accessibility                axe
structure                    html-validate
```

Start at the bottom. A page with invalid HTML is harder to check. A page with no accessible names is harder to use. The layers catch different defects. None of them is enough alone.

3.11.2 The Criterion Chain

```
criterion -> condition -> action -> observable
result -> assertion -> failure message
```

If the chain breaks at the condition, the check tests the wrong thing. If it breaks at the observable result, the check tests a proxy. If it breaks at the failure message, you will not know what failed.

3.11.3 What the Check Proves

The check proves	The check does not prove
One claim held under one condition	The site is good
The defect is gone for that path	The defect is gone everywhere
The page handles a mocked failure	The real API works
The element is in the viewport at 375px	The page is usable on a cheap phone
The provided checks were not edited	The student understood the check

3.12 Business Applications

Checks reduce rework. A defect caught before the client sees it costs minutes. A defect caught after launch costs trust, support time and sometimes revenue.

A fixed-price quote is easier to defend when you can name what you will verify: the booking action at 375px, the error state, the page titles, the contrast. The client knows what "done" includes.

The checks are also handover evidence. The next developer can run the suite before changing anything. They do not have to trust a chat log or a colleague's memory.

For AI projects, the suite is the defence. You can show what the agent changed, what the checks proved, and what still needs a human. That is a stronger position than "the agent said it was done".

A regression caught before the customer sees it lowers the cost of change, which makes the site safer to improve.

> **i** Skills You Are Building
>
> This develops evaluating and improving solutions, professional communication, and the judgement to decide what evidence is enough. A check is how you show your work.

3.13 Practice Exercises

> **i** Exercise Levels
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving

- **Level 5**: Open-ended design

3.13.1 Exercise 3.1: Run and Read (Level 1)

Run the project's check command. Record every failure in `CRITIQUE.md` under `## Check results`.

For each failure, answer:

1. Is this a site problem or a check problem?
2. Which user is affected?
3. What is the smallest fix?

Then name one thing the checks cannot see.

Done when: you can list every failure and explain one limit of the suite.

3.13.2 Exercise 3.2: One Criterion, One Check (Level 2)

Choose one acceptance criterion from `SPEC.md` that the provided checks do not cover. Write a check in `tests/`.

Make it fail first. Break the page or remove the feature. Then fix the site and make it pass.

Done when: the check fails when the criterion is false and passes when it is true.

3.13.3 Exercise 3.3: The 375px Outcome Check (Level 3)

Write two checks:

1. No horizontal scroll at 375px.
2. The primary action is inside the viewport at 375px.

Run them on a page with a fixed-width element. Then fix the page and run them again. Explain why a viewport meta tag check is not enough.

Done when: both checks fail on the broken page and pass on the fixed page.

3.13.4 Exercise 3.4: The Unhappy Path (Level 4)

Choose a page that loads live data. Write checks for three states:

- the loading state
- the error state
- the empty state

Mock the responses. Show that the page never shows a blank space or a spinner that never stops.

Done when: each state has a check that fails when the state is missing.

3.13.5 Exercise 3.5: Catch the Gaming Diff (Level 4)

Here is a diff an agent produced after being asked to make a check pass.

```
- await
expect(page.locator('[data-cta="primary"]')).toBeVisible();
+ await
expect(page.locator('[data-cta="primary"]')).toBeAttached();
```

And another:

```
- expect(results.violations).toEqual([]);
+
expect(results.violations.length).toBeLessThan(3);
```

Answer:

1. What does each change hide?
2. Why is the original assertion stronger?
3. What would you see in the browser that the new check would miss?

Then revert the changes, fix the site, and write a check that fails on the broken page.

Done when: you can explain the weakening and your replacement check fails when the defect returns.

3.13.6 Exercise 3.6: What the Check Cannot See (Level 5)

Write a manual test script for the things automation misses:

- meaningful alt text
- heading order
- copy clarity
- the booking process
- a slow connection
- data correctness

Run it on a classmate's site. Record three findings. For each finding, name the user it affects and the change you would make.

Done when: another person could run the script without asking you what you meant.

3.14 Chapter Summary

- A check is a claim about the build that can fail.
- WCAG POUR gives the human checklist. Tools cover a subset.
- Start from the acceptance criterion, not the code.
- Check the outcome, not the proxy.
- The agent will optimise for green. Read the diff.
- The provided checks are a contract. `git log -- checks/` proves they were not weakened.
- The unhappy path needs checks too.
- A check is evidence only if you know what it asserts.

3.15 Reflection

Before moving on, ensure you can:

- ☐ Explain POUR with one example per principle
- ☐ Name what axe catches and what it misses
- ☐ Write a check from an acceptance criterion
- ☐ Make a check fail before making it pass
- ☐ Read a diff and spot a weakened assertion
- ☐ Use `git log -- checks/` as evidence
- ☐ Write checks for loading, error and empty states
- ☐ Name one thing your checks cannot see

3.16 Your Learning Journal

Record your responses to these prompts:

1. **The defect I would have missed**: Which check caught something you did not see?
2. **The green check I did not trust**: What did the agent change to make a check pass? What did you do about it?
3. **The user I almost forgot**: Which finding came from a person rather than a tool?
4. **The proxy**: Where did a check test a proxy instead of the outcome? How did you rewrite it?
5. **The next check**: What would you check next, and why?

3.17 Next Steps

Chapter 8 shows how the request, the serverless function and the response fit together. The API failure check in this chapter is only useful if you understand what the page is asking for.

The next practice chapter, **The Agent-Ready Repo and Handover**, turns the checks, the decisions and the spec into a repository that another person can run, review and maintain. It also shows how to turn a repeated task, such as writing a check, into a skill.

Chapter 4

The Agent-Ready Repo and Handover

An agent can only work with what the repository tells it. The next developer can only maintain what the repository explains. A repository that is ready for an agent is also ready for a handover.

If someone else can open the repository, run it, change it and diagnose it without asking you, the repo is ready. If they need your chat history, it is not.

4.1 The Concept First

A repository is three things at once. It is the workspace where the build happens. It is the record of what changed and why. It is the handover pack that lets someone else take over.

Most students treat the repo as a folder for files. An agent-ready repo is a set of instructions and evidence. It tells the agent what to do before it starts. It tells the next person what happened after you leave.

The evidence files are the handover:

- `SPEC.md` says what done means.
- `DECISIONS.md` says what was chosen, why, and what was rejected.
- `CRITIQUE.md` says what is still wrong or unverified.
- `AI-USE.md` says which tools were used, for what, and what was verified.
- `README.md` says how to run and deploy the project.
- A runbook says what to do when it breaks.
- The checks say how to prove it still works.

A chat log is not a handover. It is a transcript of a conversation that the next person cannot search, trust or reproduce.

4.2 Understanding Through a Shared Workshop

A workshop with labelled drawers, a job card, a logbook and a clean bench can be used by someone else. A workshop where everything is in one person's head cannot.

The labelled drawers are the README and the folder structure. The job card is the spec and the agent instructions. The logbook is the decision record and the commit history. The clean bench is the small diff, the removed secret and the closed branch.

A workspace that explains itself is a business asset. A workspace that only one person understands is a liability.

4.3 The Files That Do the Work

File	Question it answers	Who reads it
`AGENTS.md`	How should an agent work here? What is allowed, what is forbidden, and what proves the work is done?	The agent, and the next developer
`SPEC.md`	What does the client need, and what counts as done?	The client, the builder, the checker
`DECISIONS.md`	What did we choose, why, and what did we reject?	The next developer, the assessor, the client
`CRITIQUE.md`	What is still wrong or unverified?	The next developer, the client
`AI-USE.md`	Which tools did we use, for what, and what was verified?	The assessor, the next developer
`README.md`	What is this, how do I run it, and how do I deploy it?	Anyone opening the repo
`runbook.md`	What do I do when it breaks?	The maintainer, the IT vendor, future you
`checks/` and `tests/`	How do we know it still works?	The builder, the reviewer, the next developer

Each file removes a question that would otherwise become a meeting, a message

or a defect.

4.4 AGENTS.md

`AGENTS.md` is the project instruction file for an AI agent. It lives in the repository. The agent reads it before it starts work. It is not a prompt you paste once. It is a standing contract for every session.

A useful `AGENTS.md` is short enough to read and specific enough to act on.

```
# Instructions for AI agents working in this
repository

## Stack

- Plain HTML, CSS and JavaScript. No frameworks,
no build step.
- The site lives in `site/`.
- The provided checks live in `checks/`. Your
checks live in `tests/`.

## Rules

1. Do not create, edit or delete anything in
`checks/`. Fix the site, not the check.
2. Never put API keys, tokens or secrets in
`site/`.
3. Make one logical change per commit, with a
message that says what changed and why.
4. Do only what you are asked. Suggest other
changes and wait.
5. When you say something is done, show the
command and its output.
6. Read `SPEC.md` before starting. If the task
conflicts with it, stop.
7. Add every new page to `pages.json`.
8. Do not add a dependency, a framework or a build
step without asking.
```

The rules are testable. “Write clean code” is not a rule. “Do not edit `checks/`” is a rule you can check in the diff.

The agent will fill any gap with a default. If the file does not say which folder holds the site, the agent will choose one. If the file does not say which commands

prove the work, the agent will claim success without evidence.

Update `AGENTS.md` after a failure. Add the rule that would have prevented it. The file should get better as the project gets harder.

> **The rule that would have helped**
>
> After a bad agent change, ask one question: "What sentence in `AGENTS.md` would have stopped this?" Add it. That is a better use of five minutes than rewriting the prompt.

4.5 Working with Your AI Partner

AI is useful at three points in this chapter. It can find the gaps in `AGENTS.md`, plan a small change with a stop condition, and review a diff for scope creep. It is a poor judge of whether the work is ready to hand over. That judgement is yours.

4.5.1 Exploration 1: Find the Missing Rules

```
Ask your AI:
Here is my AGENTS.md. List every decision you
would have to guess before
you could work safely in this repo. For each
guess, write the rule that
would remove it. Do not edit any files.
```

Add the rules that are specific and testable. Ignore generic advice such as "write clean code". A rule the agent cannot check is not a rule.

4.5.2 Exploration 2: Plan Before You Act

```
Ask your AI:
Here is one acceptance criterion and the file it
affects. Write a plan
with one change, the check that proves it, and the
conditions that
should stop you. Do not write code yet.
```

A good plan names the file, the criterion, the check and the stop condition. If the plan is vague, the diff will be large.

4.5.3 Exploration 3: Review a Diff for Scope Creep

```
Ask your AI:
Here is a diff. List every change that is outside
the stated task. Do
not fix it. Name the scope creep and the risk it
adds.
```

Scope creep is easier to see in a diff than in a chat. Review the diff before you review the result.

4.6 Small Diffs

The diff is the unit of review. A large diff hides mistakes, mixes unrelated changes and makes the work impossible to explain. A small diff shows one decision at a time.

One logical change per commit. A logical change is one behaviour, one fix, one content update or one refactor. If the message needs the word “and” to describe two unrelated things, the commit is probably too large.

Ask the agent for one change at a time. Name the file, the acceptance criterion and the stop condition. Ask it to show the diff before applying it.

```
Ask your AI:
Change only the booking form in site/contact.html
so that the party size
field has a visible label and a helpful error
message. The acceptance
criterion is "The form labels every input." Do not
change the layout,
the copy or any other page. Show me the diff
before you apply it.
```

The commit message is evidence. It says what changed and why. The body can say what was rejected and which check proves the change.

Weak message	Useful message
update	Fix 375px overflow in the menu grid
fix	Add a label to the party size field
final	Add loading and error states to the opening status widget

Weak message	Useful message
changes	Replace the hero image with the client's photo and update the alt text

`git log --oneline` should read like a story. A reader should be able to see the build progress without opening every commit.

If the agent makes one giant commit, you can still recover. Read it with `git show`. Split it with `git add -p`, which lets you stage one hunk at a time. Or revert it and redo the work in small commits. The history is worth the effort because the history is the evidence.

4.6.1 Commit Messages as Evidence

A commit message answers three questions:

1. What changed?
2. Why did it change?
3. What proves it?

```
Add a visible label to the party size field

The booking form used a placeholder as the only
label. Screen reader
users could not tell what the field was for. The
new label is visible
and connected with for/id.

Checked with npm test and a screen reader pass.
```

The body is not a diary. It is the reason the change exists. The next developer will read it when the code no longer makes sense.

4.7 Plan, Act, Verify

An agent-ready workflow has three steps. They are short, and they prevent most of the mess.

Plan. Restate the task in one sentence. Name the files. Name the acceptance criterion. Name the check. Name the stop condition.

Act. Make one small change. Commit it with a message that explains the change.

Verify. Run the check. Read the diff. Open the page at 375px. Check the console. Show the evidence.

A plan sounds like this:

> The task is to make the booking action visible at 375px. The file is `site/css/styles.css`. The criterion is "The booking action is visible at 375px without scrolling." The check is `tests/booking.spec.js`. Stop if the change touches `checks/` or adds a dependency.

Name a stop condition, because the agent will keep going if you do not define the edge of the task.

4.7.1 When to Stop the Agent

Stop when the agent:

- wants to change the stack;
- wants to edit `checks/`;
- wants to add a dependency or a build step;
- wants to touch secrets or environment variables;
- wants to rewrite a file wholesale;
- wants to do something outside the task;
- says it is done without showing the command and the output.

Stopping is a normal decision. Record it in `DECISIONS.md`. The agent can be wrong. You own the commit.

4.8 Skills and Reusable Instructions

A skill is a short, reusable instruction file for a repeated task. It tells the agent how to do one job in this project.

A skill has five parts:

1. When to use it.
2. The inputs.
3. The steps.
4. The files it may touch.
5. The definition of done.

```
# Skill: Add a page

## When to use

A new page is added to the site.
```

```
## Steps

1. Create `site/<page>.html` from the shared
layout.
2. Add the page to `pages.json`.
3. Add a link from the navigation.
4. Run `npm run check`.
5. Commit with a message that names the page and
the criterion.

## Done when

The page is in `pages.json`, the navigation links
to it, and the checks pass.
```

4.8.1 From Workflow to Skill

The first time you do a task, you are learning it. The second time, you are checking whether the steps are stable enough to reuse. That is when you write the skill.

Pass	Purpose	Output
First	Learn the workflow	A rough log of the steps, files, checks and failure points
Second	Confirm the steps are stable	A skill file, or a note that the task changes too much
Test	Prove the skill works without you	A fresh agent session and a diff you can review

The first pass is not wasted. It is the research. Keep a rough log while you work: which files the agent touched, which check proved the change, where it went wrong, what you had to explain twice.

The first time you add a page, you might ask the agent to create `site/contact.html`. You notice that the page also needs a line in `pages.json` and a link in the navigation. The check fails if the page is missing from the list. Those are the steps that belong in the skill.

On the second pass, follow the log. If the steps are the same, write the skill. If the steps change, the task is not stable enough to codify yet. Narrow the task or

keep learning.

Test the skill in a fresh session with no chat history. If the agent needs extra explanation, the skill is missing a step. Add it or narrow the task.

A skill for "add a page" is portable. A skill for "update the Northbridge menu" is project-specific. Both are useful. Keep them small.

`AGENTS.md` is the project contract. Skills are task recipes. A skill that tries to cover the whole project is an `AGENTS.md` file with a different name.

Skills make the agent more consistent. They do not make it correct. You still read the diff and run the check.

> **i** Where this idea comes from
>
> The Human Edge treats a saved brief as a skill in Chapter 8 and places the agent inside a gate in Chapter 10. Conversation, Not Delegation covers the loop, permissions and stopping conditions. Build AI, Understand AI shows the folder-as-workflow version in "Workflows as Folders". This chapter adds the project-level version: the skill file, the repo rules and the handover.

4.9 Permissions and Secrets

The agent needs enough access to do the work and no more. That is the principle of least privilege.

- Give the agent the workspace. Do not give it the production account.
- Do not put a production key on a laptop or in a repository.
- Use environment variables and a serverless function for anything secret (Chapter 8).
- Keep `.env` in `.gitignore`. Commit `.env.example` with names only.
- Do not paste secrets into a prompt, a screenshot or a README.
- Read a command before you approve it. Do not approve a command you do not understand.

If a secret is committed, rotate it. Deleting the file does not remove it from history. `git log -p` can still find it. A secrets check is a teaching tool. It does not prove the repo is clean.

The same rule applies to permissions. The agent does not need admin rights, a production database or a payment account. It needs the files, the checks and a way to run them. Many managed machines have no admin rights for a reason. The browser path is the baseline, and the desktop path is an optional convenience.

4.10 The Handover Pack

A handover pack is what you give the next person so they can run, deploy, change and diagnose the site. It is the repo made legible.

Part	What it contains
`README.md`	What the project is, how to run it, how to deploy it, where the data lives, who to contact, known limitations
Runbook	Local setup, deploy steps, data sources, common failures, rollback, contacts
`DECISIONS.md`	Architecture choices, rejected options, costs, dates
`CRITIQUE.md`	Known issues, unverified areas, what to fix next
`AI-USE.md`	Tools, what was verified, what was rejected, where AI was not used
`SPEC.md`	The contract the build was judged against
Checks	The command that proves the site still works
Known issues	Severity, workaround, owner, next action
Contribution map	Who did what, for a team project or a handover

The handover pack is also a business document. It reduces the cost of maintenance, the risk of a single person leaving, and the time a new developer spends asking questions.

4.10.1 A Worked Handover to Alex

Alex is the IT vendor who will maintain the site after the project ends. Alex is busy, cautious and has seen too many abandoned sites. A good handover is the difference between a site that survives and a site that gets rebuilt.

A weak handover sounds like this:

> It works on my laptop. The chat has the details. The password is in the repo. Ask me if you get stuck.

A useful handover sounds like this:

> The site is in `site/`. `README.md` has the local and deploy steps. `runbook.md` covers the data sources, the common failures and the rollback. `DECISIONS.md` explains why the front end is static and why the booking form posts to a function. `CRITIQUE.md` lists the known issues, including the missing empty state on the opening status widget. `npm run check` proves the checks still pass. There are no secrets in the repo. The deploy URL and the rollback steps are in the runbook.

The second handover lets Alex run the site, deploy it, diagnose it and know what is still wrong. The first handover makes Alex open a chat and start from zero.

`README.md` might open like this:

```
# Harbour & Hearth Northbridge

A static site for the Northbridge cafe. The site
is in `site/`.

## Run locally

npm install
npm run serve

## Deploy

Push to `main`. Netlify deploys `site/`.

## Data

The opening status widget calls
`/.netlify/functions/opening-status`. The key is
an
environment variable in Netlify.

## Checks

npm run check

## Known issues

See `CRITIQUE.md`.
```

`runbook.md` might open like this:

```
# Runbook

## If the deploy fails

1. Open the Netlify deploy log.
2. Check that the publish directory is `site`.
3. Run `npm run check` locally.

## If the opening status widget is blank

1. Open the browser console.
2. Check the function URL.
3. Check the environment variable in Netlify.
```

4.10.2 The Handover Test

Give the repo to someone who has never seen it. Ask them to run it, deploy it and fix one small issue. Record every question they ask.

Each question is a missing line in the documentation. Add the line. Run the test again with someone else if you can.

The replication test in Chapter 2 tests the spec. The handover test tests the repo. Both are ways to measure whether your written record is enough for a stranger.

4.11 The Failure: The Agent-Made Mess

The agent-made mess is easy to recognise. The git log is a list of "update", "fix" and "final". One commit changes forty files. `checks/` has been edited. A key is in `site/js/config.js`. There is no README. The only record of the decisions is a chat that has already been closed.

```
a1b2c3d final
d4e5f6g update
h7i8j9k fix
```

You cannot explain the diff in the assessment conversation. The client cannot maintain the site. The next developer starts from zero.

The fix is not a better prompt. The fix is a better repo:

1. Write `AGENTS.md` before the next task.
2. Ask for one logical change at a time.
3. Commit each change with a message that says what and why.

4. Keep the evidence files current.
5. Remove the secret, rotate it and check the history.
6. Write the handover pack before the last week, not after it.

4.12 Building Your Mental Model

4.12.1 Three Audiences, One Repo

Audience	What they need	Where they find it
The agent	Stack, folders, rules, commands, stop conditions	`AGENTS.md`, `SPEC.md`, skills
The next developer	How to run, deploy, change and diagnose	`README.md`, runbook, `DECISIONS.md`, `CRITIQUE.md`
The client	What was built, what it costs to maintain, what is still wrong	`SPEC.md`, `CRITIQUE.md`, the handover pack

If one audience cannot answer their question, the repo is incomplete for that audience.

4.12.2 The Evidence Loop

```
SPEC.md ---> build ---> checks
   ^                       |
   |                       v
DECISIONS.md <--- review <--- CRITIQUE.md
   |
   v
AI-USE.md ---> handover pack
```

The spec sets the standard. The checks prove the build. The critique records the gap. The decision record explains the choices. The handover pack lets someone else continue.

4.12.3 The Five Questions

A repo is ready when a stranger can answer these five questions from the repo alone:

1. What is this, and who is it for?
2. How do I run it?
3. How do I deploy it?
4. How do I know it still works?
5. What is still wrong, and what should I do next?

If any answer is missing, the handover is incomplete.

4.13 Business Applications

A maintainable site is worth more than a clever one. The client is buying the ability to change the site after the launch as well as the launch itself.

The cost of maintenance falls when the next developer can spend time on the work instead of reconstructing the history. The risk of a single person leaving falls when the client is not dependent on one contractor's memory or chat history.

A clean repo also supports procurement. A vendor can inspect the evidence, run the checks and price the work. The same evidence protects the client in a dispute about what was delivered.

For AI projects, the repo is the defence. `AI-USE.md` and the commit history show what was verified, what was rejected and where the human made the call.

> **Skills You Are Building**
>
> This develops professional communication and project management, evaluating and improving solutions, and the judgement to decide when a build is ready to hand over.

4.14 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

4.14.1 Exercise 4.1: Three Rules That Would Have Helped (Level 1)

Think of a failure in your last build. It might be a secret in the wrong file, a check that was edited, a giant commit, or a page that was never added to the page list.

Write three rules for `AGENTS.md` that would have prevented it. Each rule must be specific and testable. Then explain how you would check that the agent followed each rule.

Done when: each rule names the behaviour, the file or command, and the evidence.

4.14.2 Exercise 4.2: Recover a Messy History (Level 2)

Here is a git log from an agent-built project:

```
a1b2c3d final
d4e5f6g update
h7i8j9k fix
m0n1o2p changes
q3r4s5t update again
```

The final commit changes forty files. Inspect the diff with `git show`. Identify at least three logical changes. Write a commit message for each one.

If you want to go further, create a branch and redo the commits with `git add -p`. Do not rewrite the main branch until you can explain every change.

Done when: the new messages read like a story, and each message says what changed and why.

4.14.3 Exercise 4.3: README and Runbook (Level 3)

Write a one-page `README.md` and a one-page `runbook.md` for your repo.

The README must cover:

- what the project is
- how to run it locally
- how to deploy it
- where the data lives
- who to contact

The runbook must cover:

- local setup
- deploy steps
- common failures

- rollback
- contacts

Test both from a clean clone. Record every question you had to answer from memory, then add the answer to the docs.

Done when: someone else can run and deploy the project without asking you.

4.14.4 Exercise 4.4: A Decision Record (Level 4)

Write a `DECISIONS.md` entry for one override or significant choice. Include:

- the decision
- the options you considered
- why you chose this one
- what you rejected and what it cost
- the check or evidence that proves it works

Done when: a reader can understand the choice without reading the chat.

4.14.5 Exercise 4.5: The Handover Audit (Level 4)

Swap repositories with a classmate. Do not ask them questions. Follow their README from a clean clone.

Record every point where you had to guess, open a file to understand a step, or ask a question. Then add the missing information to their docs and return the repo with a short known-issues register.

Done when: the next person can follow the docs without the author in the room.

4.14.6 Exercise 4.6: Secrets and Permissions Review (Level 5)

Review your repo for secrets and unnecessary access.

1. Run the project's secrets check.
2. Search the current files for keys, tokens and passwords.
3. Inspect the history with `git log -p` for anything that was removed but not rotated.
4. List what the agent can read, write and run.
5. Write a short permissions policy for the project.

Use a test key or a placeholder. Do not expose a real secret.

Done when: the repo has no live secret, the history is clean or the exposed key has been rotated, and the policy says what the agent may access.

4.15 Chapter Summary

- A repo is the workspace, the record and the handover.
- `AGENTS.md` is a standing contract for the agent, applied to every session.
- Small diffs and useful commit messages make the work reviewable.
- Plan, act, verify. Define the stop condition before the agent starts.
- Skills are reusable task recipes. `AGENTS.md` is the project contract.
- Least privilege and environment variables keep secrets out of the repo.
- The handover pack lets someone else run, deploy, change and diagnose the site.
- If a stranger cannot answer the five questions from the repo, the handover is incomplete.

4.16 Reflection

Before moving on, ensure you can:

☐ Explain what an agent needs from `AGENTS.md`
☐ Write a specific, testable rule
☐ Split a large change into logical commits
☐ Write a commit message that says what changed and why
☐ Plan a task with a stop condition
☐ Explain the difference between `AGENTS.md` and a skill
☐ Keep a secret out of the repo and rotate it if it leaks
☐ Assemble a handover pack that a stranger can use

4.17 Your Learning Journal

Record your responses to these prompts:

1. **The missing rule**: What did the agent do that a better `AGENTS.md` rule would have prevented?
2. **The giant commit**: What made a change hard to review? How would you split it next time?
3. **The stop**: When did you stop the agent? What did you decide instead?
4. **The handover**: What question did a stranger ask that your repo could not answer?
5. **The secret**: What would you do first if a key appeared in the repository?

4.18 Next Steps

Chapter 17 covers the git fundamentals behind this chapter: commits, branches, diffs and the reason a history matters.

Chapter 3 covers the checks that make the handover credible. A handover pack without a way to prove the site still works is only a description.

Chapter 8 covers the serverless function and environment variable that keep a secret out of the client.

Part III

Web Foundations

Chapter 5

Structure: HTML as Information Architecture

5.1 The Concept First

Before we write any HTML, consider what it does: **organising information so both humans and machines can understand it**.

Every document you've ever read has structure. A book has chapters, sections, and paragraphs. A business report has headings, bullet points, and tables. A restaurant menu has categories, items, and descriptions. Structure is how documents communicate meaning.

HTML (HyperText Markup Language) is simply the language we use to describe document structure for the web. When you write HTML, you're not designing how something looks, you're declaring what something *is*.

A heading isn't big, bold text. A heading *is* a heading, a structural element that indicates hierarchy and importance. How it appears is a separate concern.

5.2 Understanding Through Architecture

Think of a well-designed building:

- The **foundation** supports everything above it
- **Walls and rooms** divide space into meaningful areas
- **Signs and labels** tell you what each area is for
- **Doors and corridors** connect different spaces
- **Building codes** ensure safety and accessibility

Now think about a web page:

- The **document declaration** establishes what kind of document this is
- **Semantic elements** divide content into meaningful sections
- **Tags** label what each piece of content represents
- **Links** connect pages and resources
- **Web standards** ensure accessibility across devices

An architect doesn't just make a building that stands, they create spaces that people can navigate, understand, and use. A web developer doesn't just make a page that displays, they create documents that humans can read and machines can process.

> The Architecture Mindset
>
> When you look at any web page, train yourself to see structure first, appearance second. Ask: "What are the meaningful sections here? How is the content organised? What is each element's purpose?"

5.3 Discovering Structure with Your AI Partner

Let's use AI to explore these concepts before we see any code. Remember from Chapter 1: we're using AI to build understanding, not to avoid learning.

5.3.1 Exploration 1: Document Anatomy

Before we think about HTML at all, let's think about documents:

```
Ask your AI:
If I showed you a restaurant menu, what structural
elements would you
identify? Don't use any HTML terms yet, just
describe the organisation
you see in plain English.
```

Notice what your AI identifies: categories of food, individual items, descriptions, prices, perhaps special callouts for dietary information. This is structure, pure information architecture that exists independent of how it's presented.

Now follow up:

```
Continue the conversation:
How would those same structural elements appear in
a company's
"About Us" page?
```

The specific content differs, but structural patterns repeat. Headings introduce sections. Paragraphs contain flowing text. Lists group related items. This pattern recognition is the foundation of HTML thinking.

5.3.2 Exploration 2: Semantic Meaning

The word "semantic" means "relating to meaning." Semantic HTML uses elements that describe what content *is*, not how it should *look*.

```
Ask your AI:
Explain "semantic HTML" using a library catalogue
analogy. Why would
a librarian care whether a book is labelled as
"fiction" versus just
"on shelf 3"?
```

This conversation should reveal why meaningful labels matter. A library where books are only identified by shelf location would be nearly useless. Similarly, a web page where content is only identified by its visual appearance loses meaning for search engines, screen readers, and future maintainers.

Follow up:

```
Continue the conversation:
What problems occur when web pages use visual-only
structure, like
making text big and bold instead of marking it as
a heading?
```

5.3.3 Exploration 3: The Simplest Valid Page

Now let's see actual HTML, but the simplest possible version:

```
Ask your AI:
Show me the absolute simplest valid HTML page that
contains just a
heading and a paragraph. Explain what each line
does as if I've never
seen code before.
```

Your AI should give you something like:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <title>Page Title</title>
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph of text.</p>
</body>
</html>
```

Don't just accept this, have a conversation about it:

```
Continue the conversation:
Why is lang="en" important? Who or what uses that
information?
```

```
Continue the conversation:
What happens if I leave out the DOCTYPE? Would the
page still display?
```

These conversations build understanding that copying code never provides.

5.4 From Concept to Code

Let's build your HTML understanding progressively, starting from the absolute basics.

5.4.1 The Document Container

Every HTML page starts with a declaration and a container:

```
<!DOCTYPE html>
<html lang="en">
  <!-- Everything goes here -->
</html>
```

The `<!DOCTYPE html>` tells browsers "this is a modern HTML document." The `<html>` element is the root container for everything else. The `lang` attribute declares the language, essential for screen readers and translation tools.

5.4.2 The Two Main Sections

Inside the HTML container, every page has exactly two sections:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <!-- Metadata: information ABOUT the page -->
    <meta charset="UTF-8">
    <title>My Business</title>
</head>
<body>
    <!-- Content: what people actually see -->
</body>
</html>
```

The <head> contains metadata, information about the page that doesn't appear on screen. The <body> contains the actual content visitors see and interact with.

Think of it like a business letter: the <head> is the envelope with addresses and stamps, while the <body> is the letter inside.

5.4.3 Semantic Content Elements

Now let's add meaningful content. HTML provides elements that describe *what* content is:

```
<body>
    <header>
        <h1>Sunrise Café</h1>
        <p>Fresh coffee, warm welcome</p>
    </header>

    <main>
        <section>
            <h2>Our Story</h2>
            <p>Founded in 2019, Sunrise Café began
            with a simple
               belief: great coffee brings people
               together.</p>
        </section>

        <section>
            <h2>Visit Us</h2>
            <p>123 Main Street, open daily from
            7am to 5pm.</p>
        </section>
```

```
    </main>

    <footer>
        <p>© 2024 Sunrise Café</p>
    </footer>
</body>
```

Notice what we've communicated:

- `<header>`. The introductory content for the page
- `<main>`. The primary content (there should only be one)
- `<section>`, distinct, thematically grouped content
- `<footer>`, closing information
- `<h1>`, `<h2>`, heading hierarchy (h1 is most important)
- `<p>`, paragraphs of text

A screen reader can now navigate this page by headings. Search engines understand what's primary content versus footer material. Future developers can immediately grasp the page structure.

5.4.4 Common Semantic Elements

Here are the structural elements you'll use most often:

Element	Purpose	Example Use
`<header>`	Introductory content	Site branding, navigation
`<nav>`	Navigation links	Main menu, breadcrumbs
`<main>`	Primary page content	The "meat" of the page
`<section>`	Thematic grouping	Chapters, tabbed content
`<article>`	Self-contained content	Blog post, news story
`<aside>`	Related but separate	Sidebar, pull quote
`<footer>`	Closing content	Copyright, contact links

> ⚠ Common Mistake
>
> Don't choose elements based on how they look by default. Choose them based on what the content *is*. A `<section>` isn't "a box", it's a thematic grouping of content. Appearance is CSS's job (Chapter 6).

5.4.5 Headings Create Hierarchy

Headings aren't just "big text", they create a document outline:

```
<h1>Sunrise Café</h1>           <!-- Page title
(one per page) -->
    <h2>Our Menu</h2>           <!-- Major section
    -->
        <h3>Hot Drinks</h3>     <!-- Subsection
        -->
        <h3>Cold Drinks</h3>    <!-- Subsection
        -->
    <h2>Our Story</h2>          <!-- Major section
    -->
    <h2>Contact Us</h2>         <!-- Major section
    -->
```

This hierarchy should make sense as an outline. Don't skip levels (no jumping from h2 to h4) and don't use headings just to make text bigger.

```
Ask your AI:
Show me a properly structured heading hierarchy
for a small business
website with pages for Home, Services, About, and
Contact. Then show
me a common mistake beginners make with headings.
```

5.5 Building Your Mental Model

5.5.1 The Tree Structure

HTML forms a tree, like a family tree or organisation chart:

```
html
  head
     meta
     title
  body
      header
         h1
         p
      main
         section
            h2
            p
         section
```

```
            h2
            p
      footer
          p
```

Elements contain other elements. This nesting creates relationships: the `<h2>` belongs to its `<section>`, which belongs to `<main>`, which belongs to `<body>`.

```
Ask your AI:
Explain parent, child, and sibling relationships
in HTML using a
company organisational chart as an analogy.
```

5.5.2 Tags, Elements, and Attributes

Let's clarify terminology:

```
<a href="about.html">About Us</a>
```

- **Tag**: `<a>` is the opening tag, `</a>` is the closing tag
- **Element**: The complete unit from opening to closing tag, including content
- **Attribute**: `href="about.html"` provides additional information
- **Content**: "About Us" is the text content between tags

5.5.3 The Box Model Preview

Every HTML element creates an invisible box on the page. These boxes stack and nest according to the document structure. In Chapter 6, we will learn to control these boxes with CSS. For now, understand that structure creates boxes, and boxes create layout.

```
Ask your AI:
If I put a section inside a main element, and two
paragraphs inside
that section, how many "boxes" have I created?
Describe how they
would stack.
```

5.6 Business Applications

5.6.1 Search Engine Optimisation (SEO)

Search engines read your HTML structure to understand your content. A well-structured page with clear headings, semantic sections, and meaningful hierarchy ranks better than a soup of `<div>` elements.

```
Ask your AI:
How do search engines like Google use HTML
structure to understand
what a page is about? What happens when structure
is missing or
confusing?
```

5.6.2 Accessibility

Screen readers, software used by visually impaired people, navigate by structure. Users can jump between headings, skip to main content, or list all navigation links. Without semantic HTML, this navigation is impossible.

Many jurisdictions also require accessible websites for businesses, government, and education, so accessibility carries legal as well as ethical weight.

5.6.3 Maintainability

Well-structured HTML is easier to update. When sections are clearly marked, finding and changing content takes minutes instead of hours. Six months from now, you (or someone else) will thank you for clean structure.

5.6.4 Professional Credibility

When potential employers or clients view your page source, structure tells them about your professionalism. Clean, semantic HTML signals that you understand web fundamentals.

> **i** Skills You Are Building
>
> This develops **evaluating and improving solutions** and **professional communication and project management**. Structure isn't visible to casual viewers, but it's visible to search engines, screen readers, developers, and anyone who inspects your code.

5.7 Practice Exercises

Exercise Levels

- **Level 1**: Direct application
- **Level 2**: Minor modifications
- **Level 3**: Combining concepts
- **Level 4**: Problem-solving
- **Level 5**: Open-ended design

5.7.1 Exercise 5.1: Document Analysis (Level 1)

Find a simple webpage (a local business, a restaurant, a small company). View its source (right-click → View Page Source). Identify:

1. The `<!DOCTYPE>` declaration
2. The `<head>` and `<body>` sections
3. At least three semantic elements (`<header>`, `<main>`, `<footer>`, `<nav>`, `<section>`, `<article>`)
4. The heading hierarchy (`<h1>` through `<h6>`)

Document your findings. Is this page well-structured or poorly structured? Why?

5.7.2 Exercise 5.2: Structure First (Level 2)

Without writing any HTML, create a structural outline for a personal portfolio page. List:

- What would go in the header?
- What sections would the main content have?
- What information belongs in the footer?
- What's the heading hierarchy?

Then, with your AI partner, translate your outline into semantic HTML. Compare your outline with the final code. Did the structure remain clear?

5.7.3 Exercise 5.3: Fix the Structure (Level 3)

Here's a poorly structured page. Identify the problems and rewrite it with proper semantic HTML:

```
<html>
<div>
    <div style="font-size: 32px; font-weight:
    bold;">My Business</div>
```

```
    <div style="font-size: 24px; font-weight:
    bold;">About</div>
    <div>We are a great company.</div>
    <div style="font-size: 24px; font-weight:
    bold;">Contact</div>
    <div>Email us at hello@example.com</div>
    <div style="font-size: 12px;">Copyright
    2024</div>
</div>
</html>
```

List each problem you find and explain why it matters.

5.7.4 Exercise 5.4: Business Requirements (Level 4)

A local bakery asks you to create their website. They mention:

- They want their hours prominently displayed
- They have three categories of products (bread, pastries, cakes)
- They want to show their story and values
- They need contact information and a map

Create a structural plan that addresses their requirements. Which semantic elements would you use for each requirement? Justify your choices in business terms: why does proper structure benefit this bakery?

5.7.5 Exercise 5.5: Semantic Debate (Level 5)

Should a "team members" section on an About page use `<article>` for each person, or `<section>`, or neither?

Research the HTML specification's definitions for both elements. Have a conversation with your AI partner exploring arguments for each choice. Write a short recommendation (200-300 words) defending your position with evidence from web standards.

This exercise has no single correct answer. It develops your ability to make and defend technical decisions (making and defending technology choices).

5.8 Chapter Summary

- HTML describes document structure, not appearance
- Semantic elements convey meaning to humans and machines
- Every page has `<head>` (metadata) and `<body>` (content)

- Headings create hierarchy, don't skip levels or use them for styling
- Structure benefits SEO, accessibility, maintainability, and professionalism
- The structure you create is as important as the content you write

5.9 Reflection

Before moving on, ensure you can:

☐ Explain HTML's purpose without using technical jargon
☐ Identify at least six semantic HTML elements and their purposes
☐ Describe the difference between semantic and presentational markup
☐ Create a document outline before writing code
☐ Explain why structure matters for business (SEO, accessibility, maintenance)
☐ View a page's source and assess its structural quality
☐ Have a productive conversation with AI about HTML concepts

5.10 Your Learning Journal

Record your responses to these prompts:

1. **Structure Spotting**: Look at three websites you use regularly. What structural patterns do they share? What semantic elements can you identify?
2. **AI Conversation Reflection**: What was the most useful question you asked your AI partner in this chapter? What made it effective?
3. **Concept Connection**: How does HTML structure relate to other kinds of structure you encounter (documents, buildings, organisations)?
4. **Lingering Questions**: What aspects of HTML structure are still unclear? Write these as questions to explore with your AI partner.

5.11 Next Steps

You now understand how to describe what content *is*. But a page with only HTML looks plain, black text on white background, default fonts, minimal layout.

In Chapter 6, we'll explore CSS, how to control the visual presentation of our structured content. Structure and presentation are deliberately separate in web development.

The structure you've learned to create becomes the foundation that CSS styles. Good structure makes styling straightforward; poor structure makes it a struggle.

Chapter 6

Presentation: CSS as Visual Communication

6.1 The Concept First

In Chapter 5, you learned to describe what content *is*. Now we'll control how content *looks*: its **visual communication**.

CSS (Cascading Style Sheets) determines colours, fonts, spacing, and layout. But these choices aren't arbitrary. Every visual decision communicates something: professionalism, playfulness, urgency, calm. A hospital website and a children's toy store might have identical HTML structure, but their CSS tells completely different stories.

Structure and presentation are deliberately separate. Your HTML says "this is a heading", it doesn't say "this should be blue and 24 pixels tall." CSS handles appearance. This separation means you can completely redesign a website without touching its content, or update content without breaking the design.

6.2 Understanding Through Design

Think of two cafés with identical menus. Same coffee, same pastries, same prices.

Café A: Exposed brick, warm lighting, comfortable armchairs, handwritten chalkboard menu, soft jazz playing.

Café B: White walls, fluorescent lights, plastic chairs, printed menu in a laminated folder, no music.

The *content* is identical, but would you feel the same about visiting each one? The atmosphere shapes your entire perception of the business. One feels welcoming

and worth paying premium prices; the other feels like a waiting room.

CSS is your website's atmosphere. The content (HTML) might be excellent, but if the presentation feels amateur, untrustworthy, or confusing, users leave.

> **The Atmosphere Test**
>
> Look at any website and ask: "What is this design trying to communicate?" Before you read the words, the visual design has already told you whether this is playful or serious, cheap or premium, trustworthy or suspicious.

6.3 Discovering Presentation with Your AI Partner

6.3.1 Exploration 1: Why Separate Structure and Style?

```
Ask your AI:
Why do we keep HTML (structure) and CSS
(presentation) in separate
languages? Explain using a newspaper analogy, the
same news stories
might appear in a broadsheet and a tabloid.
```

This conversation should show why separation helps: one set of content, many possible presentations. News services send identical stories to papers worldwide, but each paper styles them differently. Similarly, your HTML content can have different CSS applied for screen, print, mobile, or accessibility needs.

Follow up:

```
Continue the conversation:
What problems would occur if we put styling
directly into HTML?
What if every heading had to specify its own
colour and size?
```

6.3.2 Exploration 2: Mobile-First as Business Strategy

More than half of web traffic comes from mobile devices. But mobile-first is a design philosophy as much as a phone layout.

```
Ask your AI:
Explain mobile-first design as a business
strategy, not just a
```

```
technical approach. Why would designing for mobile
first lead to
better experiences for everyone?
```

Your AI should explain how mobile constraints force clarity: limited space means prioritising what matters, slow connections mean optimising performance, touch targets mean accessible interfaces. Designing for these constraints first creates better experiences everywhere.

```
Continue the conversation:
What happens when companies design for desktop
first and then try
to "make it work" on mobile?
```

6.3.3 Exploration 3: Visual Hierarchy

Every page has multiple elements competing for attention. Visual hierarchy tells users what to look at first, second, third, guiding them through your content intentionally.

```
Ask your AI:
How do designers create visual hierarchy, the
sense that some
elements are more important than others? What
tools do they use
besides just making things bigger?
```

This should reveal tools beyond size: colour contrast, whitespace, position, typography weight. A small red button can dominate over a large grey headline because colour attracts attention.

```
Continue the conversation:
Show me a simple example of the same three
elements (heading,
paragraph, button) arranged with different visual
hierarchies
for different purposes.
```

6.4 From Concept to Code

Let's build your CSS understanding progressively, from basic syntax to practical layouts.

6.4.1 CSS Syntax: Selectors, Properties, Values

CSS follows a simple pattern:

```
selector {
    property: value;
}
```

The **selector** identifies which HTML elements to style. The **property** is what aspect to change. The **value** is what to change it to.

```
h1 {
    color: navy;
    font-size: 2rem;
}
```

This says: "Find all `<h1>` elements. Make their text navy. Make them 2rem (roughly 32 pixels) tall."

6.4.2 Connecting CSS to HTML

CSS can live in three places:

External stylesheet (recommended for most work):

```
<head>
    <link rel="stylesheet" href="styles.css">
</head>
```

Internal stylesheet (in the HTML file):

```
<head>
    <style>
        h1 { color: navy; }
    </style>
</head>
```

Inline styles (on individual elements, generally avoid):

```
<h1 style="color: navy;">Welcome</h1>
```

External stylesheets keep presentation separate from structure. One CSS file can style an entire website, and changes apply everywhere.

```
Ask your AI:
What are the advantages and disadvantages of each
```

```
method of adding
CSS? When might you use each one?
```

6.4.3 Common Selectors

Selectors target elements in different ways:

```
/* Element selector - all paragraphs */
p {
    line-height: 1.6;
}

/* Class selector - elements with
class="highlight" */
.highlight {
    background-color: yellow;
}

/* ID selector - the one element with
id="main-header" */
#main-header {
    font-size: 2.5rem;
}

/* Descendant selector - paragraphs inside
articles */
article p {
    margin-bottom: 1em;
}
```

Classes (`.classname`) are your workhorses, reusable styles you can apply to any element. IDs (`#idname`) should be unique per page and are less commonly used in modern CSS.

6.4.4 The Box Model

Every HTML element generates a rectangular box. Understanding this box model is essential for layout:

```
              margin

             border
```

```
              padding

               content
```

- **Content**: The actual text or image
- **Padding**: Space between content and border (inside the box)
- **Border**: The edge of the box
- **Margin**: Space outside the border (between boxes)

```
.card {
    padding: 20px;       /* Space inside */
    border: 1px solid #ccc;
    margin: 10px;        /* Space outside */
}
```

> ⚠ Box-Sizing Gotcha
>
> By default, width and height don't include padding or border. A 200px box with 20px padding becomes 240px wide. Modern practice is to add `box-sizing: border-box;` so width includes everything:
>
> ```
> * {
>     box-sizing: border-box;
> }
> ```
>
> This makes layout calculations much more intuitive.

```
Ask your AI:
Explain the difference between padding and margin
using a picture
frame analogy. The picture is the content, what
are the padding,
border, and margin?
```

6.4.5 Layout with Flexbox

Flexbox is the modern way to arrange elements. It makes previously difficult tasks, like centring or equal-height columns, simple.

```
.container {
    display: flex;
}
```

This single line changes the container. Its children become "flex items" that you can arrange in rows or columns:

```
.container {
    display: flex;
    justify-content: space-between;  /* Horizontal
    distribution */
    align-items: center;             /* Vertical
    alignment */
    gap: 20px;                       /* Space
    between items */
}
```

Common flex patterns:

Navigation bar:

```
nav {
    display: flex;
    justify-content: space-between;
    align-items: center;
}
```

Centred content:

```
.hero {
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 50vh;
}
```

Card grid (with wrap):

```
.cards {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
}
.card {
```

```
    flex: 1 1 300px;  /* Grow, shrink, minimum
    300px */
}
```

```
Ask your AI:
Show me how to create a simple navigation bar with
a logo on the
left and three links on the right using flexbox.
Explain each
property you use.
```

6.4.6 Responsive Design: Media Queries

Responsive design adapts layout to different screen sizes. Media queries let you apply CSS conditionally:

```
/* Base styles (mobile-first) */
.container {
    padding: 1rem;
}

/* Styles for screens 768px and wider */
@media (min-width: 768px) {
    .container {
        padding: 2rem;
        max-width: 800px;
        margin: 0 auto;
    }
}
```

Mobile-first means your base CSS is for mobile, then you add complexity for larger screens. This approach naturally prioritises essential content.

```
/* Mobile: single column */
.features {
    display: flex;
    flex-direction: column;
    gap: 1rem;
}

/* Tablet and up: two columns */
@media (min-width: 768px) {
```

```
    .features {
        flex-direction: row;
        flex-wrap: wrap;
    }
    .feature {
        flex: 1 1 45%;
    }
}

/* Desktop: three columns */
@media (min-width: 1024px) {
    .feature {
        flex: 1 1 30%;
    }
}
```

```
Ask your AI:
Walk me through how a three-column desktop layout
should adapt as
the screen gets smaller. What should happen at
each breakpoint
and why?
```

6.4.7 Typography and Colour Basics

Typography and colour have immediate visual impact:

```
body {
    font-family: system-ui, -apple-system,
    sans-serif;
    font-size: 16px;
    line-height: 1.6;
    color: #333;
}

h1, h2, h3 {
    line-height: 1.2;
    margin-bottom: 0.5em;
}

a {
    color: #0066cc;
```

```
}

a:hover {
    color: #004499;
}
```

Using `rem` for sizes keeps everything relative to the base font size, making your design more adaptable:

```
h1 { font-size: 2.5rem; }    /* 40px if base is
16px */
h2 { font-size: 2rem; }      /* 32px */
h3 { font-size: 1.5rem; }    /* 24px */
p  { font-size: 1rem; }      /* 16px */
```

6.5 Building Your Mental Model

6.5.1 The Cascade: Why "Cascading"?

CSS rules can conflict. When multiple rules target the same element, the cascade determines which wins:

1. **Specificity**: More specific selectors win (`#id` beats `.class` beats `element`)
2. **Source order**: Later rules override earlier ones (if specificity is equal)
3. **Importance**: `!important` overrides everything (but avoid using it)

```
p { color: black; }           /* Specificity:
0,0,1 */
.highlight { color: yellow; } /* Specificity:
0,1,0 - wins */
#special { color: red; }      /* Specificity:
1,0,0 - wins over class */
```

```
Ask your AI:
Why is understanding specificity important? What
problems occur
when developers don't understand how the cascade
works?
```

6.5.2 The Flow: Block vs Inline

HTML elements have default display behaviours:

Block elements (`<div>`, `<p>`, `<h1>`, `<section>`): - Stack vertically - Take full available width - Can have width and height set

Inline elements (`<span>`, `<a>`, `<strong>`): - Flow horizontally with text - Take only the space they need - Cannot have width and height set (use `display: inline-block` if needed)

Understanding flow explains why elements appear where they do before you apply any layout CSS.

6.5.3 The Mental Stack

When you look at a webpage, think in layers:

1. **Content layer** (HTML): What exists
2. **Style layer** (CSS): How it looks
3. **Behaviour layer** (JavaScript, Chapter 7): How it responds

CSS only controls layer 2. It can't create content or make things interactive, it controls visual presentation of whatever HTML provides.

6.6 Business Applications

6.6.1 Brand Consistency

CSS variables (custom properties) enforce brand standards across an entire site:

```
:root {
    --brand-primary: #2563eb;
    --brand-secondary: #1e40af;
    --font-heading: 'Georgia', serif;
    --font-body: system-ui, sans-serif;
}

h1, h2, h3 {
    font-family: var(--font-heading);
    color: var(--brand-primary);
}
```

Change the variable once, and every element using it updates instantly. This is how large organisations maintain consistency across dozens of pages.

6.6.2 User Experience and Trust

Users judge website credibility in milliseconds, based primarily on visual design. Professional presentation builds trust; amateur presentation creates suspicion,

regardless of how good the actual content is.

```
Ask your AI:
What visual characteristics make a website feel
"trustworthy" versus
"suspicious"? Think about spacing, typography,
colour, and layout.
```

6.6.3 Mobile Reach

If your site doesn't work on mobile, you're excluding over half your potential audience. Responsive design is a baseline requirement for any business website.

6.6.4 Conversion and Visual Hierarchy

Where users look determines what they do. Visual hierarchy guides attention to calls-to-action, important information, and conversion points. Poor hierarchy means users miss what matters.

> **i** Skills You Are Building
>
> This develops **evaluating and improving solutions** and **making and defending technology choices**. CSS choices affect user experience, brand perception, and business outcomes. The ability to evaluate and implement appropriate visual design is essential professional skill.

6.7 Practice Exercises

> **i** Exercise Levels
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

6.7.1 Exercise 6.1: Style Inspection (Level 1)

Using browser DevTools (right-click → Inspect), examine a website you use regularly:

1. Find an element and identify its box model (content, padding, border, margin)

2. Locate where its colour is defined in the CSS
3. Find a media query and identify what changes at different screen sizes
4. Identify at least one flexbox container

Document your findings with screenshots.

6.7.2 Exercise 6.2: Basic Styling (Level 2)

Take the semantic HTML page from Chapter 5's exercises. Create an external stylesheet that:

1. Sets a readable base font and line height
2. Styles the header with a background colour
3. Adds appropriate padding to sections
4. Creates visual distinction between the main content and footer

Focus on readability and visual hierarchy, not decoration.

6.7.3 Exercise 6.3: Responsive Layout (Level 3)

Create a "features" section with three items. Each item has: - An icon or emoji placeholder - A heading - A short description

Using flexbox, make this: - Single column on mobile (under 768px) - Three columns on desktop

Write the CSS mobile-first, adding complexity with media queries.

6.7.4 Exercise 6.4: Design Analysis (Level 4)

Find two competing businesses in the same industry (e.g., two accounting firms, two coffee shops). Analyse their websites:

1. How does each use visual hierarchy? What's emphasised?
2. What do their colour choices communicate?
3. How does typography differ and what does it suggest?
4. Which feels more trustworthy? Why?

Write a 300-word comparison explaining how CSS choices reflect (or undermine) each brand's positioning.

6.7.5 Exercise 6.5: Design System (Level 5)

Create a mini design system for a hypothetical business. Define:

1. A colour palette (primary, secondary, accent, text, background)
2. Typography choices (headings, body, sizes)

3. Spacing scale (consistent padding/margin values)
4. A set of CSS variables implementing these choices

Document your decisions: why these colours? Why these fonts? How do these choices communicate the brand personality?

Apply your system to style a simple page, then have your AI partner critique it.

6.8 Chapter Summary

- CSS controls visual presentation separately from HTML structure
- The box model (content, padding, border, margin) explains how elements take space
- Flexbox provides flexible layout capabilities
- Media queries enable responsive design that adapts to any device
- Visual design communicates brand, builds trust, and guides user attention
- Mobile-first design creates better experiences for everyone

6.9 Reflection

Before moving on, ensure you can:

☐ Explain why structure and presentation are separate
☐ Write CSS selectors to target specific elements
☐ Describe the box model and how padding differs from margin
☐ Create flexible layouts with flexbox
☐ Write mobile-first CSS using media queries
☐ Articulate how visual design affects business outcomes
☐ Use browser DevTools to inspect and understand CSS

6.10 Your Learning Journal

Record your responses to these prompts:

1. **Design Awareness**: Look at three websites with very different visual styles. How does each CSS approach reflect its purpose and audience?
2. **Mobile Testing**: View a website you built (or one you use) on your phone. What works well? What's frustrating? How would you improve it?
3. **AI Conversation Reflection**: What CSS concept was hardest to grasp? What question to your AI partner helped clarify it?
4. **Business Connection**: Think of a business you might create a website for. What visual atmosphere would best serve that business? Why?

6.11 Next Steps

You now control both structure (HTML) and presentation (CSS). But websites today are applications to interact with as well as documents to read.

In Chapter 7, we'll add JavaScript, the third pillar of web development. JavaScript lets pages respond to user actions: clicks, scrolls, form submissions. That turns static documents into interactive pages.

JavaScript works on the HTML structure and CSS styling you have learned.

Chapter 7

Behaviour: JavaScript as User Interaction

7.1 The Concept First

HTML provides structure. CSS provides presentation. JavaScript provides **behaviour**, the ability to respond to user actions, update content dynamically, and create interactive experiences.

Consider a contact form. HTML creates the fields and the submit button. CSS makes it look professional. But what happens when someone clicks "Submit"? What validates their input? What shows them a success message? That's JavaScript.

JavaScript makes pages respond without a reload: dropdown menus expand, forms validate as you type, and content loads in place.

But **JavaScript should enhance, not replace, structure and presentation**. A well-built page works without JavaScript. JavaScript makes it work *better*.

7.2 Understanding Through Conversation

A static webpage is like a poster on a wall. You can look at it, but it can't respond to you.

JavaScript makes your page more like a conversation:

- **It listens**: "The user clicked this button"
- **It thinks**: "I should validate the form now"
- **It responds**: "Show the error message next to the email field"
- **It adapts**: "Now that they've fixed the email, remove the error"

This back-and-forth is called **event-driven programming**. Instead of running top to bottom like a recipe, JavaScript waits for things to happen (events) and responds to them.

> The Conversation Mindset
>
> When designing interactive features, think: "What might the user do?" (events) and "How should the page respond?" (handlers). Every interaction is a small conversation between user and page.

7.3 Discovering Behaviour with Your AI Partner

7.3.1 Exploration 1: Event-Driven Thinking

The event-driven model is fundamental to understanding JavaScript. Let's explore it through analogy:

```
Ask your AI:
Explain event-driven programming using a
restaurant service analogy.
The waiter doesn't constantly ask "Are you ready
to order?" every
second, how does the restaurant model work
instead?
```

This should reveal the pattern: staff don't constantly poll customers. Instead, customers signal (raise a hand, make eye contact) and staff respond. Events trigger responses.

Follow up:

```
Continue the conversation:
What "events" happen on a typical webpage? List as
many as you can
think of, things users do that a page might
respond to.
```

7.3.2 Exploration 2: The DOM

JavaScript interacts with pages through something called the DOM (Document Object Model). But what is it?

```
Ask your AI:
What is the DOM? Explain it as if describing how a
```

```
librarian has a
catalogue system that represents all the books on
the shelves. The
books are the actual HTML, what's the catalogue?
```

The DOM is JavaScript's representation of your HTML structure. When you change the DOM, the page updates visually. It's the bridge between your code and what users see.

```
Continue the conversation:
If I want to change the text inside a paragraph
using JavaScript,
am I changing the HTML file? Or something else?
Walk me through
what actually happens.
```

7.3.3 Exploration 3: Progressive Enhancement

```
Ask your AI:
Why should websites work without JavaScript?
Explain the business
case, not just the technical reasons. Consider:
who can't use
JavaScript and why?
```

This conversation should reveal important realities: some users disable JavaScript for security, some have slow connections where it fails to load, some use assistive technologies that struggle with JS-heavy sites, and some corporate firewalls block scripts. Building core functionality in HTML and CSS, then *enhancing* with JavaScript, serves more customers.

```
Continue the conversation:
Give me an example of a feature built with
progressive enhancement
versus one that fails without JavaScript. What's
the difference
in user experience?
```

7.4 From Concept to Code

Let's build your JavaScript understanding progressively, from basic syntax to handling user interactions.

7.4.1 Where JavaScript Lives

Like CSS, JavaScript can be placed in different locations:

External file (recommended):

```
<body>
    <!-- Your HTML content -->
    <script src="script.js"></script>
</body>
```

Inline in HTML (for small scripts):

```
<script>
    console.log('Hello from JavaScript!');
</script>
```

Place `<script>` tags at the end of `<body>` so HTML loads first. This ensures elements exist before JavaScript tries to interact with them.

7.4.2 Variables: Storing Information

Variables hold data that can change:

```
// Modern variable declarations
let userName = 'Alex';        // Can be reassigned
const maxItems = 10;          // Cannot be
reassigned
```

Use `const` by default, it prevents accidental changes. Use `let` only when you need to reassign the value.

```
const greeting = 'Welcome';    // Use const for
values that won't change
let itemCount = 0;             // Use let when
you'll update the value
itemCount = itemCount + 1;     // This works
// greeting = 'Hello';         // This would cause
an error
```

> ⚠ Avoid `var`
>
> You'll see `var` in older code and some AI-generated examples. It has confusing scoping rules. Modern JavaScript uses `let` and `const` exclusively. If your AI suggests `var`, ask it to use `let` or `const` instead.

7.4.3 Data Types

JavaScript handles several types of data:

```
// Strings - text in quotes
const businessName = 'Sunrise Café';

// Numbers - no quotes
const price = 4.50;
const quantity = 3;

// Booleans - true or false
const isOpen = true;
const hasDiscount = false;

// Arrays - ordered lists
const menuItems = ['Coffee', 'Tea', 'Pastry'];

// Objects - structured data
const product = {
    name: 'Cappuccino',
    price: 4.50,
    available: true
};
```

```
Ask your AI:
When would I use an array versus an object? Give
me three real
examples of data that fits each structure.
```

7.4.4 Functions: Reusable Actions

Functions group code that performs a specific task:

```
// Defining a function
function calculateTotal(price, quantity) {
    const total = price * quantity;
```

```
    return total;
}

// Calling the function
const orderTotal = calculateTotal(4.50, 3);
console.log(orderTotal);  // 13.5
```

Functions take inputs (parameters), do something, and optionally return a result. They're how you organise code into manageable, reusable pieces.

Arrow function syntax (modern, compact):

```
const calculateTotal = (price, quantity) => {
    return price * quantity;
};

// Even shorter for simple functions
const double = (number) => number * 2;
```

Both syntaxes work. Arrow functions are common in modern code, especially for short operations.

7.4.5 Finding Elements: Querying the DOM

To interact with page elements, first you must find them:

```
// Find one element by CSS selector
const header = document.querySelector('header');
const submitBtn =
document.querySelector('#submit-button');
const firstCard = document.querySelector('.card');

// Find multiple elements
const allCards =
document.querySelectorAll('.card');
```

`querySelector` uses CSS selector syntax, the same patterns you learned in Chapter 6. This consistency makes finding elements intuitive.

7.4.6 Changing Elements

Once you've found an element, you can modify it:

```
// Change text content
const heading = document.querySelector('h1');
heading.textContent = 'Welcome Back!';

// Change HTML content (use carefully)
const container =
document.querySelector('.message');
container.innerHTML = '<strong>Success!</strong>
Your order is confirmed.';

// Change styles
heading.style.color = 'navy';
heading.style.fontSize = '2rem';

// Add or remove CSS classes (preferred for
styling)
heading.classList.add('highlighted');
heading.classList.remove('hidden');
heading.classList.toggle('active');
```

> **Classes Over Inline Styles**
>
> Instead of setting `element.style.color = 'red'`, add a class that has those styles. This keeps your CSS in CSS and your JavaScript focused on behaviour.

7.4.7 Events: Responding to Users

Events are things that happen, clicks, key presses, form submissions, page loads. Event listeners wait for specific events and run code when they occur:

```
const button =
document.querySelector('#submit-button');

button.addEventListener('click', function() {
    console.log('Button was clicked!');
});
```

The pattern is: `element.addEventListener(eventType, handlerFunction)`

Common events:

Event	Triggered When
`click`	User clicks the element
`submit`	Form is submitted
`input`	Input value changes
`keydown`	Key is pressed
`mouseover`	Mouse enters element
`load`	Page finishes loading

7.4.8 A Complete Example: Toggle Menu

Let's combine these concepts into a practical feature, a mobile navigation toggle:

```
<button id="menu-toggle">Menu</button>
<nav id="main-nav" class="hidden">
    <a href="/">Home</a>
    <a href="/about">About</a>
    <a href="/contact">Contact</a>
</nav>
```

```
.hidden {
    display: none;
}
```

```
const menuButton =
document.querySelector('#menu-toggle');
const navigation =
document.querySelector('#main-nav');

menuButton.addEventListener('click', function() {
    navigation.classList.toggle('hidden');
});
```

This is progressive enhancement: the HTML structure exists, the CSS hides the menu by default on mobile, and JavaScript adds the toggle behaviour. If JavaScript fails, you could add `class="hidden"` via CSS media queries so the menu shows on larger screens regardless.

7.4.9 Form Validation Example

Here's a practical business example, validating an email field:

```
<form id="contact-form">
    <label for="email">Email:</label>
    <input type="email" id="email" name="email"
    required>
    <span id="email-error" class="error
    hidden"></span>
    <button type="submit">Send</button>
</form>
```

```
const form =
document.querySelector('#contact-form');
const emailInput =
document.querySelector('#email');
const errorSpan =
document.querySelector('#email-error');

emailInput.addEventListener('input', function() {
    // Check if it looks like an email
    if (emailInput.validity.valid) {
        errorSpan.classList.add('hidden');
        errorSpan.textContent = '';
    } else {
        errorSpan.classList.remove('hidden');
        errorSpan.textContent = 'Please enter a
        valid email address';
    }
});

form.addEventListener('submit', function(event) {
    if (!emailInput.validity.valid) {
        event.preventDefault();  // Stop form
        submission
        errorSpan.classList.remove('hidden');
        errorSpan.textContent = 'Please enter a
        valid email address';
    }
});
```

This validates as the user types and again on submission. Note: HTML5 provides `type="email"` validation too. JavaScript improves the feedback on top of that baseline.

```
Ask your AI:
Walk me through this form validation code line by
line. What does
event.preventDefault() do and why is it important
here?
```

7.5 Building Your Mental Model

7.5.1 The Event Loop

JavaScript doesn't run all at once. It:

1. Loads and runs initial code
2. Waits for events
3. When an event occurs, runs the handler
4. Returns to waiting

This is why you register event listeners, you're telling JavaScript "when this happens, run this code." The rest of the time, JavaScript is idle, waiting.

```
Ask your AI:
Explain the JavaScript event loop using an office
receptionist
analogy. The receptionist handles visitors
(events) but also has
other tasks. How do they prioritise?
```

7.5.2 The Three Layers

Reinforce this mental model from earlier chapters:

```
  JavaScript (Behaviour)                What
responds

  CSS (Presentation)                    What it
looks like

  HTML (Structure)                      What
exists
```

Each layer builds on the one below. JavaScript manipulates the DOM (HTML

structure), which CSS then styles. Keeping these layers separate makes code maintainable.

7.5.3 Debugging with Console

The browser console is your main debugging tool:

```
console.log('The value is:', someVariable);  //
Print values
console.error('Something went wrong!');       //
Print errors
console.table(arrayOfObjects);                //
Print tables
```

> **Your Most Important Tool: Browser Developer Console**
>
> Every modern browser has a built-in developer console. Open it with **F12** (or right-click any page and select **Inspect**). It has three tabs you will use constantly:
>
> - **Elements**: see the actual HTML and CSS the browser received. Useful for checking whether AI-generated markup is structured correctly, or why a style is not applying.
> - **Console**: see `console.log()` output, JavaScript errors, and warnings. This is where you debug behaviour.
> - **Network**: see every file and API request the page makes. Useful for debugging fetch calls and checking what data comes back.
>
> Get comfortable here. When AI gives you code that does not work, the dev console tells you *why*. It shows you what the browser actually received, not what you intended to send.

```
Ask your AI:
What are common JavaScript debugging techniques?
How do I figure out
why my code isn't working?
```

7.6 Business Applications

7.6.1 User Engagement

Interactive features keep users engaged. Think: image galleries, accordion FAQs, live search filtering. Each interaction is an opportunity to help users find what they need.

7.6.2 Form Validation and Data Quality

Client-side validation prevents bad data before it reaches your server. This improves data quality, reduces support requests, and creates a smoother user experience.

> ⚠ Security Note
>
> Client-side validation improves user experience, but *never* trust it for security. Users can disable JavaScript or modify your code. Always validate on the server too.

7.6.3 Dynamic Content

Loading content without page refreshes (covered more in Chapter 8) makes applications feel faster. Users don't wait for full page reloads when they just need one small update.

7.6.4 Analytics and User Behaviour

JavaScript can track how users interact with your site: what they click, how far they scroll, where they spend time. This data informs business decisions. (Always be transparent about tracking and respect privacy.)

> ℹ Skills You Are Building
>
> This develops **evaluating and improving solutions** and **translating stakeholder needs into technical requirements**. Interactive features must serve user needs. JavaScript for its own sake creates complexity without value. The ability to identify when interactivity helps versus hinders is professional judgment.

7.7 Practice Exercises

> ℹ Exercise Levels
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

7.7.1 Exercise 7.1: Console Exploration (Level 1)

Open any webpage and open the browser console (F12 → Console). Try these:

1. Type `document.title` and press Enter. What appears?
2. Type `document.querySelector('h1')` to find the main heading
3. Type `document.querySelector('h1').textContent = 'I changed this!'`

What happened? Refresh the page, is your change permanent? Why or why not?

7.7.2 Exercise 7.2: Button Interaction (Level 2)

Create an HTML page with a button and a paragraph. Write JavaScript that:

1. Listens for clicks on the button
2. Changes the paragraph text when clicked
3. Also changes the paragraph's colour (via adding a class)

Test it in your browser. Use `console.log()` to confirm your click handler runs.

7.7.3 Exercise 7.3: Character Counter (Level 3)

Create a textarea with a character limit display, like social media post composers:

1. Show "0 / 280 characters" below the textarea
2. Update the count as the user types (use the `input` event)
3. Change the counter colour to red when approaching or exceeding the limit
4. Disable the submit button if over the limit

This combines DOM queries, event listeners, conditionals, and class manipulation.

7.7.4 Exercise 7.4: Progressive Enhancement Audit (Level 4)

Find a website that heavily uses JavaScript (a web application). Disable JavaScript in your browser (Settings → Privacy → JavaScript) and try to use the site.

1. What still works?
2. What breaks completely?
3. For the broken features, how could they have been built with progressive enhancement?

Write a 300-word assessment of this site's JavaScript dependency and its business implications.

7.7.5 Exercise 7.5: Interactive Feature Design (Level 5)

A local bookshop wants their website to have an interactive "Book Finder" feature. Users should be able to filter books by genre, search by title, and sort by price or rating.

1. Design the HTML structure needed
2. Identify what events you'd listen for
3. Describe what JavaScript would do for each interaction
4. Consider: what should work without JavaScript?

Don't write the full code, design the approach. Document your thinking, then discuss your design with your AI partner. What did they suggest differently?

7.8 Chapter Summary

- JavaScript adds behaviour, the ability to respond to user actions
- Event-driven programming means listening for events and responding
- The DOM is JavaScript's interface to the page structure
- Progressive enhancement builds on working HTML/CSS, not replacing it
- Debugging with `console.log()` helps trace problems
- Interactive features should serve user needs, not demonstrate technical skill

7.9 Reflection

Before moving on, ensure you can:

☐ Explain event-driven programming in plain language
☐ Write basic JavaScript: variables, functions, conditionals
☐ Find elements in the DOM using `querySelector`
☐ Attach event listeners to respond to user actions
☐ Modify element content, classes, and styles via JavaScript
☐ Articulate why progressive enhancement matters for business
☐ Use the browser console for basic debugging

7.10 Your Learning Journal

Record your responses to these prompts:

1. **Interaction Audit**: Look at a website you use frequently. What interactive features does it have? For each one, describe what event triggers it and what the response is.

2. **Progressive Enhancement Thinking**: Think of a common web feature (like a dropdown menu or image carousel). How would you build it so it works without JavaScript?
3. **AI Conversation Reflection**: What JavaScript concept was hardest to grasp? What question to your AI partner helped clarify it?
4. **Business Value**: When does adding JavaScript interactivity help users, and when does it just add complexity? Give examples of each.

7.11 Next Steps

You can now build pages that respond to users. But modern web applications do more, they connect to services, fetch data, and send information to servers.

In Chapter 8, we'll explore APIs, how your JavaScript can request data from external services and submit data to be processed. That connects your pages to external services and the wider web.

The JavaScript fundamentals you've learned become the foundation for asynchronous programming and data-driven interfaces.

Chapter 8

Connection: APIs as Business Integration

8.1 The Concept First

So far, everything you've built lives in isolation. Your HTML, CSS, and JavaScript create an experience, but that experience is self-contained, it doesn't connect to anything beyond itself.

Real business applications are different. They display live weather data. They process payments. They show inventory levels. They sync with customer databases. They pull content from external services.

They do it through **APIs** (Application Programming Interfaces).

An API is a defined way for two systems to communicate. When your JavaScript asks a weather service "What's the temperature in Perth?", it's using an API. When a customer's payment details go to Stripe for processing, that's an API. When your portfolio site pulls your latest GitHub projects, that's an API.

APIs connect your website to those services instead of leaving it standalone.

8.2 Understanding Through Contracts

Think of an API as a contract between two parties. Like any good contract, it specifies:

- **What you can ask for** (the endpoints)
- **How to ask** (the request format)
- **What you'll receive** (the response format)
- **What happens if something goes wrong** (error handling)

Imagine a restaurant:

- The **menu** lists what you can order (available endpoints)
- You place an order **in a specific way**, verbally to a waiter, on a printed slip (request format)
- The kitchen prepares your food and the waiter delivers it (response)
- If something's unavailable, they tell you (error response)

You don't need to know how the kitchen works. You just need to know how to order and what to expect.

> **The Contract Mindset**
>
> When working with any API, first understand the contract: What can I request? What format does it expect? What will I receive? What errors might occur? Read the documentation before writing code.

8.3 Discovering Connection with Your AI Partner

8.3.1 Exploration 1: How APIs Work

Let's build understanding before we see code:

```
Ask your AI:
Explain REST APIs using a library system analogy.
The library has
books (resources). How would a REST API let
someone check what
books exist, borrow a book, return a book, and add
new books?
```

This should reveal the core pattern: APIs expose *resources* (data) and let you perform *operations* on them. The operations typically map to HTTP methods: GET (read), POST (create), PUT/PATCH (update), DELETE (remove).

```
Continue the conversation:
What's the difference between GET and POST
requests? Why would you
use one versus the other?
```

8.3.2 Exploration 2: Data Formats

APIs need a common language for data. That language is usually JSON.

```
Ask your AI:
Why is JSON the standard format for web API data?
What problem does
it solve, and what did people use before JSON?
```

JSON (JavaScript Object Notation) is readable by humans and easily parsed by machines. It's become the standard format for web data exchange.

```
Continue the conversation:
Show me what a JSON response might look like for a
product catalogue
with three items. Each item has a name, price,
description, and
whether it's in stock.
```

8.3.3 Exploration 3: What Can Go Wrong?

Network requests fail. Servers go down. Data doesn't match expectations. Professional applications handle these gracefully.

```
Ask your AI:
What can go wrong when calling an API? List the
different types of
failures and explain what each HTTP status code
range (2xx, 4xx, 5xx)
means.
```

Understanding failure modes is essential. Your code must handle them or your users will see cryptic errors, or worse, a completely broken page.

```
Continue the conversation:
How should a professional application respond to
each type of failure?
What should the user see?
```

8.4 From Concept to Code

Let's build your API understanding progressively, from fetching data to handling complex scenarios.

8.4.1 The Fetch API

JavaScript's `fetch()` function makes HTTP requests:

```
fetch('https://api.example.com/products')
    .then(response => response.json())
    .then(data => {
        console.log(data);
    });
```

This fetches data from a URL, converts the response to JSON, and logs it. But what's with all those `.then()` calls?

8.4.2 Understanding Asynchronous Code

API requests take time. The server might be across the world. Your code can't just wait, it would freeze the entire page.

JavaScript handles this with **asynchronous** code. Instead of stopping and waiting, it says "start this request, and when it's done, run this code."

The `.then()` pattern is called a Promise. But modern JavaScript has a cleaner syntax: `async/await`.

8.4.3 Async/Await: Readable Asynchronous Code

```
async function loadProducts() {
    const response = await
    fetch('https://api.example.com/products');
    const data = await response.json();
    console.log(data);
}

loadProducts();
```

`await` pauses the function (not the whole page) until the operation completes. The `async` keyword marks functions that use `await`.

This reads more naturally: "Fetch the URL. Wait. Parse the JSON. Wait. Log the data."

8.4.4 A Complete Example: Displaying API Data

Let's fetch real data and display it:

```
<section id="products">
    <h2>Our Products</h2>
    <div id="product-list">Loading...</div>
```

```
</section>
```

```
async function displayProducts() {
    const container =
    document.querySelector('#product-list');

    try {
        const response = await
        fetch('https://fakestoreapi.com/products?limit=3');
        const products = await response.json();

        // Clear loading message
        container.innerHTML = '';

        // Create HTML for each product
        products.forEach(product => {
            const productCard =
            document.createElement('div');
            productCard.classList.add('product-card');
            productCard.innerHTML = `
                <h3>${product.title}</h3>
                <p>$${product.price}</p>
            `;
            container.appendChild(productCard);
        });

    } catch (error) {
        container.innerHTML = '<p>Sorry, products
        could not be loaded.</p>';
        console.error('Fetch failed:', error);
    }
}

displayProducts();
```

Key elements:

1. Show a loading state initially
2. Fetch data with `await`
3. Parse JSON with `await`
4. Create DOM elements for each item
5. Handle errors with `try/catch`

```
Ask your AI:
Walk me through this code step by step. What does
forEach do?
What does createElement do? What happens if the
fetch fails?
```

8.4.5 Working with JSON Data

JSON data looks like JavaScript objects:

```
{
    "id": 1,
    "title": "Product Name",
    "price": 29.99,
    "category": "electronics",
    "inStock": true,
    "tags": ["featured", "sale"]
}
```

Accessing data uses dot notation or brackets:

```
const product = await response.json();

console.log(product.title);       // "Product
Name"
console.log(product.price);       // 29.99
console.log(product.tags[0]);     // "featured"
console.log(product['category']); // "electronics"
```

For arrays of items:

```
const products = await response.json();

products.forEach(product => {
    console.log(product.title);
});

// Or find specific items
const expensive = products.filter(p => p.price >
50);
const firstElectronic = products.find(p =>
p.category === 'electronics');
```

8.4.6 Error Handling in Depth

Applications need to handle several failure scenarios:

```
async function loadData() {
    try {
        const response = await
        fetch('https://api.example.com/data');

        // Check if the response was successful
        if (!response.ok) {
            throw new Error(`HTTP error:
            ${response.status}`);
        }

        const data = await response.json();
        return data;

    } catch (error) {
        if (error.name === 'TypeError') {
            // Network failure - couldn't reach
            the server
            console.error('Network error:',
            error);
            return { error: 'Unable to connect.
            Please check your internet.' };
        } else {
            // Other error (including our HTTP
            error)
            console.error('Fetch error:', error);
            return { error: 'Something went wrong.
            Please try again.' };
        }
    }
}
```

⚠ Check `response.ok`

A `fetch()` that reaches the server won't throw an error even if the server returns a 404 or 500 status. You must check `response.ok` to detect these failures.

8.4.7 Loading States

Users should never see a blank screen while waiting:

```
async function loadWithStatus() {
    const container =
    document.querySelector('#content');
    const loadingMessage =
    document.querySelector('#loading');

    // Show loading state
    loadingMessage.classList.remove('hidden');
    container.innerHTML = '';

    try {
        const response = await
        fetch('https://api.example.com/data');
        if (!response.ok) throw new Error('Fetch
        failed');

        const data = await response.json();
        displayData(data, container);

    } catch (error) {
        container.innerHTML = '<p
        class="error">Unable to load
        content.</p>';

    } finally {
        // Always hide loading, whether success or
        failure
        loadingMessage.classList.add('hidden');
    }
}
```

The `finally` block runs regardless of success or failure, perfect for cleanup like hiding loading spinners.

8.4.8 HTTP Methods: Beyond GET

So far we've used GET (fetching data). Other methods exist:

```
// POST - create new data
const response = await
```

```
fetch('https://api.example.com/orders', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json'
    },
    body: JSON.stringify({
        product: 'Coffee',
        quantity: 2
    })
});

// PUT - replace existing data
await fetch('https://api.example.com/orders/123',
{
    method: 'PUT',
    headers: { 'Content-Type': 'application/json'
    },
    body: JSON.stringify({ product: 'Tea',
    quantity: 3 })
});

// DELETE - remove data
await fetch('https://api.example.com/orders/123',
{
    method: 'DELETE'
});
```

Method	Purpose	Has Body?
GET	Read data	No
POST	Create new data	Yes
PUT	Replace data completely	Yes
PATCH	Update part of data	Yes
DELETE	Remove data	Usually no

8.4.9 CORS: Why Some Requests Fail

You might encounter this error:

> Access to fetch at 'https://other-site.com/api' has been blocked by CORS policy

CORS (Cross-Origin Resource Sharing) is a security feature. By default, JavaScript

can only fetch from the same domain the page is on. To fetch from a different domain, that domain must explicitly allow it.

```
Ask your AI:
Explain CORS in simple terms. Why does it exist?
What's it protecting
against?
```

As a frontend developer, you can't fix CORS on a third-party server. Your options:

1. Use APIs that allow CORS (most public APIs do)
2. Ask the API provider to enable CORS
3. Use a backend proxy (your server fetches the data)

> **Public APIs for Practice**
>
> Many APIs welcome frontend requests: JSONPlaceholder, OpenWeatherMap, GitHub API, and more. These are great for learning.

8.5 Building Your Mental Model

8.5.1 Request-Response Cycle

Every API interaction follows this pattern:

```
              Request

   Browser                              Server
  (Your JS)                        (API)
                    Response
```

1. Your JavaScript builds a request (URL, method, body, headers)
2. Browser sends it across the network
3. Server processes and responds
4. Your JavaScript handles the response

Everything that can go wrong, network issues, server errors, malformed data, happens in this cycle.

8.5.2 Synchronous vs Asynchronous

```
Synchronous (blocking):
Step 1 → Wait → Step 2 → Wait → Step 3
```

```
Asynchronous (non-blocking):
Step 1 → Start Step 2 (don't wait) → Step 3
                ↓
         Step 2 completes → Handle result
```

`async/await` makes asynchronous code look synchronous while keeping the page responsive.

8.5.3 The Data Flow

```
API Response (JSON string)
        ↓
   response.json() (parse)
        ↓
JavaScript Objects/Arrays
        ↓
   Process & Transform
        ↓
   Create DOM Elements
        ↓
   Page Updates
```

Understanding this flow helps debug issues: Is the problem in fetching? Parsing? Processing? Rendering?

8.6 Business Applications

8.6.1 Real-Time Data

Stock prices, sports scores, weather. APIs provide live data that keeps users engaged and informed. Without APIs, you'd need to manually update your site constantly.

8.6.2 Payment Processing

No one stores credit cards themselves (the liability is enormous). Payment APIs like Stripe, PayPal, and Square handle the sensitive parts, letting you focus on the shopping experience.

8.6.3 Third-Party Services

Maps (Google Maps, Mapbox), analytics (Google Analytics), authentication (Auth0), email (SendGrid). APIs let you use sophisticated services without building them yourself.

8.6.4 Scalability and Architecture

Separating your frontend (the user interface) from your backend (the data and business logic) via APIs allows each to scale independently. Your static site can be served from a CDN worldwide while your API runs on dedicated servers.

8.6.5 Business Integration

Connect your website to: - Inventory management systems - Customer relationship management (CRM) - Email marketing platforms - Accounting software

> **i** Skills You Are Building
>
> This develops **making and defending technology choices** and **evaluating and improving solutions**. API integration is fundamental to modern web development. Understanding how to evaluate, implement, and handle errors in API connections is essential professional knowledge.

8.7 Practice Exercises

> **i** Exercise Levels
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

8.7.1 Exercise 8.1: Your First API Call (Level 1)

Use the JSONPlaceholder API (`https://jsonplaceholder.typicode.com/posts/1`) to:

1. Fetch a single post using `fetch()` and `async/await`
2. Log the response to the console
3. Display the post's title and body in HTML elements

Open DevTools Network tab to see the request and response.

8.7.2 Exercise 8.2: Displaying a List (Level 2)

Fetch multiple posts from `https://jsonplaceholder.typicode.com/posts?_limit=5` and:

1. Create an HTML card for each post showing title and body
2. Add proper loading and error states
3. Style the cards with CSS

Handle what happens if the fetch fails.

8.7.3 Exercise 8.3: Dynamic Filtering (Level 3)

Using the JSONPlaceholder posts API:

1. Fetch all posts (or limit to 20)
2. Create a search input field
3. As the user types, filter the displayed posts to those whose title contains the search term
4. Update the display without re-fetching

This combines API fetching with JavaScript event handling and DOM manipulation.

8.7.4 Exercise 8.4: Error Handling Scenarios (Level 4)

Create a page that fetches from an API and handles all these scenarios gracefully:

1. Successful fetch Display data
2. Network error (try an invalid URL) Show connection error message
3. 404 error (fetch a non-existent resource) Show “not found” message
4. Loading state Show spinner while waiting

Test each scenario and document how you handled them. Take screenshots showing each state.

8.7.5 Exercise 8.5: API Integration Proposal (Level 5)

A local gym wants to display class schedules from their booking system on their website. The booking system has an API.

Research and propose:

1. What data would you need from the API?
2. How would you display it? (Sketch the UI)
3. What loading states and error handling are needed?
4. How often should data refresh?
5. What happens if the API is unavailable?

Write a 400-word proposal. Discuss your approach with your AI partner, what did you miss?

8.8 Chapter Summary

- APIs enable systems to communicate through defined contracts
- `fetch()` with `async/await` makes readable asynchronous code
- JSON is the standard data format for web APIs
- Always handle loading states and errors
- Check `response.ok`, fetch doesn't throw on HTTP errors
- CORS protects users but can block legitimate requests
- APIs connect your site to the broader ecosystem of services

8.9 Reflection

Before moving to the Portfolio Project, ensure you can:

- ☐ Explain what an API is without technical jargon
- ☐ Write a `fetch()` request using `async/await`
- ☐ Parse JSON responses and access nested data
- ☐ Handle loading states appropriately
- ☐ Implement error handling with `try/catch`
- ☐ Explain the difference between GET and POST
- ☐ Articulate why APIs matter for business applications

8.10 Your Learning Journal

Record your responses to these prompts:

1. **API Discovery**: Find three public APIs that could be useful for business websites. What data do they provide? What would you build with them?

2. **Error Empathy**: Think about times you've seen error messages on websites. Which were helpful? Which were frustrating? How would you do better?

3. **AI Conversation Reflection**: What API concept was most confusing? What question to your AI partner helped clarify it?

4. **Business Integration**: Think of a local business. What data might they want to display from external sources? What APIs might provide it?

8.11 Next Steps

You've now completed the Web Foundations chapters. You understand:

- **Structure** (HTML): What content exists
- **Presentation** (CSS): How content looks
- **Behaviour** (JavaScript): How content responds
- **Connection** (APIs): How content integrates with external services

In Chapter 9, you'll combine these skills to build a complete portfolio website, your first substantial web project. This portfolio will demonstrate your capabilities to future employers or clients.

Then, in the content management chapters, we shift focus from building pages to managing content at scale with content management systems.

Chapter 9

Project: Portfolio Website

> **Example implementations available online**
>
> The online edition at books.borck.education (https://books.borck.education) includes links to example implementations, starter code, and companion repositories for each project.

9.1 Project Overview

Your first substantial project brings together the Web Foundations chapters: Structure (HTML), Presentation (CSS), Behaviour (JavaScript), and Connection (APIs). You'll build a personal portfolio website, a professional asset that demonstrates your capabilities to future employers or clients.

Treat it as a career asset as well as an assignment. A well-crafted portfolio helps launch a career in web development or digital business.

9.2 Learning Outcomes Addressed

- **ULO 1**: Design, implement, and evaluate effective business web applications
- **ULO 3**: Analyse stakeholder requirements to inform design decisions
- **ULO 5**: Evaluate and apply appropriate web technologies

9.3 The Business Case

Consider this project from a business perspective. Your portfolio serves multiple stakeholders:

- **Potential employers** want to see what you can build and how you think
- **Clients** want confidence that you understand professional standards
- **You** need a platform to show growth over time

Ask yourself: What does your portfolio need to communicate? Not just "I can code" but "I understand business needs, I write clean code, I think about users."

9.4 Requirements

9.4.1 Functional Requirements

Your portfolio must include:

1. **Home page**: Clear introduction establishing who you are and what you do
2. **About section**: Your background, skills, and professional interests
3. **Projects section**: At least three projects or work samples with descriptions
4. **Contact information**: How someone can reach you professionally
5. **Responsive design**: Works well on mobile, tablet, and desktop
6. **API integration**: At least one meaningful use of external data

9.4.2 Technical Requirements

Requirement	Standard
HTML	Semantic HTML5 (appropriate use of `<header>`, `<main>`, `<section>`, `<article>`, `<footer>`, etc.)
CSS	Mobile-first responsive design with Flexbox/Grid
JavaScript	At least one interactive feature using event listeners
API	At least one API call displaying external data
Accessibility	Basic WCAG 2.1 AA compliance (semantic structure, alt text, colour contrast, keyboard navigation)
Version control	Git repository with meaningful commit history
Deployment	Live URL (GitHub Pages, Netlify, or similar)

9.4.3 What Counts as API Integration?

Choose one or more:

- **GitHub API**: Display your repositories or contribution activity
- **Weather API**: Show local weather (perhaps themed to where you're based)
- **Quote API**: Display rotating inspirational or industry quotes
- **JSON data file**: If external APIs are problematic, create your own JSON file of project data and fetch it
- **Other**: Any public API that adds genuine value (not just demonstrating you can fetch data)

The integration should serve a purpose for someone viewing your portfolio, beyond proving you can make an API call.

9.5 Project Approach

9.5.1 Phase 1: Planning (Before Any Code)

1. **Define your audience**: Who will view this portfolio? What do they need?
2. **Content inventory**: List all the content you'll need (text, images, project descriptions)
3. **Structure outline**: Sketch your page structure, what sections, what hierarchy?
4. **Visual direction**: What atmosphere suits your professional identity?

```
Ask your AI:
Help me plan a portfolio website structure. I want
to showcase myself
as [your professional focus]. What sections should
I include, and
what should each communicate to a potential
employer?
```

9.5.2 Phase 2: HTML Structure

Build your complete HTML structure before styling:

1. Create semantic markup for all pages/sections
2. Validate your HTML (W3C Validator)
3. Test that the content makes sense without any CSS
4. Ensure heading hierarchy is logical (h1 □ h2 □ h3, no skipping)

This phase should produce an "ugly but functional" page, all content present, properly structured, but unstyled.

9.5.3 Phase 3: CSS Styling

Apply styles mobile-first:

1. Base styles (typography, colours, spacing)
2. Mobile layout (single column, readable)
3. Tablet breakpoint (adjust layout as needed)
4. Desktop breakpoint (multi-column where appropriate)
5. Interactive states (hover, focus)

```
Ask your AI:
Review this CSS for my portfolio. Am I following
mobile-first
principles? What could be simplified or improved?
```

9.5.4 Phase 4: JavaScript Interactivity

Add behaviour that enhances the experience:

1. Navigation toggle for mobile
2. Smooth scrolling to sections (optional)
3. Form validation (if you have a contact form)
4. Any other interactive features

Remember progressive enhancement: your site should work without JavaScript.

9.5.5 Phase 5: API Integration

Implement your API feature:

1. Choose an appropriate API
2. Handle loading states
3. Handle errors gracefully
4. Display data meaningfully

9.5.6 Phase 6: Testing and Refinement

1. Test on multiple devices (or device emulators)
2. Test with keyboard navigation only
3. Run accessibility audit (Lighthouse, WAVE)
4. Fix any issues found
5. Get feedback from peers

9.6 AI Collaboration Guidelines

9.6.1 How to Use AI Effectively

This project is an opportunity to practice the collaboration skills from Chapter 1. Use AI to:

- Explore approaches before committing
- Debug issues you can't resolve
- Get feedback on code quality
- Understand why something works, not just that it works

Do not use AI to:

- Generate your entire project and submit it
- Write content about yourself (you know yourself better)
- Skip understanding what the code does

9.6.2 Documentation Requirements

Keep an AI collaboration log documenting:

1. **Prompts that helped**: What questions got useful responses?
2. **Modifications you made**: How did you adapt AI suggestions?
3. **AI errors or limitations**: What did AI get wrong? How did you know?
4. **Learning moments**: What did the collaboration teach you?

This log demonstrates critical thinking: it shows you directing the AI rather than copying its output.

9.6.3 Example Log Entry

```
Prompt: "Show me how to create a responsive
navigation that collapses
to a hamburger menu on mobile"

AI Response: Provided HTML, CSS, and JavaScript
for a toggle menu.

What I Modified:
- Changed class names to match my naming
convention
- Added ARIA attributes for accessibility (AI
omitted these)
- Simplified the CSS transitions

What AI Got Wrong:
```

```
- Used var instead of const/let
- Didn't include focus states for keyboard users

What I Learned:
- The toggle pattern uses classList.toggle() which
I hadn't used before
- Need to remember accessibility attributes AI
often forgets
```

9.7 Evaluation Criteria

9.7.1 Structure and Semantics (20%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
HTML semantics	Excellent use of semantic elements; clear document outline	Good semantic structure with minor issues	Basic semantic elements present	Mostly `<div>` elements; poor semantics
Heading hierarchy	Logical hierarchy throughout; single h1	Mostly logical; minor inconsis-tencies	Headings present but some skipped levels	Headings used for styling, not structure
Document validity	Valid HTML5; no errors	Minor validation warnings	Some validation errors	Multiple validation errors

9.7.2 Presentation and Responsiveness (25%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Mobile-first CSS	Clear mobile-first approach; logical breakpoints	Mostly mobile-first with some exceptions	Responsive but desktop-first	Not responsive or broken on devices

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Visual design	Professional appearance; clear hierarchy; consistent styling	Attractive design; minor inconsistencies	Functional design; some visual issues	Unprofessional or inconsistent styling
Layout	Effective use of Flexbox/Grid; balanced whitespace	Good layouts; minor spacing issues	Basic layouts work	Layout breaks or looks cramped

9.7.3 Interactivity and JavaScript (20%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
JavaScript functionality	Multiple interactive features working well	At least one feature working well	Basic interactivity present	JavaScript errors or non-functional
Code quality	Clean, well-organised code; uses const/let	Mostly clean; minor issues	Functional but messy	Disorganised or copied without understanding
Progressive enhancement	Site fully works without JS	Core content works without JS	Some functionality lost without JS	Site unusable without JS

9.7.4 API Integration (15%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Implementation	API data displayed meaningfully; adds value	API working; reasonable use	API works but minimal value	API not working or missing

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Error handling	Loading states, error messages, graceful fallbacks	Loading and error states present	Some error handling	No error handling

9.7.5 Accessibility and Best Practices (10%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Accessibility	Keyboard navigable; good contrast; alt text; ARIA where needed	Most accessibility basics met	Some accessibility consideration	Major accessibility barriers
Performance	Fast loading; optimised images	Reasonable performance	Some performance issues	Slow or bloated

9.7.6 AI Collaboration and Reflection (10%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Collaboration log	Detailed log showing critical engagement; modifications documented	Good documentation; shows thinking	Basic log with limited reflection	Missing or superficial log
Reflection	Insightful reflection on process and learning	Good reflection on key learnings	Basic reflection	Missing or generic reflection

9.8 Submission Checklist

Before submitting, verify:

Technical

- ☐ Live URL works and all pages/sections accessible
- ☐ Site works on mobile, tablet, and desktop
- ☐ No JavaScript errors in console
- ☐ API integration functioning
- ☐ HTML validates (W3C Validator)
- ☐ Git repository has meaningful commit history

Content

- ☐ All required sections present (Home, About, Projects, Contact)
- ☐ Content is professional and represents you well
- ☐ Images have alt text
- ☐ No placeholder or lorem ipsum text

Documentation

- ☐ AI collaboration log complete
- ☐ Reflection document (500-750 words) covering:
  - Your development process
  - Challenges encountered and how you solved them
  - What you learned
  - How AI collaboration helped (or didn't)
  - What you would do differently

9.9 What to Submit

1. **Live URL**: Where your portfolio can be viewed
2. **Repository URL**: Your GitHub (or similar) repository
3. **AI collaboration log**: Document (PDF or Markdown)
4. **Reflection**: 500-750 word document

9.10 Getting Started

Begin with these conversations with your AI partner:

```
Ask your AI:
I'm planning a portfolio website to showcase my
web development
learning. Help me think through what sections I
need and what each
should communicate to someone considering hiring a
junior developer.
```

```
Ask your AI:
What makes a portfolio stand out? Not visually
flashy features, but
professional qualities that employers actually
notice. What should
I prioritise?
```

```
Ask your AI:
I want to integrate an API into my portfolio. Help
me evaluate
options: GitHub API vs a weather API vs a quotes
API. What would
make each one add genuine value versus feel
gimmicky?
```

This portfolio is yours. AI can help you build it, but your decisions, your content, and your professional identity should drive it.

9.11 Common Pitfalls to Avoid

1. **Over-designing**: Simple and professional beats flashy and distracting
2. **Under-writing**: "Check out my projects" tells employers nothing; explain what you did and learned
3. **Ignoring mobile**: Test on actual devices, not just browser resize
4. **Forgetting accessibility**: Keyboard navigation, colour contrast, alt text
5. **Copying AI output blindly**: If you can't explain it, you shouldn't submit it
6. **Leaving console errors**: Always check the browser console
7. **Placeholder content**: "Lorem ipsum" signals incompleteness

9.12 After Submission

Your portfolio is a living document. After this project:

- Update it as you complete new work
- Refine based on feedback
- Keep the code clean and the content current

Keep improving the portfolio after you submit it.

Part IV

Content Management for Business

Chapter 10

Why CMS? The Business Value of Managed Content

10.1 The Concept First

In the Web Foundations chapters, you built websites by writing code. Every heading, every paragraph, every image, you placed them precisely where they belonged. You had complete control.

Most business websites need regular content updates, and most content updates shouldn't require a developer.

Consider a small business website. The owner wants to:

- Add a new service offering next week
- Update prices when costs change
- Post news about an upcoming event
- Change the phone number when they move offices

If every change requires editing HTML files and redeploying, that business has a problem. They either pay a developer for simple updates or let their site become stale.

A **Content Management System (CMS)** solves this by separating content from code. The developer builds the structure and design once. Then, the business owner, without touching code, can create, edit, and publish content through a friendly interface.

That makes a CMS a **business architecture decision**, one that affects costs, maintenance, and who controls what.

10.2 Understanding Through Roles

Think about how a newspaper works:

- **Journalists** write articles. They focus on content quality, not page layout.
- **Editors** review and approve content before publication.
- **Designers** create the newspaper's visual templates and structure.
- **Printers** produce the physical paper at scale.

Each role has its own expertise and focuses on its domain. The journalist doesn't need to understand printing presses; they need a system that accepts their article and handles everything else.

A CMS creates similar role separation for websites:

- **Content creators** write and manage content through an interface
- **Administrators** control permissions, workflows, and settings
- **Developers** build themes, extend functionality, and handle technical concerns
- **Visitors** see the published result

Content creators never see code. Developers rarely touch content. Each role works in their comfort zone, and the system coordinates.

> The Handoff Test
>
> A website is truly "done" when you can hand it to the client and they can manage their own content. If they need to call a developer for every text change, the project succeeded technically but failed practically.

10.3 Discovering CMS Concepts with Your AI Partner

10.3.1 Exploration 1: Build vs Buy

When should a business build a custom-coded site versus use a CMS? This is a foundational business decision.

```
Ask your AI:
When should a business build a completely custom
website versus using
a CMS like WordPress? Create a decision framework
with criteria like
budget, content volume, update frequency, and
technical resources.
```

This conversation should reveal that CMS becomes increasingly valuable as:

- Content updates become more frequent
- More non-technical people need to contribute
- The site needs to scale without developer intervention
- Long-term maintenance costs matter

Follow up:

```
Continue the conversation:
What are the trade-offs? What do you lose by using
a CMS instead
of building everything custom?
```

10.3.2 Exploration 2: Stakeholder Perspectives

Different people care about different things. Understanding stakeholder needs is essential for recommending solutions.

```
Ask your AI:
In a website project for a small business, who are
all the
stakeholders? What does each stakeholder need, and
what are their
potential concerns about using a CMS?
```

Stakeholders typically include:

- Business owner (wants control, worries about costs)
- Marketing team (wants easy publishing, worries about limitations)
- IT/Developer (wants maintainability, worries about security)
- End users (want good experience, don't care how it's built)

```
Continue the conversation:
How might these stakeholders have conflicting
needs? How would you
balance them when recommending a CMS solution?
```

10.3.3 Exploration 3: Content Strategy First

Technology should serve content, not the other way around. Content strategy asks: What content do we need? Who creates it? How is it organised?

```
Ask your AI:
Explain content strategy for a small business
website. What questions
should we answer before choosing any technology?
```

```
Walk me through how
content strategy informs CMS requirements.
```

This should reveal that technology decisions come *after* understanding:

- What content types exist (pages, posts, products, events)
- Who creates and approves content
- How often content changes
- How content relates to business goals

10.4 From Concept to Code (Understanding CMS Architecture)

While this chapter focuses on concepts, let's understand what a CMS does technically.

10.4.1 The Core Separation

Without a CMS:

```
HTML File → Browser
(Content is embedded in code)
```

With a CMS:

```
Database (Content) + Templates (Design) → CMS
Engine → HTML → Browser
(Content and presentation are separate)
```

The CMS combines content from a database with design templates to generate the HTML that browsers display. Change the content in the database, and the generated HTML changes, without touching any code.

10.4.2 CMS Types

Not all CMSs work the same way:

Traditional CMS (Coupled)

- Content management and website delivery are one system
- Examples: WordPress, Drupal, Joomla
- Good for: Standard websites, blogs, small-medium businesses
- The CMS generates and serves the web pages

Headless CMS (Decoupled)

- Content management is separate from content delivery
- Content is accessed via API, displayed by a separate frontend
- Examples: Contentful, Strapi, Sanity
- Good for: Multi-channel content (web, mobile, kiosk), developer-led projects
- We'll explore this in Chapter 13

Website Builders (All-in-one)

- Simplified CMS with built-in design tools
- Examples: Squarespace, Wix, Webflow
- Good for: Simple sites, non-technical users, quick launches
- Limited flexibility but minimal technical requirement

10.4.3 WordPress: The Dominant CMS

WordPress powers over 40% of all websites: nearly half of the web runs on this one platform.

Why? Because it hits a sweet spot:

- **Free and open source**: No licensing costs
- **Massive ecosystem**: Thousands of themes and plugins
- **Flexible**: Blogs, business sites, e-commerce, anything
- **Accessible**: Non-developers can manage content
- **Extensible**: Developers can customise deeply

We'll spend the WordPress chapters learning to use WordPress properly, evaluate it professionally, and extend it when needed.

10.4.4 The CMS Decision Framework

When evaluating whether a project needs a CMS:

Factor	Suggests CMS	Suggests Custom Code
Content updates	Frequent (weekly+)	Rare (yearly)
Content editors	Non-technical staff	Developers only
Content volume	Growing over time	Fixed/static
Budget for updates	Limited ongoing budget	Developer always available
Time to launch	Faster with templates	Longer but fully custom

Factor	Suggests CMS	Suggests Custom Code
Unique requirements	Standard patterns work	Highly unusual needs

```
Ask your AI:
A local restaurant wants a website with their
menu, location, and
hours. They update the menu monthly. The owner
isn't technical.
Their budget is limited. Walk me through whether
they need a CMS
and what type might be appropriate.
```

10.5 Building Your Mental Model

10.5.1 Content as Structured Data

In a CMS, content is structured data with defined fields:

```
Blog Post:
  Title (text, required)
  Content (rich text)
  Author (relationship to User)
  Category (relationship to Category)
  Featured Image (media)
  Publish Date (date)
  Status (draft/published)
```

This structure enables:

- Consistent presentation (every post has the same fields)
- Searchability (find all posts by author)
- Relationships (posts belong to categories)
- Validation (required fields must be filled)

```
Ask your AI:
For a small business website with services, team
members, and
testimonials, what content types would I need?
What fields would
each content type have?
```

10.5.2 The Template Layer

Templates define how structured content becomes visual HTML:

```
Template: "Single Blog Post"

  {{ post.featured_image }}
  <h1>{{ post.title }}</h1>
  <p>By {{ post.author.name }}</p>
  {{ post.content }}
```

The template pulls fields from the content and arranges them. Change the template, and every post's appearance changes. Change one post's content, and only that post changes.

This separation lets designers work on templates while writers work on content, and neither needs to understand the other's domain.

10.5.3 The Permission Layer

CMSs control who can do what:

- **Administrator**: Full access to everything
- **Editor**: Can manage all content but not settings
- **Author**: Can write and publish their own content
- **Contributor**: Can write but not publish (needs approval)

These roles prevent accidents ("The intern deleted the homepage") and enable workflows ("All posts need editor approval before publishing").

10.6 Business Applications

10.6.1 Client Handoff

A website project succeeds when the client can maintain the site independently after launch. A CMS-based site can be handed off with training:

"Here's how you add a new blog post. Here's how you update your hours. Here's how you add a team member."

Custom-coded sites often create ongoing dependency.

10.6.2 Reduced Maintenance Costs

Every developer-required change has a cost. With a CMS:

- Content changes: Client handles directly ($0)
- Design tweaks: Occasional developer work
- Major features: Developer project

Without a CMS:

- Content changes: Developer required ($$$)
- Everything requires technical involvement

Over years, this difference compounds.

10.6.3 Content Governance

For organisations with multiple content contributors, governance matters:

- Who can publish without approval?
- Who reviews content before it goes live?
- Who can delete pages?
- Who can access sensitive sections?

A CMS with proper roles and workflows provides this structure. Custom code rarely includes such considerations.

10.6.4 Scalability

Need to add 50 new product pages? With a CMS:

1. Create a "Product" content type with appropriate fields
2. Content team adds 50 products through the interface
3. Products appear on the site automatically

Without a CMS: a developer writes and deploys 50 pages of HTML.

> **Skills You Are Building**
>
> This develops **translating stakeholder needs into technical requirements** and **making and defending technology choices**. Recommending a CMS isn't a technical decision, it's understanding who needs to do what and choosing architecture accordingly.

10.7 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

10.7.1 Exercise 10.1: CMS Identification (Level 1)

Visit five local business websites. For each:

1. Try to identify if they're using a CMS (check for `/wp-admin/`, `?p=123` URLs, or use tools like BuiltWith)
2. If CMS, which one?
3. Does the site appear actively maintained?

Document your findings. What patterns do you notice about businesses that use CMS versus those that don't?

10.7.2 Exercise 10.2: Stakeholder Analysis (Level 2)

For a hypothetical dental practice website, identify:

1. All stakeholders (beyond just the dentist)
2. What each stakeholder needs from the website
3. What each stakeholder would need to be able to do themselves
4. Potential conflicts between stakeholder needs

Create a stakeholder matrix mapping needs to CMS requirements.

10.7.3 Exercise 10.3: Content Audit (Level 3)

Choose a small business website (could be a local business or a fictional scenario). Conduct a content audit:

1. List every content type on the site (pages, posts, products, etc.)
2. For each type, define the fields it would need
3. Estimate how often each content type changes
4. Identify who would likely update each type

Based on your audit, would you recommend a CMS? Why or why not?

10.7.4 Exercise 10.4: CMS Recommendation (Level 4)

A boutique fitness studio approaches you. They want a website with:

- Class schedules (change weekly)
- Instructor profiles (change occasionally)
- Pricing information (change quarterly)
- Blog posts about fitness tips (new posts monthly)
- Photo gallery (updated after events)

The owner is comfortable with computers but not technical. Budget is moderate. They want to manage most updates themselves.

Write a 300-word recommendation addressing:

- Whether they need a CMS
- What type of CMS would suit them
- What content types you'd create
- What the client could manage versus what needs developer involvement

10.7.5 Exercise 10.5: Build vs Buy Analysis (Level 5)

A tech startup wants a unique, highly interactive product landing page with animated demonstrations, custom calculators, and unusual layouts. They update content quarterly and have developers on staff.

Another client, a law firm, wants a professional site with attorney profiles, practice areas, blog, and contact forms. They update content weekly and have no technical staff.

For each:

1. Argue the case for using a CMS
2. Argue the case against using a CMS
3. Make and defend your recommendation

This exercise has no single correct answer. Develop your ability to weigh trade-offs and justify decisions (making and defending technology choices).

10.8 Chapter Summary

- A CMS separates content from code, enabling non-technical users to manage websites
- Choosing a CMS is a business architecture decision as well as a technical one
- Different stakeholders have different needs; CMS choice should balance these

- Content strategy (what content, who creates it, how often it changes) precedes technology choice
- WordPress dominates the CMS market due to its flexibility, ecosystem, and accessibility
- Role-based permissions enable content governance and workflow control

10.9 Reflection

Before moving on, ensure you can:

☐ Explain what a CMS does without using technical jargon
☐ Identify when a project would benefit from a CMS versus custom development
☐ List at least three different stakeholder roles and their CMS-related needs
☐ Describe the difference between traditional and headless CMS
☐ Explain why WordPress is so widely used
☐ Articulate the long-term cost implications of CMS versus custom code

10.10 Your Learning Journal

Record your responses to these prompts:

1. **CMS Encounters**: Think about websites you use regularly. Which are clearly CMS-powered? How can you tell? What does this suggest about when CMS makes sense?

2. **Stakeholder Empathy**: Put yourself in the shoes of a non-technical business owner. What would frustrate you about needing to contact a developer for every content change? What would you want from a content management interface?

3. **AI Conversation Reflection**: What question about CMS selection did your AI partner help clarify? What factors hadn't you considered before the conversation?

4. **Business Thinking**: How does thinking about "who updates what" change how you approach web projects? Why might this question be more important than "what technology should we use"?

10.11 Next Steps

You now understand why CMS matters and when to use one. In Chapter 11, we'll look closely at WordPress as a professional platform.

We'll set up a local development environment, understand WordPress's architecture, and learn to evaluate it critically. You will learn how the platform works and when it is the right (or wrong) choice for a project.

Chapter 11

WordPress as Platform

11.1 The Concept First

WordPress powers over 40% of the web: nearly half of all websites run on a single platform. Understanding why reveals something important about how technology decisions are made in business.

Treat WordPress as a **platform**, a foundation that others build upon:

- **Software** does specific things. You use it as-is.
- **A platform** provides a foundation. An ecosystem builds on top of it.

Your smartphone is a platform. The core provides basics (calls, settings), but the app ecosystem gives it most of its capability. You don't judge your phone by its built-in apps alone, you judge it by what you can do with the apps others have built.

WordPress works the same way. The core handles content management. Themes control appearance. Plugins add functionality. The ecosystem, over 60,000 plugins, thousands of themes, is what makes WordPress suitable for almost any website.

Learning WordPress means learning to evaluate and assemble these pieces professionally.

11.2 Understanding Through Ecosystems

Imagine opening a restaurant. You could:

Option A: Build everything from scratch. Design your own stoves, create your own recipes, invent your own point-of-sale system.

Option B: Use standard commercial kitchen equipment, adapt proven recipes, install established POS software, then focus your creativity on your unique menu and dining experience.

Option B gets you open faster, with tested components, and lets you focus on what makes your restaurant special.

WordPress provides the kitchen equipment. Themes provide the interior design. Plugins provide specialised tools (reservation systems, online ordering). You focus on your content and business logic.

> **The Platform Evaluation Question**
>
> When evaluating any platform, ask: "How active is the ecosystem?" A platform with a dying ecosystem becomes a liability. WordPress's ecosystem is massive and active, a significant factor in its dominance.

11.3 Discovering WordPress with Your AI Partner

11.3.1 Exploration 1: Platform vs Software

Let's clarify the distinction between platforms and ordinary software:

```
Ask your AI:
What makes something a "platform" versus just
"software"? Compare
WordPress and Microsoft Word. Both help create
documents. Why is
WordPress a platform while Word is (primarily)
software?
```

This should reveal characteristics of platforms:

- Third parties build on them
- An ecosystem of extensions exists
- The value increases as the ecosystem grows
- Users can customise without changing the core

```
Continue the conversation:
What risks come with choosing a platform? What
happens if the
ecosystem declines or the platform changes
direction?
```

11.3.2 Exploration 2: Theme and Plugin Evaluation

The WordPress ecosystem contains gems and garbage. Professional evaluation skills are essential:

```
Ask your AI:
How do I evaluate whether a WordPress theme or
plugin is trustworthy
and well-built? What specific signals indicate
quality, and what red
flags should make me cautious?
```

Quality signals include:

- Active development (recent updates)
- Large install base with good reviews
- Responsive support
- Clean code (if you can inspect it)
- Clear documentation

Red flags include:

- No updates in over a year
- Few installations despite being old
- Poor reviews mentioning security or support
- Abandoned by developer
- Excessive permission requests

```
Continue the conversation:
If I find two plugins that do the same thing, one
with 100,000
installs and one with 1,000 installs, is the
popular one always
better? When might the less popular option be the
right choice?
```

11.3.3 Exploration 3: WordPress Architecture

Understanding how WordPress is built helps with troubleshooting and customisation:

```
Ask your AI:
Explain WordPress's architecture using a
restaurant analogy. The
database stores something, themes do something,
```

```
plugins do something,
and the WordPress core coordinates. What's each
part doing in
restaurant terms?
```

A typical analogy might map:

- Database = Storage (pantry, inventory, recipes)
- Core = Kitchen operations (combining ingredients into dishes)
- Themes = Dining room design (how food is presented)
- Plugins = Specialised equipment (espresso machine, wine fridge)

```
Continue the conversation:
When something goes wrong on a WordPress site, how
does understanding
architecture help diagnose the problem? Give me an
example.
```

11.4 From Concept to Code

Let's set up WordPress locally and understand its core components.

11.4.1 Local Development with LocalWP

Never develop directly on a live website. Local development means:

- You can experiment without breaking anything
- You don't need internet access
- Changes are instant (no upload time)
- You can test risky changes safely

LocalWP (formerly Local by Flywheel) is the simplest way to run WordPress locally:

1. Download from localwp.com
2. Install and open
3. Click "Create a new site"
4. Choose a name, username, and password
5. Wait for setup to complete

You now have a WordPress site running on your computer.

LocalWP handles PHP, MySQL, and the web server behind a simple interface.

> **Alternatives**
>
> Other local development options include XAMPP, MAMP, and Docker-based setups. LocalWP is recommended for beginners because it's WordPress-specific and handles configuration automatically.

11.4.2 The WordPress Admin Interface

Click "WP Admin" in LocalWP to access your site's dashboard. This is the content management interface, where non-developers spend their time.

Dashboard Areas:

Area	Purpose
Dashboard	Overview and quick actions
Posts	Blog posts and news
Media	Images, files, uploads
Pages	Static pages (About, Contact)
Comments	Reader comments (if enabled)
Appearance	Themes, menus, widgets
Plugins	Add functionality
Users	Account management
Settings	Site configuration

Spend time exploring each area. Understanding the admin interface is essential, this is what you'll train clients to use.

11.4.3 Posts vs Pages

WordPress has two main content types by default:

Posts are:

- Time-based (have publish dates)
- Categorised and tagged
- Appear in feeds and archives
- Good for: Blog articles, news, updates

Pages are:

- Timeless (no dates emphasised)
- Hierarchical (can have parent pages)
- Not categorised
- Good for: About, Contact, Services, static content

The distinction is organisational rather than technical. Posts answer "What's new?" Pages answer "What do we do?"

11.4.4 Understanding Themes

A WordPress theme controls:

- Visual appearance (colours, fonts, layout)
- Template structure (what appears where)
- Available features (some themes add functionality)

The theme doesn't contain your content, it presents your content. Change themes, and content remains while appearance changes.

Exploring Themes:

1. Go to Appearance Themes
2. Click "Add New"
3. Browse featured, popular, or search for specific needs
4. Use "Live Preview" to see a theme with your content
5. Activate when ready

Theme Evaluation Criteria:

Factor	Questions to Ask
Purpose fit	Is this designed for sites like yours?
Responsiveness	Does it work well on mobile?
Speed	Is it lightweight or bloated?
Customisation	Can you change colours/fonts without code?
Updates	Is it actively maintained?
Reviews	What do other users say?
Support	Can you get help if needed?

```
Ask your AI:
I'm building a site for a local bakery. What
should I look for in a
WordPress theme? What features matter, and what
should I avoid?
```

11.4.5 The Template Hierarchy

WordPress uses a template hierarchy to determine which template file displays content. Understanding this helps when you need to customise.

```
Specific
General


Single Post:
single-post-{slug}.php → single-post.php →
single.php → singular.php → index.php

Single Page:
page-{slug}.php → page-{id}.php → page.php →
singular.php → index.php

Archive:
category-{slug}.php → category.php → archive.php →
index.php
```

WordPress looks for the most specific template first, falling back to more general ones. This allows themes to have special templates for specific pages while using defaults for everything else.

```
Ask your AI:
If I want my "About" page to look different from
other pages, what
file would I create in my theme? Walk me through
how WordPress
determines which template to use.
```

11.4.6 Basic Customisation

Most modern themes include a Customiser (Appearance Customise) for no-code adjustments:

- Site identity (logo, tagline)
- Colours
- Typography (some themes)
- Header/footer layout
- Homepage settings
- Menus

The Customiser shows live previews, experiment freely before publishing changes.

Menus:

1. Appearance Menus

2. Create a new menu
3. Add pages, posts, custom links, or categories
4. Arrange by dragging
5. Assign to a theme location (Primary, Footer, etc.)

Widgets (if your theme uses them):

1. Appearance Widgets
2. Drag widgets to sidebar/footer areas
3. Configure each widget's settings

11.4.7 Settings That Matter

In Settings, pay attention to:

General:

- Site Title and Tagline (appears in browser tabs, search results)
- Timezone (affects scheduled posts)

Reading:

- Homepage displays: Latest posts vs. static page
- Posts per page

Permalinks:

- URL structure: Choose "Post name" for clean URLs (`/about/` instead of `/?p=123`)
- Change this early, changing later breaks existing links

Discussion:

- Comment settings (enable/disable, moderation rules)

11.5 Building Your Mental Model

11.5.1 The WordPress Stack

```
Browser (What visitors see)

Theme (Presentation layer)

Plugins (Extended functionality)

WordPress Core (Content engine)
```

```
  PHP + MySQL (Server technology)
```

When something goes wrong, think through these layers:

- Display issue? Probably theme
- Feature not working? Probably plugin
- General malfunction? Possibly core or server
- Can't log in? Database or user settings

11.5.2 Content Separation

A useful mental model: **content lives in the database, presentation lives in the theme**.

```
Database (content):
  Post: "Our Spring Menu"
      Title
      Body content
      Featured image reference
      Categories, tags

Theme (presentation):
  How to display a post
      Where title appears
      Image placement
      Typography, colours
```

This separation is why you can change themes without losing content.

11.5.3 The Hook System

WordPress allows code to "hook" into specific points in the execution flow. While you won't write hooks yet, understanding they exist explains how plugins modify WordPress without changing core files:

- Plugins register functions to run at specific points
- WordPress calls these functions when reaching those points
- Multiple plugins can hook into the same point

This architecture is why WordPress is extensible without modification, a defining platform characteristic.

11.6 Business Applications

11.6.1 Market Demand

WordPress skills are in demand. Whether freelancing, agency work, or in-house roles, WordPress knowledge is marketable because:

- Huge market share means abundant projects
- Businesses have existing WordPress sites needing maintenance
- Lower barrier to entry than custom development
- Clients often specifically request WordPress

11.6.2 Cost Structure

For clients, WordPress offers:

- No licensing fees (open source)
- Lower development costs (existing ecosystem)
- Reduced maintenance costs (client self-service)
- Lower switching costs (many developers available)

11.6.3 Flexibility

The plugin ecosystem means you rarely need custom development:

- E-commerce: WooCommerce
- Forms: Gravity Forms, WPForms, Contact Form 7
- SEO: Yoast, RankMath
- Security: Wordfence, Sucuri
- Backup: UpdraftPlus, VaultPress
- Caching: WP Super Cache, W3 Total Cache

Often, the professional skill is knowing which plugins to combine, not building from scratch.

11.6.4 When WordPress Isn't Right

Professional judgment includes knowing when *not* to use WordPress:

- Highly custom web applications (better: custom framework)
- Real-time features (better: specialised platforms)
- Very simple one-page sites (better: static HTML or website builders)
- Maximum performance requirements (better: headless architecture)

Skills You Are Building

This develops **making and defending technology choices** and **assessing emerging technologies**. Choosing WordPress requires evaluating its ecosystem, understanding its architecture, and articulating why it's appropriate (or not) for specific situations.

11.7 Practice Exercises

Exercise Levels

- **Level 1**: Direct application
- **Level 2**: Minor modifications
- **Level 3**: Combining concepts
- **Level 4**: Problem-solving
- **Level 5**: Open-ended design

11.7.1 Exercise 11.1: Local Setup (Level 1)

Install LocalWP and create a WordPress site called "Practice Site":

1. Document the process with screenshots
2. Log into the admin dashboard
3. Create one page and one post
4. Change the permalink structure to "Post name"
5. Upload an image to the Media Library

Verify everything works by viewing the site in your browser.

11.7.2 Exercise 11.2: Theme Exploration (Level 2)

Using your Practice Site:

1. Install and preview three different free themes
2. For each theme, document:
    - What kind of site it seems designed for
    - What customisation options it offers
    - How it handles responsiveness (test at different sizes)
3. Choose one and activate it
4. Use the Customiser to adjust site identity and colours

11.7.3 Exercise 11.3: Content Structure (Level 3)

Create content for a fictional coffee shop on your Practice Site:

1. Create pages: Home, About, Menu, Contact
2. Create a menu in Appearance Menus and assign it
3. Set the homepage to display a static page
4. Create three blog posts about coffee (news, events, tips)
5. Assign the posts to categories you create

Evaluate: Is the content discoverable? Can a visitor navigate easily?

11.7.4 Exercise 11.4: Theme Evaluation Report (Level 4)

Research three WordPress themes suitable for a professional services business (lawyer, accountant, consultant). For each:

1. Document key features
2. Check last update date and developer responsiveness
3. Read at least 5 reviews
4. Test the demo for mobile responsiveness
5. Identify potential limitations

Write a 400-word recommendation explaining which theme you'd choose and why. Discuss trade-offs.

11.7.5 Exercise 11.5: Platform Comparison (Level 5)

A client asks: "Should I use WordPress or Squarespace for my boutique hotel website?"

Research both platforms and write a balanced comparison (500 words) addressing:

1. Cost (initial and ongoing)
2. Ease of use for the client
3. Design flexibility
4. Booking system integration options
5. Long-term maintenance considerations
6. Your recommendation with justification

This exercise develops technology evaluation skills (assessing emerging technologies).

11.8 Chapter Summary

- WordPress is a platform whose ecosystem is much of its value

- Professional evaluation of themes and plugins is an essential skill
- Local development with LocalWP provides a safe environment to learn
- Understanding WordPress architecture (core, themes, plugins, database) aids troubleshooting
- The template hierarchy determines which template displays content
- WordPress isn't always the right choice; professional judgment matters

11.9 Reflection

Before moving on, ensure you can:

☐ Explain the difference between a platform and software
☐ Set up WordPress locally using LocalWP
☐ Navigate the WordPress admin interface confidently
☐ Explain the difference between posts and pages
☐ Evaluate a theme's quality using specific criteria
☐ Describe how the template hierarchy works
☐ Articulate when WordPress is and isn't appropriate

11.10 Your Learning Journal

Record your responses to these prompts:

1. **Platform Thinking**: What other platforms do you use daily? What makes their ecosystems valuable? How does this inform your understanding of WordPress?
2. **Theme Exploration**: What surprised you when exploring WordPress themes? What features seem most valuable for business sites?
3. **AI Conversation Reflection**: What question about WordPress architecture or evaluation did your AI partner help clarify?
4. **Professional Judgment**: If a client asked why they shouldn't just use Wix, how would you respond? When *would* Wix be the better choice?

11.11 Next Steps

You now understand WordPress as a platform and can set up and configure a basic site. In Chapter 12, we'll explore how to extend WordPress with plugins, both using existing plugins professionally and understanding how they work.

That is how you customise WordPress for specific business requirements without starting from scratch.

Chapter 12

Extending Platforms

12.1 The Concept First

In Chapter 11, you learned that WordPress's value lies in its ecosystem. Now we'll learn to extend WordPress professionally, using plugins wisely, customising safely, and making business decisions about when to build versus when to buy.

Plugins make WordPress more capable, but they also create risk. Every plugin you add:

- Depends on someone else maintaining it
- Could conflict with other plugins
- Might introduce security vulnerabilities
- Adds complexity to troubleshoot

Professional WordPress development means installing the *right* plugins and knowing when custom development makes more sense.

12.2 Understanding Through Build vs Buy

Every feature request presents a choice:

Buy (Use existing plugin):

- Faster to implement
- Someone else maintains it
- Tested by thousands of users
- May not fit exact requirements

Build (Custom development):

- Exact fit for requirements

- You control maintenance
- No dependency on external developers
- Higher upfront cost

Neither is always better. The professional skill is evaluating trade-offs.

Consider a business that needs a booking system:

Factor	Plugin (e.g., Bookly)	Custom Build
Implementation time	Hours	Weeks
Upfront cost	$0-300	$5,000-20,000
Fits requirements	80-90%	100%
Ongoing updates	Plugin developer	You
Customisation	Limited	Unlimited
Risk of abandonment	Medium	None (you own it)

For most small businesses, the plugin wins. For a booking-centric business with unique requirements, custom might justify its cost.

> **The 80% Rule**
>
> If an existing solution meets 80% of requirements, adapting workflows to fit the tool often costs less than building the remaining 20% custom. Perfect fit isn't always worth the price.

12.3 Discovering Extensions with Your AI Partner

12.3.1 Exploration 1: Plugin Evaluation Framework

When multiple plugins solve the same problem, how do you choose?

```
Ask your AI:
I need a contact form plugin for a WordPress
business site. Create
a framework for evaluating options. What criteria
matter, and how
would I compare Contact Form 7, WPForms, and
Gravity Forms?
```

Your evaluation framework should include:

- Feature match to requirements
- Ease of use (for you and the client)

- Performance impact
- Price (free vs. premium features)
- Support and documentation
- Update frequency and developer reputation
- Integration with other tools

```
Continue the conversation:
What questions should I ask the client before
recommending a contact
form plugin? What requirements might change my
recommendation?
```

12.3.2 Exploration 2: Build vs Buy Decision

Let's work through a realistic scenario:

```
Ask your AI:
A client runs a yoga studio and needs an online
booking system.
Students should book classes, see instructor
schedules, and manage
their memberships. Walk me through the decision
process: should we
use an existing plugin like Bookly, integrate a
third-party service
like Mindbody, or build custom? What factors
matter most?
```

This conversation should reveal:

- Volume of bookings (justifies complexity)
- Unique requirements (standard vs. unusual workflows)
- Integration needs (payment, email, calendars)
- Budget reality (short-term vs. long-term costs)
- Client technical capacity (who maintains this?)

```
Continue the conversation:
What if the client says "I want exactly what my
competitor has but
for half the price"? How do I manage expectations?
```

12.3.3 Exploration 3: Technical Debt

Plugins have hidden costs:

```
Ask your AI:
What is "plugin bloat" in WordPress? How do too
many plugins create
technical debt? Give me a concrete example of how
this affects a
real site.
```

Technical debt accumulates when:

- Plugins conflict with each other
- Updates break functionality
- Performance degrades
- Security vulnerabilities compound
- Nobody understands all the pieces

```
Continue the conversation:
If I inherited a WordPress site with 47 plugins,
how would I audit
which ones are actually needed? What's a safe
process?
```

12.3.4 Exploration 4: AI-Assisted WordPress Development

AI is particularly effective at writing WordPress PHP code:

```
Ask your AI:
I want to add a feature to my WordPress site that
displays "This
post was updated X days ago" at the top of posts
that have been
modified in the last 30 days. Walk me through how
to implement
this with a filter in my child theme's
functions.php. Explain
each part of the code.
```

This conversation should demonstrate:

- How to structure requests for WordPress PHP code
- The importance of asking for explanations, not just code
- How filters modify content in WordPress
- Testing approach for custom PHP

```
Continue the conversation:
Now I want to make this a simple plugin instead of
putting it in
functions.php. What changes are needed? What are
the advantages
of a plugin over functions.php for this feature?
```

12.4 From Concept to Code

Let's learn to extend WordPress professionally.

12.4.1 Installing and Managing Plugins

Installing from the repository:

1. Plugins → Add New
2. Search for the plugin
3. Click "Install Now"
4. Click "Activate"

Installing premium plugins:

1. Download the .zip file from the vendor
2. Plugins → Add New → Upload Plugin
3. Select the .zip file
4. Install and activate

Managing plugins:

- Deactivate plugins you're not using (don't just leave them inactive, delete them)
- Update plugins regularly (but test first on staging)
- Document why each plugin is installed
- Review plugins quarterly, needs change

12.4.2 Essential Plugin Categories

Most business WordPress sites need plugins in these categories:

Security:

- Wordfence or Sucuri: Firewall, malware scanning, login protection
- Essential. WordPress is a common target

Backup:

- UpdraftPlus: Scheduled backups to cloud storage
- Non-negotiable, databases get corrupted, hosts fail

Performance:

- WP Super Cache or W3 Total Cache: Page caching
- Smush or ShortPixel: Image optimisation
- Noticeable impact on user experience and SEO

SEO:

- Yoast SEO or RankMath: Meta tags, sitemaps, readability
- Important for discoverability

Forms:

- WPForms, Gravity Forms, or Contact Form 7
- Nearly every site needs contact forms

```
Ask your AI:
For a small business WordPress site, what's the
minimum set of
plugins I should consider essential? What does
each one protect
against or enable?
```

12.4.3 Evaluating Plugin Quality

Before installing any plugin, check:

In the repository listing:

- Last updated (within 3-6 months)
- Active installations (higher = more tested)
- WordPress version compatibility
- Star rating and reviews (read the negative ones)
- Support forum activity (does developer respond?)

In reviews, watch for:

- Security issues mentioned
- Conflicts with common plugins/themes
- Broken updates
- Abandoned development
- Excessive resource usage

Test before deploying:

- Install on a staging site first

- Test core functionality
- Check for conflicts with existing plugins
- Monitor performance impact

12.4.4 Plugin Conflicts

Plugins can conflict because they:

- Modify the same functionality
- Use incompatible JavaScript libraries
- Compete for database resources
- Override the same hooks

Diagnosing conflicts:

1. Note the symptoms (error messages, broken features)
2. Deactivate all plugins
3. Reactivate one by one, testing each time
4. Identify the conflicting combination
5. Decide: replace one, find alternative, or contact support

Preventing conflicts:

- Minimise plugin count (each one adds risk)
- Choose well-maintained plugins
- Test updates on staging first
- Keep WordPress core updated

12.4.5 Child Themes: Safe Customisation

If you modify a theme directly, your changes disappear when the theme updates. Child themes solve this.

A child theme:

- Inherits everything from the parent theme
- Lets you override specific files
- Preserves your changes through parent theme updates

Creating a child theme:

1. Create a folder in `/wp-content/themes/` named `{parent-theme}-child`
2. Create `style.css` with required headers:

```
/*
 Theme Name: Twenty Twenty-Four Child
 Template: twentytwentyfour
*/
```

```
/* Your custom CSS goes here */
```

3. Create `functions.php` to enqueue parent styles:

```
<?php
function child_theme_enqueue_styles() {
    wp_enqueue_style(
        'parent-style',
        get_template_directory_uri() .
        '/style.css'
    );
}
add_action('wp_enqueue_scripts',
'child_theme_enqueue_styles');
```

4. Activate the child theme in Appearance Themes

Now any CSS you add to the child's `style.css` applies on top of the parent, and any template files you create in the child override the parent's versions.

```
Ask your AI:
I want to change the footer on my WordPress site
to include
additional business information. Walk me through
doing this
properly with a child theme, so my changes survive
theme updates.
```

12.4.6 Functions.php: Small Customisations

For small code customisations, the child theme's `functions.php` is appropriate:

```
<?php
// Remove WordPress version from head (minor
security)
remove_action('wp_head', 'wp_generator');

// Change excerpt length
function custom_excerpt_length($length) {
    return 25; // Words
}
add_filter('excerpt_length',
```

```
'custom_excerpt_length');

// Add custom image size
add_image_size('featured-thumb', 400, 300, true);
```

> ⚠ functions.php Caution
>
> Syntax errors in functions.php can break your entire site. Test on staging, and consider a plugin like Code Snippets that isolates code pieces and lets you disable them if they cause problems.

12.4.7 PHP Fundamentals for WordPress

WordPress is built on PHP. While you can accomplish a lot without writing PHP, understanding the basics opens up more customisation options, and AI is excellent at helping you write WordPress PHP code.

PHP basics:

PHP code runs on the server and generates HTML sent to the browser. PHP files mix HTML and PHP:

```
<?php
// PHP code goes between these tags
$site_name = "My Business";
$current_year = date('Y');
?>

<!DOCTYPE html>
<html>
<head>
    <title><?php echo $site_name; ?></title>
</head>
<body>
    <footer>
        <p>&copy; <?php echo $current_year; ?>
        <?php echo $site_name; ?></p>
    </footer>
</body>
</html>
```

Key PHP concepts:

```php
<?php
// Variables start with $
$price = 29.99;
$product_name = "Widget";
$is_available = true;

// Arrays hold multiple values
$categories = ['Electronics', 'Home', 'Garden'];
$product = [
    'name' => 'Widget',
    'price' => 29.99,
    'stock' => 15
];

// Access array values
echo $categories[0];        // Electronics
echo $product['price'];     // 29.99

// Conditionals
if ($product['stock'] > 0) {
    echo "In stock";
} else {
    echo "Out of stock";
}

// Loops
foreach ($categories as $category) {
    echo "<li>$category</li>";
}

// Functions
function format_price($amount) {
    return '$' . number_format($amount, 2);
}
echo format_price(29.99);  // $29.99
```

12.4.8 WordPress Template Tags

WordPress provides functions (called “template tags”) that output common elements:

```php
<?php
// Site information
bloginfo('name');           // Site title
bloginfo('description');    // Site tagline
home_url();                 // Home page URL
get_template_directory_uri(); // Theme folder URL

// Current page/post
the_title();               // Post/page title
the_content();             // Post/page content
the_excerpt();             // Post excerpt
the_permalink();           // URL to current post
the_post_thumbnail();      // Featured image

// Conditional checks
if (is_home()) { }         // Is this the blog
page?
if (is_single()) { }       // Is this a single
post?
if (is_page('about')) { }  // Is this the About
page?
if (is_logged_in()) { }    // Is user logged in?
```

12.4.9 The WordPress Loop

The Loop is how WordPress displays posts. It's the core pattern you'll see in every theme:

```php
<?php if (have_posts()) : ?>
    <?php while (have_posts()) : the_post(); ?>
        <article>
            <h2><a href="<?php the_permalink();
            ?>"><?php the_title(); ?></a></h2>
            <div class="meta">
                Posted on <?php the_date(); ?> by
                <?php the_author(); ?>
            </div>
            <div class="content">
                <?php the_excerpt(); ?>
            </div>
            <a href="<?php the_permalink();
            ?>">Read more</a>
```

```
        </article>
    <?php endwhile; ?>
<?php else : ?>
    <p>No posts found.</p>
<?php endif; ?>
```

Understanding this pattern helps you customise how posts display.

12.4.10 Hooks: Actions and Filters

WordPress uses "hooks" to let you modify behaviour without editing core files. There are two types:

Actions: do something at a specific point:

```
<?php
// Add code to the <head> section
function add_custom_meta() {
    echo '<meta name="author" content="My
    Company">';
}
add_action('wp_head', 'add_custom_meta');

// Add code to the footer
function add_footer_script() {
    echo '<script>console.log("Page
    loaded");</script>';
}
add_action('wp_footer', 'add_footer_script');

// Do something when a post is published
function notify_on_publish($post_id) {
    // Send notification, update external system,
    etc.
}
add_action('publish_post', 'notify_on_publish');
```

Filters: modify data before it's used:

```
<?php
// Modify the excerpt length
function custom_excerpt_length($length) {
    return 30; // 30 words instead of default 55
```

```
}
add_filter('excerpt_length',
'custom_excerpt_length');

// Add content after every post
function add_author_bio($content) {
    if (is_single()) {
        $content .= '<div
        class="author-bio">Written by ' .
        get_the_author() . '</div>';
    }
    return $content;
}
add_filter('the_content', 'add_author_bio');

// Modify the page title
function custom_title($title) {
    return $title . ' | My Business';
}
add_filter('the_title', 'custom_title');
```

```
Ask your AI:
I want to add a "Last updated" date below each
blog post title on
my WordPress site. Walk me through how to do this
with a filter in
my child theme's functions.php.
```

12.4.11 Building a Simple Plugin with AI

AI is good at writing WordPress plugins.

Example: A plugin to display business hours

```
Ask your AI:
I need a WordPress plugin that:
1. Adds a settings page where I can enter business
hours for each day
2. Provides a shortcode [business_hours] to
display the hours
3. Shows "Open" or "Closed" based on current time
```

```
Create this plugin with clear comments explaining
each part.
```

AI might generate something like:

```php
<?php
/**
 * Plugin Name: Simple Business Hours
 * Description: Display business hours with a
 shortcode
 * Version: 1.0
 * Author: Your Name
 */

// Prevent direct access
if (!defined('ABSPATH')) {
    exit;
}

// Add settings page to admin menu
function sbh_add_menu() {
    add_options_page(
        'Business Hours',           // Page title
        'Business Hours',           // Menu title
        'manage_options',           // Capability
        required
        'business-hours',           // Menu slug
        'sbh_settings_page'         // Function to
        display page
    );
}
add_action('admin_menu', 'sbh_add_menu');

// Display settings page
function sbh_settings_page() {
    ?>
    <div class="wrap">
        <h1>Business Hours Settings</h1>
        <form method="post" action="options.php">
            <?php
            settings_fields('sbh_settings');
            do_settings_sections('business-hours');
```

```
            submit_button();
            ?>
        </form>
    </div>
    <?php
}

// Register settings
function sbh_register_settings() {
    register_setting('sbh_settings', 'sbh_hours');

    add_settings_section(
        'sbh_main_section',
        'Weekly Hours',
        null,
        'business-hours'
    );

    $days = ['Monday', 'Tuesday', 'Wednesday',
    'Thursday', 'Friday', 'Saturday', 'Sunday'];

    foreach ($days as $day) {
        add_settings_field(
            'sbh_' . strtolower($day),
            $day,
            'sbh_day_field',
            'business-hours',
            'sbh_main_section',
            ['day' => strtolower($day)]
        );
    }
}
add_action('admin_init', 'sbh_register_settings');

// Render day input field
function sbh_day_field($args) {
    $hours = get_option('sbh_hours', []);
    $day = $args['day'];
    $value = isset($hours[$day]) ? $hours[$day] :
    '9:00 AM - 5:00 PM';
    echo '<input type="text" name="sbh_hours[' .
    $day . ']" value="' . esc_attr($value) . '"
```

```
    placeholder="e.g., 9:00 AM - 5:00 PM or
    Closed">';
}

// Shortcode to display hours
function sbh_display_hours($atts) {
    $hours = get_option('sbh_hours', []);
    $days = ['monday', 'tuesday', 'wednesday',
    'thursday', 'friday', 'saturday', 'sunday'];

    $output = '<div class="business-hours">';
    $output .= '<table>';

    foreach ($days as $day) {
        $display_hours = isset($hours[$day]) ?
        $hours[$day] : 'Not set';
        $output .= '<tr>';
        $output .= '<td><strong>' . ucfirst($day)
        . '</strong></td>';
        $output .= '<td>' .
        esc_html($display_hours) . '</td>';
        $output .= '</tr>';
    }

    $output .= '</table>';
    $output .= '</div>';

    return $output;
}
add_shortcode('business_hours',
'sbh_display_hours');
```

To install this plugin:

1. Create a folder `simple-business-hours` in `/wp-content/plugins/`
2. Save the code as `simple-business-hours.php` in that folder
3. Activate in Plugins Installed Plugins
4. Configure in Settings Business Hours
5. Use `[business_hours]` shortcode in any page

AI Plugin Development

When asking AI to write WordPress plugins: 1. Be specific about functionality 2. Request comments explaining the code 3. Test thoroughly on a staging site 4. Review the code, understand what it does before installing 5. Ask AI to explain any parts you don't understand

12.4.12 Customising Theme Templates

You can override any parent theme template by copying it to your child theme and modifying it.

Common templates to customise:

Template	Purpose
`header.php`	Site header, navigation
`footer.php`	Site footer
`single.php`	Single blog post
`page.php`	Individual pages
`archive.php`	Post listing pages
`functions.php`	Theme functions

Example: Custom single post template

Copy `single.php` from parent theme to child theme, then modify:

```
<?php get_header(); ?>

<main class="site-content">
    <?php while (have_posts()) : the_post(); ?>
        <article class="post">
            <?php if (has_post_thumbnail()) : ?>
                <div class="featured-image">
                    <?php
                    the_post_thumbnail('large');
                    ?>
                </div>
            <?php endif; ?>

            <header>
                <h1><?php the_title(); ?></h1>
                <p class="meta">
```

```
                    Published <?php echo
                    get_the_date(); ?>
                    in <?php the_category(', ');
                    ?>
                </p>
            </header>

            <div class="content">
                <?php the_content(); ?>
            </div>

            <footer class="post-footer">
                <?php the_tags('<p
                class="tags">Tags: ', ', ',
                '</p>'); ?>

                <div class="author-box">
                    <?php echo
                    get_avatar(get_the_author_meta('ID'),
                    64); ?>
                    <div class="author-info">
                        <h4><?php the_author();
                        ?></h4>
                        <p><?php
                        the_author_meta('description');
                        ?></p>
                    </div>
                </div>
            </footer>
        </article>

        <?php comments_template(); ?>

    <?php endwhile; ?>
</main>

<?php get_sidebar(); ?>
<?php get_footer(); ?>
```

```
Ask your AI:
I want to modify the archive page template in my
child theme to
```

```
display posts in a grid layout instead of a list.
Show me how
to create a custom archive.php that uses CSS Grid
and displays
featured images with post titles.
```

12.4.13 When to Build Custom

Consider custom development when:

- No plugin meets the core requirement
- Plugins would require heavy modification anyway
- The feature is central to the business model
- Security or performance requirements are extreme
- Long-term cost of plugin licensing exceeds development

Even then, consider:

- A small custom plugin (focused, maintainable)
- Modifying an existing plugin (with a child plugin approach)
- Combining simpler plugins with custom glue code

Full custom rarely means starting from zero.

12.5 Building Your Mental Model

12.5.1 The Extension Pyramid

```
                    Custom Code      (Last
                  resort)

                    Child Themes     (Theme
               customisation)

                    Premium Plugins  (Paid,
           supported)

                    Free Plugins     (Widely
       tested)

                    WordPress Core
   (Foundation)
```

```
```

Start from the bottom: can WordPress core do it? Can a well-established free plugin do it? Only climb the pyramid when lower levels don't work.

12.5.2 The Dependency Chain

Every plugin creates a dependency:

```
Your Site
    Depends on Plugin A
          Depends on Plugin A's Developer
                Depends on Developer's continued
              interest
                       Depends on WordPress
                   compatibility
```

More plugins = more dependencies = more things that can break.

12.5.3 The True Cost Formula

When evaluating options:

```
True Cost = Upfront Cost + (Yearly Maintenance ×
Years) + Risk Cost

Plugin:
- Upfront: $0-300 (purchase/license)
- Maintenance: $50-100/year (updates, support
subscription)
- Risk: plugin abandonment, security issues

Custom:
- Upfront: $5,000-20,000 (development)
- Maintenance: $500-2,000/year (updates, hosting,
developer retainer)
- Risk: developer availability, scope creep
```

Over 5 years, the math often favours plugins for standard features.

12.6 Business Applications

12.6.1 ROI Calculation

Present plugin decisions to clients in business terms:

"A custom booking system would cost $15,000 to build and $2,000/year to maintain. The plugin costs $200/year. Over 5 years:

- Custom: $15,000 + ($2,000 × 5) = $25,000
- Plugin: $200 × 5 = $1,000

The plugin doesn't do *everything* you want, but is that extra functionality worth $24,000?"

12.6.2 Risk Communication

Help clients understand risks:

"This plugin has 500,000 active installations and is updated monthly. It's well-maintained. But if the developer stops working on it, we'd need to find an alternative. That's a risk with any plugin, which is why we document everything and maintain backups."

12.6.3 Managing Expectations

When clients want custom features:

"We can definitely build that custom. But let me show you how three existing plugins handle similar needs. If any of these work for you, you'll save significant money and get something battle-tested by thousands of users."

That positions you as an advisor as well as an implementer.

12.6.4 Security Responsibility

Plugins are the most common WordPress security vulnerability. Your role:

- Evaluate plugin security before installation
- Keep plugins updated
- Monitor for vulnerability announcements
- Have a response plan if something is compromised
- Document which plugins are installed and why

> **Skills You Are Building**
>
> This develops **making and defending technology choices** and **evaluating and improving solutions**. Professional plugin decisions require balancing features, costs, risks, and long-term maintenance, exactly the judgment

businesses need from technology advisors.

12.7 Practice Exercises

Exercise Levels

- **Level 1**: Direct application
- **Level 2**: Minor modifications
- **Level 3**: Combining concepts
- **Level 4**: Problem-solving
- **Level 5**: Open-ended design

12.7.1 Exercise 12.1: Plugin Research (Level 1)

On your local WordPress site:

1. Search for and compare three SEO plugins (e.g., Yoast, RankMath, All in One SEO)
2. For each, document: active installs, last updated, rating, key features
3. Install one and explore its settings
4. Write a brief summary of which you'd recommend and why

12.7.2 Exercise 12.2: Create a Child Theme (Level 2)

Create a child theme for your local site:

1. Follow the child theme creation process
2. Add custom CSS that changes the site's link colour
3. Verify the change appears
4. Confirm the parent theme still functions

Document the process with screenshots.

12.7.3 Exercise 12.3: PHP Customisation with AI (Level 3)

Use AI to add a custom feature to your WordPress site:

1. Choose one of these features (or propose your own):
    - Display "Reading time: X minutes" above each post
    - Add a "Back to top" button that appears when scrolling
    - Show related posts at the end of each blog post
    - Add a custom greeting based on time of day in the header

2. Ask your AI to help you implement it in your child theme's functions.php
3. For the code AI provides:
    - Read through it and identify what each part does
    - Ask AI to explain anything you don't understand
    - Test it on your local site
    - Document what you learned
4. Write 200 words explaining: What did the code do? What PHP concepts were used? What would you modify?

12.7.4 Exercise 12.4: Build a Plugin with AI (Level 4)

Create a simple WordPress plugin using AI assistance:

1. Define a specific need (examples):
    - A "Coming Soon" banner for selected pages
    - Social media share buttons for posts
    - A shortcode that displays a styled call-to-action box
    - A widget that shows recent posts from a specific category
2. Ask AI to create the plugin, requesting comments that explain the code
3. Install and test the plugin on your local site
4. Document:
    - The prompts you used with AI
    - How you tested the plugin
    - Any modifications you made
    - What you learned about WordPress plugin structure

12.7.5 Exercise 12.5: Plugin Audit (Level 3)

Your local site probably has several plugins now. Conduct an audit:

1. List every installed plugin
2. For each, document: purpose, last update, whether it's essential
3. Identify any that could be removed
4. Identify any that should be replaced with better alternatives
5. Create a recommendation document

12.7.6 Exercise 12.6: Build vs Buy Analysis (Level 4)

A client needs an event calendar for their community centre. Events have dates, times, locations, descriptions, and categories. Visitors should be able to filter by category and view a monthly calendar.

Research options and write a 400-word recommendation:

1. Identify two or three plugin options
2. Evaluate each against requirements
3. Consider what custom development would entail
4. Make a recommendation with justification
5. Address risks and mitigation

12.7.7 Exercise 12.7: Extension Strategy (Level 5)

You're taking over a WordPress site for a small marketing agency. They need:

- Portfolio showcase
- Team member profiles
- Client testimonials
- Contact forms
- Blog
- Newsletter signup

Currently, they have 23 plugins installed, some of which seem redundant.

Develop an extension strategy document (500 words):

1. What categories of plugins are essential?
2. How would you evaluate their current plugins?
3. What's your approach to consolidation?
4. How would you communicate changes to the client?
5. What ongoing maintenance would you recommend?

This exercise develops professional consulting skills.

12.8 Chapter Summary

- Every extension decision weighs build vs. buy trade-offs
- Plugin evaluation requires systematic criteria, not just feature lists
- Child themes allow safe customisation that survives updates
- PHP is WordPress's foundation, understanding basics enables deeper customisation
- The Loop and hooks (actions/filters) are core WordPress patterns
- AI is good at writing WordPress plugins and theme customisations
- Technical debt from plugins accumulates silently
- Professional judgment means recommending the *right* solution, not the most impressive one

12.9 Reflection

Before moving on, ensure you can:

- ☐ Evaluate plugins using systematic criteria
- ☐ Articulate build vs. buy trade-offs to a non-technical client
- ☐ Create and use a child theme for customisation
- ☐ Explain basic PHP syntax (variables, arrays, conditionals, loops)
- ☐ Describe what the WordPress Loop does
- ☐ Explain the difference between actions and filters
- ☐ Use AI to help write simple WordPress plugins
- ☐ Identify signs of a poorly maintained plugin
- ☐ Explain technical debt in business terms

12.10 Your Learning Journal

Record your responses to these prompts:

1. **Extension Audit**: Look at a WordPress site you use or manage. How many plugins does it have? Do they all seem necessary? What would you consolidate?
2. **Trade-off Thinking**: Think of a feature request that could be solved by either plugin or custom development. What factors would tip your recommendation one way or the other?
3. **AI Conversation Reflection**: What question about plugin evaluation or build vs. buy decisions did your AI partner help clarify?
4. **Professional Communication**: How would you explain to a client why you're recommending a paid plugin instead of a free one? What value does the paid version provide?

12.11 Next Steps

You now know how to extend WordPress wisely with plugins and child themes. But there's another approach entirely: using WordPress as a content backend while building the frontend with modern tools.

In Chapter 13, we'll explore headless WordPress, using the WordPress REST API to power a React frontend. This bridges your CMS knowledge with the modern frontend skills from the next part and represents how many larger organisations use WordPress today.

Chapter 13

Headless Architecture

13.1 The Concept First

In previous chapters, WordPress handled everything: it stored content, managed users, and generated the HTML visitors see. The content management and the website delivery were one unified system.

Headless architecture separates these concerns. WordPress becomes a *content backend*, storing and managing content, while a separate *frontend* application handles presentation. They communicate through APIs.

Why would you do this? Because sometimes you want:

- WordPress's excellent content management
- A modern frontend built with React or other frameworks
- Content delivered to multiple channels (web, mobile apps, kiosks)
- The performance benefits of static or cached frontends
- Complete design freedom unconstrained by WordPress themes

This approach is increasingly common for larger projects, media companies, and organisations with sophisticated frontend needs.

13.2 Understanding Through Separation

Imagine a restaurant operation:

Traditional restaurant (traditional WordPress):

- Kitchen and dining room in one building
- Chefs prepare food, servers deliver it directly
- Simple, everything coordinated, but limited to that one location

Central kitchen + multiple outlets (headless):

- Central kitchen prepares food (WordPress manages content)
- Multiple dining locations serve customers (web, mobile, etc.)
- Food travels via delivery (API)
- Each outlet can have different ambiance (different frontends)
- Kitchen changes don't require redesigning dining rooms

The central kitchen can supply a fine dining restaurant, a casual café, and a food truck, each with completely different presentations of the same underlying food.

> **Headless = API-First**
>
> "Headless" means the CMS has no "head", no built-in frontend. It's all backend. Content exits only through APIs, and something else (the "head") must display it.

13.3 Discovering Headless with Your AI Partner

13.3.1 Exploration 1: Architecture Trade-offs

Every architecture has trade-offs:

```
Ask your AI:
Compare traditional WordPress versus headless
WordPress architecture.
What does each approach do well? What problems
does each create?
Create a decision matrix.
```

This should reveal:

Traditional WordPress advantages:

- Simpler to set up and maintain
- Non-technical users can preview content
- Huge ecosystem of themes
- Lower technical barrier

Headless advantages:

- Frontend flexibility
- Better performance potential
- Multi-channel content delivery
- Modern developer experience

```
Continue the conversation:
When would a small business with limited technical
resources choose
headless? When would a tech company with
developers still choose
traditional WordPress?
```

13.3.2 Exploration 2: APIs as Contracts

The API is the contract between backend and frontend:

```
Ask your AI:
Explain how the WordPress REST API works as a
"contract" between the
content backend and the frontend application. What
does the frontend
expect? What does WordPress promise to provide?
```

The API contract specifies:

- What endpoints exist (`/wp-json/wp/v2/posts`)
- What data format to expect (JSON)
- What parameters can be sent (filters, pagination)
- What errors might occur

Frontend developers can build against this contract without needing to understand PHP or WordPress internals.

```
Continue the conversation:
What happens if WordPress changes its API in an
update? How do you
manage API versioning and prevent breaking the
frontend?
```

13.3.3 Exploration 3: Business Cases

Where does headless make practical sense?

```
Ask your AI:
Give me three real-world scenarios where headless
WordPress would
be the right architecture choice, and explain why
traditional
WordPress wouldn't work as well.
```

Common scenarios include:

- Media companies publishing to web, mobile apps, and smart devices
- E-commerce with heavily customised, high-performance frontends
- Organisations integrating content into existing applications
- Sites requiring the fastest possible load times (static generation)

```
Continue the conversation:
What about a local bakery's website? Would
headless make sense there?
Why or why not?
```

13.4 From Concept to Code

The sections below use the WordPress REST API to feed content to an external frontend.

13.4.1 The WordPress REST API

WordPress includes a built-in REST API. Every WordPress site (version 4.7+) automatically exposes content through standardised endpoints.

Default endpoints:

Endpoint	Returns
`/wp-json/wp/v2/posts`	Blog posts
`/wp-json/wp/v2/pages`	Pages
`/wp-json/wp/v2/categories`	Categories
`/wp-json/wp/v2/tags`	Tags
`/wp-json/wp/v2/media`	Media items
`/wp-json/wp/v2/users`	Users (public info)

Try it yourself: If your local WordPress site is running, visit `http://your-site.local/wp-json/wp/v2/posts` in your browser. You'll see JSON data for your posts.

13.4.2 Fetching Posts from WordPress

Using the JavaScript skills from Chapter 8:

```
async function getWordPressPosts() {
    const response = await
    fetch('http://your-site.local/wp-json/wp/v2/posts');

    if (!response.ok) {
        throw new Error('Failed to fetch posts');
    }

    const posts = await response.json();
    return posts;
}
```

Each post object contains:

```
{
    "id": 1,
    "date": "2024-01-15T10:30:00",
    "title": {
        "rendered": "Welcome to Our Blog"
    },
    "content": {
        "rendered": "<p>This is the post
        content...</p>"
    },
    "excerpt": {
        "rendered": "<p>A brief excerpt...</p>"
    },
    "slug": "welcome-to-our-blog",
    "featured_media": 42,
    "_links": { ... }
}
```

Note: `title.rendered` and `content.rendered` contain the processed HTML, ready for display.

13.4.3 Filtering and Pagination

The API supports query parameters:

```
// Get only 5 posts
fetch('/wp-json/wp/v2/posts?per_page=5')

// Get page 2 of results
```

```
fetch('/wp-json/wp/v2/posts?per_page=5&page=2')

// Filter by category (ID)
fetch('/wp-json/wp/v2/posts?categories=3')

// Search posts
fetch('/wp-json/wp/v2/posts?search=coffee')

// Order by date descending
fetch('/wp-json/wp/v2/posts?orderby=date&order=desc')
```

Common parameters:

Parameter	Purpose	Example
`per_page`	Results per page (max 100)	`?per_page=10`
`page`	Page number	`?page=2`
`search`	Search term	`?search=recipe`
`categories`	Filter by category ID	`?categories=5`
`tags`	Filter by tag ID	`?tags=12`
`orderby`	Sort field	`?orderby=title`
`order`	Sort direction	`?order=asc`
`_embed`	Include related data	`?_embed`

13.4.4 Getting Related Data with _embed

By default, the API returns IDs for related content (like featured images). The `_embed` parameter includes that data directly:

```
// Without _embed: featured_media is just an ID
(42)
// With _embed: full image data is embedded
fetch('/wp-json/wp/v2/posts?_embed')
```

The embedded data appears in `_embedded`:

```
const posts = await response.json();
const firstPost = posts[0];

// Access embedded featured image
const featuredImage =
firstPost._embedded?.['wp:featuredmedia']?.[0];
if (featuredImage) {
```

```
    console.log(featuredImage.source_url); //
    Image URL
}

// Access embedded author
const author =
firstPost._embedded?.['author']?.[0];
if (author) {
    console.log(author.name); // Author name
}
```

13.4.5 A Complete Example: Displaying Posts

Let's build a simple frontend that displays WordPress posts:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Headless Blog</title>
    <style>
        .post-card {
            border: 1px solid #ddd;
            padding: 1rem;
            margin-bottom: 1rem;
            border-radius: 4px;
        }
        .post-card h2 {
            margin-top: 0;
        }
        .loading, .error {
            padding: 1rem;
            text-align: center;
        }
        .error {
            color: red;
        }
    </style>
</head>
<body>
    <main>
        <h1>Latest Posts</h1>
```

```
        <div id="posts-container">
            <p class="loading">Loading
            posts...</p>
        </div>
    </main>

    <script>
        async function loadPosts() {
            const container =
            document.querySelector('#posts-container');

            try {
                const API =
                'http://your-site.local';
                const response = await fetch(
                    `${API}/wp-json/wp/v2/posts?_embed&per_page=5`
                );

                if (!response.ok) {
                    throw new Error(`HTTP error:
                    ${response.status}`);
                }

                const posts = await
                response.json();

                // Clear loading message
                container.innerHTML = '';

                // Render each post
                posts.forEach(post => {
                    const card =
                    document.createElement('article');
                    card.classList.add('post-card');

                    // Get featured image if
                    available
                    const featuredImage =
                    post._embedded?.['wp:featuredmedia']?.[0];
                    const imageHtml =
                    featuredImage
                        ? `<img
```

```
                        src="${featuredImage.source_url}"
                        alt="${featuredImage.alt_text
                        || ''}" style="max-width:
                        100%;">`
                        : '';

                    card.innerHTML = `
                        ${imageHtml}
                        <h2>${post.title.rendered}</h2>
                        <div>${post.excerpt.rendered}</div>
                        <a href="#">Read more</a>
                    `;

                    container.appendChild(card);
                });

            } catch (error) {
                container.innerHTML = `
                    <p class="error">Failed to
                    load posts:
                    ${error.message}</p>
                `;
                console.error('Error loading
                posts:', error);
            }
        }

        loadPosts();
    </script>
</body>
</html>
```

Save this as an HTML file and open it in a browser. It fetches posts from your local WordPress site and displays them independently of WordPress themes.

```
Ask your AI:
Walk me through this code. What happens at each
step? How would I
modify it to also show the post date and author
name?
```

13.4.6 Fetching a Single Post

To get a specific post by ID or slug:

```
// By ID
fetch('/wp-json/wp/v2/posts/42')

// By slug
fetch('/wp-json/wp/v2/posts?slug=welcome-to-our-blog')
```

13.4.7 Custom Post Types

If WordPress has custom post types (like "products" or "events"), they may need to be exposed via the API. In a plugin or theme:

```
// Register custom post type with REST API support
register_post_type('product', array(
    'public' => true,
    'show_in_rest' => true,  // This exposes it to
    the API
    'rest_base' => 'products', // Endpoint:
    /wp-json/wp/v2/products
    // ... other arguments
));
```

Once exposed, fetch them like any other content:

```
fetch('/wp-json/wp/v2/products?_embed')
```

13.4.8 Authentication for Protected Content

Public content is accessible without authentication. For protected content or write operations, you need authentication.

Application Passwords (WordPress 5.6+):

1. In WordPress: Users Your Profile Application Passwords
2. Generate a password for your application
3. Use Basic Authentication in requests:

```
const credentials =
btoa('username:application-password');

fetch('/wp-json/wp/v2/posts', {
    headers: {
```

```
        'Authorization': `Basic ${credentials}`
    }
});
```

> **Security Note**
>
> Never expose application passwords in frontend JavaScript, they'd be visible to anyone. Authentication is typically handled by a backend server that proxies requests to WordPress.

13.4.9 CORS Considerations

If your frontend and WordPress are on different domains, you'll encounter CORS. WordPress needs to allow requests from your frontend's origin.

A plugin or functions.php snippet can enable CORS:

```
// Allow CORS from specific origin
add_action('rest_api_init', function() {
    remove_filter('rest_pre_serve_request',
    'rest_send_cors_headers');
    add_filter('rest_pre_serve_request',
    function($value) {
        header('Access-Control-Allow-Origin:
        https://your-frontend-domain.com');
        header('Access-Control-Allow-Methods: GET,
        POST, OPTIONS');
        header('Access-Control-Allow-Credentials:
        true');
        return $value;
    });
});
```

For local development, both often run on localhost (different ports), which typically works without extra configuration.

13.5 Building Your Mental Model

13.5.1 Traditional vs Headless Architecture

Traditional WordPress:

```
          WordPress

    Content          Theme
   Database       (Presentation)


                          HTML

                    Browser
```

Headless WordPress:

```
  WordPress                  Frontend
          API      (React, etc.)
   Content
   Database       JSON

                         HTML

                            Browser
```

In headless, WordPress's theming layer isn't used. It's purely a content repository.

13.5.2 When Headless Makes Sense

Scenario	Why Headless?
Multi-channel delivery	Same content to web, iOS app, Android app, smart displays
Maximum performance	Static site generation, edge caching
Complex frontend needs	Single-page applications, heavy JavaScript interactivity
Developer preferences	Modern frontend frameworks, component architecture
Integration requirements	Content feeds into existing applications

13.5.3 When Traditional Makes Sense

Scenario	Why Traditional?
Content editors need preview	See exactly what's published
Simple sites	Blog, brochure site, small business
Limited technical resources	No frontend developers available
Plugin-dependent features	Many plugins expect traditional setup
Budget constraints	Two systems cost more to maintain

13.5.4 The Complexity Cost

Headless adds complexity:

- Two codebases instead of one
- Two deployments to manage
- Content preview is harder
- Some WordPress plugins don't work
- More technical expertise required

This complexity has costs, so use headless when the benefits outweigh them.

13.6 Business Applications

13.6.1 Multi-Channel Content

Organisations increasingly need content everywhere:

- Corporate website
- Mobile applications
- Internal dashboards
- Digital signage
- Partner integrations

With headless, content is created once and consumed everywhere. WordPress becomes the "single source of truth."

13.6.2 Performance and SEO

Headless frontends can be:

- Statically generated (fastest possible)
- Cached at the edge (CDN)
- Optimised specifically for performance

This matters for SEO and user experience. Google measures page speed.

13.6.3 Security Benefits

In headless architecture:

- WordPress can be hidden from the public (private network)
- Attack surface is reduced
- Frontend has no direct database access
- Compromising the frontend doesn't compromise content

For high-security needs, this separation is valuable.

13.6.4 Developer Experience

Modern frontend developers often prefer:

- React, Vue, or similar frameworks
- Component-based architecture
- TypeScript
- Modern build tools

Headless lets them use their preferred tools while using WordPress's content management.

> **Skills You Are Building**
>
> This develops **making and defending technology choices** and **assessing emerging technologies**. Headless architecture is a significant trend in web development. Understanding when it applies, and when it doesn't, demonstrates professional judgment.

13.7 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

13.7.1 Exercise 13.1: Explore the API (Level 1)

Using your local WordPress site:

1. Open a browser and visit `/wp-json/wp/v2/posts`

2. Explore the JSON structure, identify title, content, date, and other fields
3. Try different endpoints: `/pages`, `/categories`, `/media`
4. Add `?_embed` and observe how the response changes

Document your findings with screenshots.

13.7.2 Exercise 13.2: Build a Post List (Level 2)

Create an HTML page that:

1. Fetches the 3 most recent posts from your WordPress site
2. Displays title, date, and excerpt for each
3. Handles loading and error states
4. Styles the output with basic CSS

Test it in your browser.

13.7.3 Exercise 13.3: Filter and Search (Level 3)

Extend your post list to include:

1. A search input that filters posts by search term
2. A category dropdown that filters by category
3. Pagination (Next/Previous buttons)

This combines API parameters with JavaScript event handling.

13.7.4 Exercise 13.4: Architecture Recommendation (Level 4)

A regional newspaper wants to modernise their digital presence. They have:

- A 15-year-old WordPress site with 10,000+ articles
- A mobile app in development
- Plans for a digital subscriber paywall
- A team of journalists comfortable with WordPress
- One full-stack developer

Write a 400-word recommendation addressing:

- Should they go headless, traditional, or hybrid?
- What are the risks of each approach?
- What would you implement first?
- How would you handle the transition?

13.7.5 Exercise 13.5: Architecture Design (Level 5)

Design a headless architecture for a restaurant group with:

- 5 restaurant locations (different brands)
- Shared menu items across some locations
- Location-specific events and specials
- A central marketing team
- Mobile app plans for ordering

Create a diagram showing:

- What WordPress manages
- What the API provides
- What the frontend(s) display
- How content flows from creation to display

Write a 500-word explanation of your design decisions.

13.8 Chapter Summary

- Headless architecture separates content management from presentation
- The WordPress REST API exposes content as JSON data
- Frontends fetch and display content independently
- `_embed` includes related data like images and authors
- Headless adds complexity but enables multi-channel delivery and modern frontends
- Architecture choice depends on requirements, resources, and trade-offs

13.9 Reflection

Before moving to the Business Website Project, ensure you can:

☐ Explain headless architecture without jargon
☐ Fetch WordPress content using the REST API
☐ Use API parameters to filter and paginate results
☐ Access embedded data like featured images
☐ Articulate when headless makes sense and when it doesn't
☐ Describe the trade-offs of headless vs. traditional WordPress

13.10 Your Learning Journal

Record your responses to these prompts:

1. **Architecture Thinking**: When you visit websites, can you guess which might be using headless architecture? What clues suggest headless vs. traditional?

2. **API Exploration**: What surprised you about the WordPress REST API? What data was available that you didn't expect?
3. **AI Conversation Reflection**: What question about headless architecture did your AI partner help clarify?
4. **Trade-off Assessment**: Think of a website you might build. Would headless be appropriate? Why or why not?

13.11 Next Steps

You've completed the content management chapters. You understand:

- Why CMS matters and when to use one
- WordPress as a platform with an ecosystem
- How to extend WordPress professionally
- Headless architecture and the REST API

In Chapter 14, you'll build a complete WordPress business site, applying everything from this part. This project demonstrates your ability to make and implement technology decisions for real business needs.

Then, in the modern frontend chapters, we'll build on the API knowledge from this chapter to create React frontends, bringing modern frontend development into your skillset.

Chapter 14

Project: Business Website with WordPress

> **Example implementations available online**
>
> The online edition at books.borck.education (https://books.borck.education) includes links to example implementations, WordPress backups, and companion resources for each project.

14.1 Project Overview

The content management chapters taught you to think strategically: when to use a CMS, how to evaluate platforms and plugins, and how WordPress architecture works. Now you'll apply that knowledge to build a complete business website.

This project simulates a real client engagement. You'll analyse requirements, make technology decisions, implement a solution, and document your choices professionally. The deliverables mirror what you'd provide to an actual client.

14.2 Learning Outcomes Addressed

- **ULO 2**: Demonstrate project management and professional skills
- **ULO 3**: Analyse stakeholder requirements to inform design decisions
- **ULO 4**: Select and integrate appropriate technologies

14.3 Business Scenario

Choose one of these scenarios or propose your own:

14.3.1 Option A: Sunrise Café

A local café expanding to two locations. They need:

- Menu with categories (drinks, food, specials)
- Location pages for each café
- Blog for news and events
- Contact forms for catering inquiries
- Instagram feed integration

14.3.2 Option B: Hartley & Associates Accountants

A small accounting firm establishing online presence. They need:

- Service pages (tax, bookkeeping, advisory)
- Team member profiles
- Blog for tax tips and updates
- Client testimonials
- Secure contact form for inquiries

14.3.3 Option C: Community Garden Network

A non-profit connecting community gardens. They need:

- Garden listings with locations and details
- Events calendar
- Volunteer signup forms
- Resource library (downloadable guides)
- Newsletter signup

14.3.4 Option D: Your Own Scenario

Propose a real or fictional business. Must include:

- At least three distinct content types
- Clear business goals for the website
- Identifiable stakeholders
- Reasonable scope for this project

14.4 Requirements

14.4.1 Phase 1: Business Analysis

Before touching WordPress, complete:

Business Case Document (500-750 words):

1. **Business Overview**: What does this business do? What's their market position?
2. **Website Goals**: What should the website achieve? (leads, information, sales, community)
3. **Target Audience**: Who visits this site? What do they need?
4. **Success Metrics**: How will we know the website is working?

Stakeholder Analysis:

Stakeholder	Needs	Concerns	Priority
(e.g., Owner)	…	…	High/Med/Low

Content Strategy:

- What content types are needed?
- Who creates and maintains each type?
- How often does each type update?
- What's the approval workflow?

Site Map:

Create a visual or text-based site map showing:

- All pages and their hierarchy
- Content types and relationships
- Navigation structure

14.4.2 Phase 2: Technology Decisions

Document your decisions with justification:

Theme Selection:

- Evaluate at least three themes
- Document criteria and scores
- Justify your final selection
- Note any limitations you'll work around

Plugin Selection:

For each functional requirement, document:

- The need it addresses
- Options you evaluated
- Your selection and why
- Any concerns or trade-offs

Minimum plugins to evaluate:

- Contact forms (e.g., WPForms, Contact Form 7, Gravity Forms)
- SEO (e.g., Yoast, RankMath)
- Security (e.g., Wordfence, Sucuri)
- Backup (e.g., UpdraftPlus)
- Any scenario-specific needs

14.4.3 Phase 3: Implementation

Build the site in LocalWP:

Core Setup:

☐ WordPress installed and configured
☐ Permalinks set to "Post name"
☐ Site title and tagline configured
☐ Timezone correct
☐ Admin user secured (strong password)

Theme Implementation:

☐ Theme installed and activated
☐ Customiser settings configured (colours, fonts, logo)
☐ Menus created and assigned
☐ Homepage configured (static page or blog)
☐ Child theme created for customisations

Content:

☐ All pages from site map created
☐ All content types populated with realistic content
☐ Images appropriately sized and with alt text
☐ No placeholder/lorem ipsum text

Plugins:

☐ Essential plugins installed and configured
☐ Contact forms working and tested
☐ SEO plugin configured (titles, descriptions)
☐ Security plugin configured
☐ Backup plugin configured

Customisation (Child Theme):

☐ Child theme created and activated
☐ At least one CSS customisation implemented
☐ Customisation survives theme updates

14.4.4 Phase 4: Testing and Documentation

Testing Checklist:

- ☐ All pages load without errors
- ☐ Navigation works correctly
- ☐ Forms submit successfully
- ☐ Mobile responsive (test at multiple sizes)
- ☐ No JavaScript errors in console
- ☐ Images load properly
- ☐ Links not broken

Technical Documentation:

Create a handover document that includes:

1. **Login credentials** (for a hypothetical client)
2. **Theme information** (name, version, any modifications)
3. **Plugin inventory** (name, purpose, configuration notes)
4. **Content management guide** (how to add/edit content types)
5. **Maintenance recommendations** (backups, updates, security)

14.5 AI Collaboration Guidelines

14.5.1 How to Use AI Effectively

Use AI as a consultant you're learning from:

```
Ask your AI:
I'm choosing between Theme A and Theme B for a
café website. Here's
what I know about each... Help me think through
the decision. What
factors might I be missing?
```

```
Ask your AI:
I need to customise the footer in my child theme
to include business
hours. Walk me through how to do this safely, and
explain each step
so I understand what I'm doing.
```

14.5.2 Documentation Requirements

Keep an AI collaboration log documenting:

1. **Technology decisions**: How did AI help evaluate options?
2. **Implementation challenges**: What problems did you solve with AI help?
3. **Modifications**: How did you adapt AI suggestions?
4. **Learning moments**: What did you understand better after AI conversation?

14.5.3 Client Recommendation

Write a summary (300-400 words) as if presenting to the client:

- What did you build and why?
- What are the key features?
- How do they manage content?
- What ongoing maintenance is needed?
- What future enhancements might they consider?

14.6 Evaluation Criteria

14.6.1 Business Analysis (25%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Business case	Clear, comprehensive, demonstrates business understanding	Good overview with minor gaps	Basic understanding shown	Incomplete or superficial
Stake-holder analysis	All stakeholders identified with clear needs	Most stake-holders covered	Some stakeholders identified	Missing or unclear
Content strategy	Thorough, realistic, actionable	Good strategy with minor gaps	Basic strategy present	Missing or impractical
Site map	Complete, logical hierarchy	Good structure, minor issues	Basic structure present	Incomplete or illogical

14.6.2 Technology Decisions (25%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Theme evaluation	Systematic, documented, justified	Good evaluation with rationale	Basic comparison made	No clear evaluation
Plugin selection	Evaluated options, documented trade-offs	Good selection with reasons	Functional choices made	Random or unjustified
Decision documentation	Clear professional documentation	Good documentation	Basic documentation	Missing or poor

14.6.3 Implementation (30%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
WordPress setup	Correct configuration, secure	Mostly correct	Functional but issues	Significant problems
Theme/customisation	Well-configured, child theme works	Good customisation	Basic customisation	No customisation
Content quality	Professional, realistic, complete	Good content	Basic content	Placeholder content
Plugin configuration	All plugins properly configured	Most configured correctly	Basic configuration	Unconfigured or broken
Mobile responsiveness	Excellent on all devices	Good responsive behaviour	Works but issues	Broken on mobile

14.6.4 Documentation (20%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Technical docs	Complete handover-ready documentation	Good documentation	Basic documentation	Incomplete
Client recommendation	Professional, clear, actionable	Good summary	Basic summary	Missing or poor
AI collaboration log	Detailed, shows critical thinking	Good documentation	Basic log	Missing or superficial

14.7 Submission Checklist

Business Analysis:

- ☐ Business case document (500-750 words)
- ☐ Stakeholder analysis table
- ☐ Content strategy document
- ☐ Site map (visual or text)

Technical Deliverables:

- ☐ WordPress export file (.xml)
- ☐ Child theme files (zipped)
- ☐ Screenshot of completed site (homepage and two other pages)
- ☐ Plugin list with configuration notes

Documentation:

- ☐ Technical handover document
- ☐ Client recommendation summary (300-400 words)
- ☐ AI collaboration log

14.8 What to Submit

1. **Business analysis package** (PDF): Business case, stakeholder analysis, content strategy, site map
2. **WordPress export** (.xml file): Content export from Tools Export
3. **Child theme** (.zip file): Your child theme folder
4. **Technical documentation** (PDF): Handover document
5. **Recommendations and reflection** (PDF): Client recommendation + AI collaboration log

14.9 Getting Started

Begin with these conversations with your AI partner:

```
Ask your AI:
I'm building a WordPress site for [your scenario].
Help me think
through what questions I should be asking in the
business analysis
phase. What do I need to understand before making
technology decisions?
```

```
Ask your AI:
For a [type of business] website, what are the
essential plugins I
should consider? Not just what's popular, but what
this specific
type of business actually needs.
```

```
Ask your AI:
I'm writing a handover document for a client. What
should it include
so they can manage their WordPress site
independently? What do
non-technical clients most need to know?
```

14.10 Common Pitfalls to Avoid

1. **Skipping business analysis**: Jumping to WordPress without understanding requirements leads to rework
2. **Over-complicating**: Installing 20 plugins when 5 would suffice
3. **Using placeholder content**: "Lorem ipsum" signals incomplete work
4. **Ignoring mobile**: Always test responsive behaviour
5. **No documentation**: A site without documentation isn't ready for handover
6. **Copying AI output blindly**: AI helps you understand, you make the decisions
7. **Forgetting security**: Security and backup plugins are non-negotiable

14.11 Professional Context

This project mirrors real WordPress consulting work:

1. **Discovery**: Understanding business needs before proposing solutions
2. **Recommendation**: Evaluating options and justifying choices
3. **Implementation**: Building the solution professionally
4. **Handover**: Documenting for ongoing maintenance

These skills are directly applicable to freelance work, agency positions, or managing WordPress sites in any organisation.

14.12 After Submission

Your WordPress project demonstrates:

- Business analysis skills
- Technology evaluation and decision-making
- Practical WordPress implementation
- Professional documentation

Consider these as portfolio pieces. They show employers you can think beyond code.

Part V

Modern Frontend Development

Chapter 15

Component Thinking: React

15.1 The Concept First

In Chapter 13, you learned to fetch content from WordPress via API. But vanilla JavaScript DOM manipulation becomes unwieldy as applications grow. Every change requires finding elements, updating them, keeping track of what's displayed.

React solves this by introducing a different mental model: **component thinking**.

Instead of thinking "I need to update this paragraph when data changes," you think "This component displays data. When data changes, React handles the update."

Components are self-contained pieces of UI. Each component:

- Has its own logic
- Manages its own state (data that changes)
- Renders its own output
- Can be reused anywhere

That changes how you build interfaces. Once you think in components, you'll see every UI as a composition of reusable pieces.

15.2 Understanding Through LEGO

Consider how LEGO works:

- Each brick is **self-contained** with a consistent interface (the studs)
- Complex structures emerge from **combining simple pieces**
- The same brick can be used in **many different creations**
- You can **replace** one brick without rebuilding everything

- Instructions show **composition**, not monolithic assembly

React components work the same way:

- Each component is self-contained with consistent interfaces (props)
- Complex UIs emerge from combining simple components
- The same component can be reused throughout your application
- You can update one component without touching others
- Your code shows composition: components containing components

Treat a webpage as a tree of components, each responsible for its own piece of the interface.

> The Decomposition Instinct
>
> When you see any interface, practice breaking it into components. A navigation bar? That's a `Nav` component containing `NavLink` components. A product card? That's a `ProductCard` containing `Image`, `Title`, `Price`, and `Button` components. This instinct transfers to any component framework.

15.3 Discovering Components with Your AI Partner

15.3.1 Exploration 1: UI Decomposition

Let's practice component thinking before writing code:

```
Ask your AI:
Look at a typical e-commerce product listing page
(grid of products,
filters on the side, pagination at bottom). How
would you break this
into React components? Draw a diagram showing
which components
contain which other components.
```

Your diagram might look like:

```
ProductPage
  FilterSidebar
      CategoryFilter
      PriceFilter
      RatingFilter
  ProductGrid
      ProductCard (repeated)
          ProductImage
```

```
          ProductTitle
          ProductPrice
          AddToCartButton
  Pagination
```

```
Continue the conversation:
For each component you identified, what data would
it need? Where
would that data come from?
```

15.3.2 Exploration 2: State vs Props

React has two ways data lives in components: **state** (data the component owns and can change) and **props** (data passed from a parent).

```
Ask your AI:
Explain the difference between state and props in
React using a
family analogy. Props are like what? State is like
what? How do
they interact?
```

A common analogy:

- **Props** are like instructions parents give to children: "Here's your allowance amount, here's your bedtime." The child receives them but doesn't change them.
- **State** is like the child's own possessions: "I have $5 saved, I'm currently awake." The child controls these and they can change.

```
Continue the conversation:
For a shopping cart component, what would be props
and what would
be state? Walk me through why each belongs in its
category.
```

15.3.3 Exploration 3: Data Flow

Data in React flows one direction: parent to child (downward).

```
Ask your AI:
React data flows "one way", from parent to child
components. Why is
```

```
this design choice valuable? What problems does it
prevent compared
to data flowing any direction?
```

One-way data flow means:

- You always know where data came from
- Debugging is simpler (trace upward to find the source)
- Components are predictable
- Changes don't cause cascading side effects

```
Continue the conversation:
If data flows only downward, how does a child
component communicate
back to a parent? Give me a simple example.
```

15.4 From Concept to Code

Let's build your React understanding progressively.

15.4.1 Setting Up React

The simplest way to start is with Vite, a modern build tool:

```
npm create vite@latest my-react-app -- --template
react
cd my-react-app
npm install
npm run dev
```

This creates a React project and starts a development server. Open `http://localhost:5173` to see it running.

> **Prerequisites**
>
> You need Node.js installed. Download from nodejs.org if you haven't already.

15.4.2 JSX: HTML in JavaScript

React uses JSX, a syntax that lets you write HTML-like code in JavaScript:

```
function Welcome() {
    return (
```

```
        <div>
            <h1>Hello, World!</h1>
            <p>Welcome to React.</p>
        </div>
    );
}
```

This looks like HTML but it's actually JavaScript. JSX gets transformed into function calls that create elements.

Key JSX differences from HTML:

HTML	JSX
`class="..."`	`className="..."`
`for="..."`	`htmlFor="..."`
`onclick="..."`	`onClick={...}`
Self-closing optional	Self-closing required: `<img />`

JavaScript expressions go inside curly braces:

```
function Greeting({ name }) {
    const currentHour = new Date().getHours();
    const greeting = currentHour < 12 ? 'Good
    morning' : 'Good afternoon';

    return (
        <h1>{greeting}, {name}!</h1>
    );
}
```

15.4.3 Your First Component

Components are functions that return JSX:

```
function ProductCard() {
    return (
        <div className="product-card">
            <h2>Coffee Mug</h2>
            <p>$12.99</p>
            <button>Add to Cart</button>
        </div>
    );
```

```
}
```

Use components like HTML elements:

```
function App() {
    return (
        <div>
            <h1>Our Products</h1>
            <ProductCard />
            <ProductCard />
            <ProductCard />
        </div>
    );
}
```

Three product cards appear, but they're all identical. To make them different, we need props.

15.4.4 Props: Configuring Components

Props pass data from parent to child:

```
function ProductCard({ title, price }) {
    return (
        <div className="product-card">
            <h2>{title}</h2>
            <p>${price}</p>
            <button>Add to Cart</button>
        </div>
    );
}

function App() {
    return (
        <div>
            <h1>Our Products</h1>
            <ProductCard title="Coffee Mug"
            price={12.99} />
            <ProductCard title="Tea Cup"
            price={9.99} />
            <ProductCard title="Water Bottle"
            price={14.99} />
        </div>
```

```
    );
}
```

Now each card displays different data. The component is **reusable**, same structure, different content.

Props can be any JavaScript value:

```
<ProductCard
    title="Coffee Mug"
    price={12.99}
    inStock={true}
    tags={['kitchen', 'drinkware']}
    onAddToCart={() => console.log('Added!')}
/>
```

15.4.5 State: Data That Changes

Props come from outside; **state** lives inside the component and can change.

```
import { useState } from 'react';

function Counter() {
    const [count, setCount] = useState(0);

    return (
        <div>
            <p>Count: {count}</p>
            <button onClick={() => setCount(count
            + 1)}>
                Increment
            </button>
        </div>
    );
}
```

`useState` returns two things:

1. The current value (`count`)
2. A function to update it (`setCount`)

When you call `setCount`, React re-renders the component with the new value. You don't manually update the DOM. React handles it.

```
Ask your AI:
Walk me through what happens when I click the
button in this Counter
component. What does React do behind the scenes?
```

15.4.6 More State Examples

Toggle:

```
function Toggle() {
    const [isOn, setIsOn] = useState(false);

    return (
        <button onClick={() => setIsOn(!isOn)}>
            {isOn ? 'ON' : 'OFF'}
        </button>
    );
}
```

Form input:

```
function SearchBox() {
    const [query, setQuery] = useState('');

    return (
        <div>
            <input
                type="text"
                value={query}
                onChange={(e) =>
                setQuery(e.target.value)}
                placeholder="Search..."
            />
            <p>Searching for: {query}</p>
        </div>
    );
}
```

15.4.7 Lists and Keys

To render a list, use JavaScript's `map`:

```
function ProductList({ products }) {
    return (
        <div className="product-list">
            {products.map(product => (
                <ProductCard
                    key={product.id}
                    title={product.title}
                    price={product.price}
                />
            ))}
        </div>
    );
}
```

The `key` prop helps React track which items changed. Use a unique identifier (like `id`), not the array index.

15.4.8 useEffect: Side Effects and Data Fetching

Components sometimes need to do things besides rendering: fetch data, set up subscriptions, update the document title. These are **side effects**.

```
import { useState, useEffect } from 'react';

function PostList() {
    const [posts, setPosts] = useState([]);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState(null);

    useEffect(() => {
        async function fetchPosts() {
            try {
                const response = await fetch(

                    'https://jsonplaceholder.typicode.com/posts?_limit=5'
                );
                if (!response.ok) throw new
                Error('Failed to fetch');
                const data = await
                response.json();
                setPosts(data);
            } catch (err) {
```

```
                setError(err.message);
            } finally {
                setLoading(false);
            }
        }

        fetchPosts();
    }, []); // Empty array means "run once on
    mount"

    if (loading) return <p>Loading...</p>;
    if (error) return <p>Error: {error}</p>;

    return (
        <ul>
            {posts.map(post => (
                <li
                key={post.id}>{post.title}</li>
            ))}
        </ul>
    );
}
```

The `useEffect` hook:

- Runs after the component renders
- The empty `[]` dependency array means "run only once"
- Perfect for data fetching when component loads

15.4.9 Connecting to WordPress

Remember the WordPress REST API from Chapter 13? Let's fetch WordPress posts:

```
import { useState, useEffect } from 'react';

function WordPressPosts() {
    const [posts, setPosts] = useState([]);
    const [loading, setLoading] = useState(true);

    useEffect(() => {
        fetch('http://your-site.local/wp-json/wp/v2/posts?_embed')
            .then(res => res.json())
```

```
            .then(data => {
                setPosts(data);
                setLoading(false);
            })
            .catch(err => {
                console.error(err);
                setLoading(false);
            });
    }, []);

    if (loading) return <p>Loading posts...</p>;

    return (
        <div>
            {posts.map(post => (
                <article key={post.id}>
                    <h2 dangerouslySetInnerHTML={{
                    __html: post.title.rendered }}
                    />
                    <div
                    dangerouslySetInnerHTML={{
                    __html: post.excerpt.rendered
                    }} />
                </article>
            ))}
        </div>
    );
}
```

> **dangerouslySetInnerHTML**
>
> WordPress returns HTML in `rendered` fields. React requires `dangerouslySetInnerHTML` to insert raw HTML. Only use this with trusted content (like your own WordPress site), never with user input.

15.4.10 Component Composition

Build complex UIs by composing simple components:

```
function App() {
    return (
        <div className="app">
```

```
            <Header />
            <main>
                <Sidebar />
                <PostList />
            </main>
            <Footer />
        </div>
    );
}

function Header() {
    return (
        <header>
            <Logo />
            <Navigation />
        </header>
    );
}

function Navigation() {
    return (
        <nav>
            <NavLink to="/">Home</NavLink>
            <NavLink to="/about">About</NavLink>
            <NavLink
            to="/contact">Contact</NavLink>
        </nav>
    );
}
```

Each component is small, focused, and understandable. The composition creates the full application.

15.5 Building Your Mental Model

15.5.1 The Component Tree

Your application forms a tree of components:

```
App
  Header
    Logo
```

```
      Navigation
          NavLink (×3)
  Main
      Sidebar
      PostList
          PostCard (×n)
  Footer
```

Data flows down this tree. State lives in the component that "owns" it, often a parent that passes it to children.

15.5.2 When State Should "Lift Up"

If two sibling components need the same data, the state should live in their common parent:

```
Before (broken):                After (working):
ProductFilter (has state)       ProductPage (has
state)
ProductList (needs state)          ProductFilter
(receives props)
(How does List get Filter's        ProductList
(receives props)
 state? They're siblings!)
```

This is called "lifting state up", moving state to the lowest common ancestor.

15.5.3 The Rendering Cycle

```
State Changes
     ↓
React Re-renders Component
     ↓
New JSX Generated
     ↓
React Compares to Previous
     ↓
Only Changed Parts Update in DOM
```

You don't touch the DOM. You declare what should appear based on state. React efficiently updates only what changed.

15.6 Business Applications

15.6.1 Reusability

Build a `Button` component once with variants (primary, secondary, danger). Use it everywhere. Update the design in one place, and every button updates.

15.6.2 Maintainability

When a feature needs updating, you know which component to modify. Components are isolated, changes don't ripple unexpectedly.

15.6.3 Team Collaboration

Different developers can work on different components simultaneously. The interface (props) defines how components connect, teams don't need to understand each other's implementation details.

15.6.4 Testing

Components can be tested in isolation. Does `ProductCard` render correctly with these props? Does `Counter` increment when clicked? Unit tests are straightforward.

15.6.5 Design Systems

React components naturally form design systems. Your `Button`, `Card`, `Input`, and `Modal` components become a library that enforces consistency across the application.

> **Skills You Are Building**
>
> This develops **evaluating and improving solutions** and **making and defending technology choices**. React's component model is influential across the industry, understanding it prepares you for Vue, Angular, and other frameworks that share similar concepts.

15.7 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

15.7.1 Exercise 15.1: First Component (Level 1)

Create a React project with Vite and build a `Greeting` component that:

1. Accepts a `name` prop
2. Displays "Hello, {name}!"
3. Is used three times in `App` with different names

15.7.2 Exercise 15.2: Stateful Component (Level 2)

Build a `LikeButton` component that:

1. Displays a heart icon (can be emoji □)
2. Shows a like count starting at 0
3. Increments the count when clicked
4. Changes colour when count > 0

15.7.3 Exercise 15.3: List Rendering (Level 3)

Build a `TaskList` component that:

1. Accepts an array of tasks as props
2. Renders each task as a `TaskItem` component
3. Each `TaskItem` shows the task text and a "Complete" button
4. Clicking "Complete" removes the task (hint: lift state up)

15.7.4 Exercise 15.4: Data Fetching (Level 4)

Build a `UserList` component that:

1. Fetches users from `https://jsonplaceholder.typicode.com/users`
2. Displays loading state while fetching
3. Handles errors gracefully
4. Renders each user as a `UserCard` with name and email
5. Allows clicking a user to show more details

15.7.5 Exercise 15.5: WordPress Integration (Level 5)

Build a mini blog frontend that:

1. Fetches posts from your local WordPress site (or JSONPlaceholder)
2. Displays posts in a `PostList` component
3. Allows clicking a post to see full content in a `PostDetail` component
4. Includes a search box that filters posts by title
5. Handles loading and error states throughout

Document your component structure and explain your state management decisions.

15.8 Chapter Summary

- Components are reusable, self-contained UI pieces
- Props pass data from parent to child (read-only)
- State is data the component owns and can change
- `useState` creates state; `useEffect` handles side effects
- Data flows one direction: parent to child
- Component thinking is transferable to other frameworks

15.9 Reflection

Before moving on, ensure you can:

☐ Break down a UI into a component hierarchy
☐ Create functional components with JSX
☐ Pass data with props and explain why props are read-only
☐ Use `useState` to manage component state
☐ Fetch data with `useEffect`
☐ Render lists with `map` and explain why keys matter
☐ Explain how React's one-way data flow works

15.10 Your Learning Journal

Record your responses to these prompts:

1. **Decomposition Practice**: Pick any website you use. Sketch how you'd break it into React components. What would be reusable?

2. **State Analysis**: Think about a form you've filled out online. What state would it need? Where would that state live?

3. **AI Conversation Reflection**: What React concept was hardest to grasp? What question to your AI partner helped clarify it?
4. **Mental Model Shift**: How is thinking in components different from the vanilla JavaScript approach in earlier chapters? What's easier? What's harder?

15.11 Next Steps

You can now build component-based interfaces and fetch data from APIs. But styling components from scratch takes time.

In Chapter 16, we'll explore CSS frameworks such as Bootstrap and Tailwind, which provide pre-built styles and utilities. Combined with React, these tools enable rapid professional development without designing every button and layout from scratch.

Chapter 16

Rapid Development: CSS Frameworks

16.1 The Concept First

In Chapter 6, you learned CSS from scratch, selectors, box model, flexbox, media queries. But building every project from zero is slow, and consistency across large applications is hard.

CSS frameworks solve this by providing pre-built styles, components, and design decisions. Instead of designing every button, input, and card yourself, you adopt tested patterns that work together.

Using a framework is **leverage**. Professional developers use frameworks to move faster while maintaining quality. The skill is knowing when to use them, which to choose, and how to customise them for specific needs.

16.2 Understanding Through Design Systems

Large organisations don't let every designer create their own button styles. They create **design systems**, documented collections of components, colours, typography, and patterns that ensure consistency.

Google has Material Design. Apple has Human Interface Guidelines. Salesforce has Lightning Design System. These define how interfaces should look and behave across products.

CSS frameworks are design systems you can adopt immediately:

- **Bootstrap**: Component-focused system with pre-built UI elements

- **Tailwind CSS**: Utility-focused system with building blocks for custom designs
- **Foundation**, **Bulma**, **Chakra UI**: Other popular options

Each makes different trade-offs. Understanding those trade-offs is how you choose between them.

> **Constraints Enable Speed**
>
> Paradoxically, constraints often speed up creative work. When you don't have to decide every colour, spacing, and font size, you can focus on structure and user experience. Frameworks provide productive constraints.

16.3 Discovering Frameworks with Your AI Partner

16.3.1 Exploration 1: Framework Philosophies

Bootstrap and Tailwind represent different approaches:

```
Ask your AI:
Compare the philosophies behind Bootstrap and
Tailwind CSS. Bootstrap
gives you pre-built components. Tailwind gives you
utility classes.
When would you choose each approach? What are the
trade-offs?
```

This should reveal:

Bootstrap approach:

- "Here's a button: `.btn .btn-primary`"
- Faster to start
- Consistent out of the box
- Can look generic without customisation

Tailwind approach:

- "Build your button: `bg-blue-500 text-white px-4 py-2 rounded`"
- More markup
- Maximum flexibility
- Requires more design decisions

```
Continue the conversation:
I'm building a dashboard for a startup with tight
deadlines. Which
```

```
framework would you recommend and why?
```

16.3.2 Exploration 2: Constraints as Features

How do limitations help?

```
Ask your AI:
How do constraints help creativity? Apply this
concept to CSS
frameworks. Why might having fewer choices
actually improve outcomes?
```

Constraints reduce decision fatigue. When a framework says "primary buttons are this colour, secondary buttons are that colour," you're not spending mental energy on decisions that don't differentiate your product.

```
Continue the conversation:
When do framework constraints become limiting? At
what point should
a project move away from a framework?
```

16.3.3 Exploration 3: The "Generic" Problem

```
Ask your AI:
A client says their Bootstrap site "looks like
every other Bootstrap
site." How would you respond? What can be done?
```

This reveals that frameworks are starting points, not endpoints. Customisation (colours, typography, spacing) makes a generic framework site distinctive.

16.4 From Concept to Code

16.4.1 Bootstrap: Component-Based Framework

Bootstrap provides pre-styled components you assemble.

Setup in HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport"
    content="width=device-width, initial-scale=1">
    <title>Bootstrap Demo</title>
    <link rel="stylesheet"
    href=
"https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
>
</head>
<body>
    <!-- Your content -->
    <script
    src=
"https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
></script>
</body>
</html>
```

Buttons:

```
<button class="btn btn-primary">Primary</button>
<button class="btn
btn-secondary">Secondary</button>
<button class="btn btn-outline-danger">Danger
Outline</button>
<button class="btn btn-lg btn-success">Large
Green</button>
```

Cards:

```
<div class="card" style="width: 18rem;">
    <img src="image.jpg" class="card-img-top"
    alt="...">
    <div class="card-body">
        <h5 class="card-title">Card Title</h5>
        <p class="card-text">Some descriptive text
        here.</p>
        <a href="#" class="btn btn-primary">Learn
        More</a>
    </div>
</div>
```

Grid System:

```
<div class="container">
    <div class="row">
        <div class="col-md-4">Column 1</div>
        <div class="col-md-4">Column 2</div>
        <div class="col-md-4">Column 3</div>
    </div>
</div>
```

The grid is 12 columns wide. `col-md-4` means "4 columns wide on medium screens and up." On smaller screens, columns stack vertically.

Responsive utilities:

```
<div class="d-none d-md-block">Hidden on mobile,
visible on medium+</div>
<div class="d-block d-md-none">Visible on mobile,
hidden on medium+</div>
```

```
Ask your AI:
I want a navigation bar that collapses into a
hamburger menu on
mobile. Show me how to build this with Bootstrap.
```

16.4.2 Tailwind CSS: Utility-First Framework

Tailwind provides low-level utility classes you combine.

Setup (simplest via CDN for learning):

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
    content="width=device-width, initial-scale=1">
    <title>Tailwind Demo</title>
    <script
    src="https://cdn.tailwindcss.com"></script>
</head>
<body>
    <!-- Your content -->
</body>
</html>
```

> **Production Setup**
>
> For production, install Tailwind via npm and build a custom CSS file. The CDN is for learning and prototyping only.

Buttons (you design them):

```
<button class="bg-blue-500 text-white px-4 py-2
rounded hover:bg-blue-600">
    Primary Button
</button>

<button class="bg-gray-200 text-gray-800 px-4 py-2
rounded hover:bg-gray-300">
    Secondary Button
</button>

<button class="border border-red-500 text-red-500
px-4 py-2 rounded hover:bg-red-500
hover:text-white">
    Danger Outline
</button>
```

Understanding the classes:

Class	Meaning
`bg-blue-500`	Background colour (blue, shade 500)
`text-white`	Text colour white
`px-4`	Padding horizontal (left/right), size 4
`py-2`	Padding vertical (top/bottom), size 2
`rounded`	Border radius
`hover:bg-blue-600`	On hover, darker blue

Cards:

```
<div class="max-w-sm rounded overflow-hidden
shadow-lg">
    <img class="w-full" src="image.jpg" alt="...">
    <div class="px-6 py-4">
        <h2 class="font-bold text-xl mb-2">Card
        Title</h2>
        <p class="text-gray-700 text-base">
```

```
            Some descriptive text here.
        </p>
    </div>
    <div class="px-6 pt-4 pb-2">
        <button class="bg-blue-500 text-white px-4
        py-2 rounded">
            Learn More
        </button>
    </div>
</div>
```

Flexbox layout:

```
<div class="flex gap-4">
    <div class="flex-1 bg-gray-100 p-4">Column
    1</div>
    <div class="flex-1 bg-gray-100 p-4">Column
    2</div>
    <div class="flex-1 bg-gray-100 p-4">Column
    3</div>
</div>
```

Responsive design:

```
<!-- Single column on mobile, three columns on
medium screens -->
<div class="flex flex-col md:flex-row gap-4">
    <div class="flex-1">Column 1</div>
    <div class="flex-1">Column 2</div>
    <div class="flex-1">Column 3</div>
</div>
```

Tailwind's breakpoint prefixes: `sm:`, `md:`, `lg:`, `xl:`, `2xl:`.

```
Ask your AI:
Build me a responsive navigation bar with Tailwind
that has a logo
on the left and links on the right, and stacks
vertically on mobile.
```

16.4.3 Using Frameworks with React

Both frameworks work well with React.

Bootstrap with React:

```
npm install bootstrap
```

```
// In your main.jsx or App.jsx
import 'bootstrap/dist/css/bootstrap.min.css';

function ProductCard({ title, price }) {
    return (
        <div className="card" style={{ width:
        '18rem' }}>
            <div className="card-body">
                <h5
                className="card-title">{title}</h5>
                <p
                className="card-text">${price}</p>
                <button className="btn
                btn-primary">Add to Cart</button>
            </div>
        </div>
    );
}
```

For components with JavaScript (modals, dropdowns), consider `react-bootstrap`:

```
npm install react-bootstrap bootstrap
```

```
import Button from 'react-bootstrap/Button';
import Card from 'react-bootstrap/Card';

function ProductCard({ title, price }) {
    return (
        <Card style={{ width: '18rem' }}>
            <Card.Body>
                <Card.Title>{title}</Card.Title>
                <Card.Text>${price}</Card.Text>
                <Button variant="primary">Add to
                Cart</Button>
            </Card.Body>
        </Card>
    );
}
```

Tailwind with React:

```
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Configure `tailwind.config.js`:

```
export default {
    content: [
        "./index.html",
        "./src/**/*.{js,ts,jsx,tsx}",
    ],
    theme: {
        extend: {},
    },
    plugins: [],
}
```

Add to your CSS file:

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Use in components:

```
function ProductCard({ title, price }) {
    return (
        <div className="max-w-sm rounded shadow-lg
        p-4">
            <h2 className="font-bold text-xl
            mb-2">{title}</h2>
            <p
            className="text-gray-700">${price}</p>
            <button className="mt-4 bg-blue-500
            text-white px-4 py-2 rounded
            hover:bg-blue-600">
                Add to Cart
            </button>
        </div>
    );
}
```

16.4.4 Customisation

Bootstrap customisation (via Sass):

```
// Override variables before importing Bootstrap
$primary: #your-brand-color;
$font-family-base: 'Your Font', sans-serif;
$border-radius: 0.5rem;

@import "bootstrap/scss/bootstrap";
```

Tailwind customisation (via config):

```
// tailwind.config.js
export default {
    theme: {
        extend: {
            colors: {
                'brand': '#your-brand-color',
                'brand-dark': '#darker-version',
            },
            fontFamily: {
                'sans': ['Your Font',
                'sans-serif'],
            },
        },
    },
}
```

Then use: `bg-brand`, `text-brand-dark`, etc.

16.5 Building Your Mental Model

16.5.1 The Abstraction Spectrum

```
More Abstraction                              Less
Abstraction
(faster, less control)
(slower, more control)

Component Libraries  →  Bootstrap  →  Tailwind  →
Custom CSS
(MUI, Chakra)           (pre-built)    (utilities)
```

```
(from scratch)
```

Move right for more control, left for more speed. Project needs determine the right position.

16.5.2 When to Choose What

Situation	Consider
Rapid prototyping	Bootstrap or component library
Admin dashboards	Bootstrap (consistent patterns)
Marketing sites	Tailwind (unique designs)
Highly custom UI	Tailwind or custom CSS
Team with designers	Tailwind (matches design tokens)
Team without designers	Bootstrap (built-in design decisions)
Learning CSS	Start custom, add frameworks later

16.5.3 The Class Explosion Concern

Tailwind markup can look verbose:

```
<button class="bg-blue-500 hover:bg-blue-700
text-white font-bold py-2 px-4 rounded
focus:outline-none focus:shadow-outline">
    Submit
</button>
```

Solutions:

1. **Extract components** (in React, this is natural)
2. **Use `@apply` in CSS** (creates reusable classes)
3. **Accept the trade-off** (markup complexity vs CSS complexity)

```
Ask your AI:
When does Tailwind's utility approach become
unmanageable? How do
teams keep Tailwind codebases maintainable on
large projects?
```

16.6 Business Applications

16.6.1 Development Speed

Frameworks reduce time-to-market:

- Pre-built responsive grids
- Tested browser compatibility
- Accessible components (often)
- Documentation and examples

What takes days in custom CSS takes hours with frameworks.

16.6.2 Consistency at Scale

Large applications need consistent interfaces. Frameworks enforce consistency:

- All buttons look like buttons
- All cards have the same shadow
- Spacing follows a system
- Typography is unified

This consistency improves user experience and reduces maintenance.

16.6.3 Team Efficiency

New team members can contribute faster when using standard frameworks:

- Well-documented
- Familiar patterns
- Searchable solutions online
- Reduced onboarding time

16.6.4 Mobile-First by Default

Modern frameworks are mobile-responsive out of the box. Bootstrap's grid and Tailwind's responsive prefixes handle responsive design that would take a lot of custom CSS.

> **i** Skills You Are Building
>
> This develops **making and defending technology choices** and **evaluating and improving solutions**. Framework selection is a business decision, speed vs. customisation, team skills, project timeline. Professional judgment means choosing appropriately, not defaulting to what you know.

16.7 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

16.7.1 Exercise 16.1: Bootstrap Basics (Level 1)

Create an HTML page using Bootstrap that includes:

1. A navigation bar with a brand name and three links
2. A hero section with a heading, paragraph, and button
3. A three-column grid of cards
4. A footer with centred text

Use only Bootstrap classes, no custom CSS.

16.7.2 Exercise 16.2: Tailwind Basics (Level 2)

Recreate the same layout from Exercise 16.1 using Tailwind:

1. Navigation bar
2. Hero section
3. Three-column responsive grid
4. Footer

Compare the experience. Which felt faster? Which gave more control?

16.7.3 Exercise 16.3: React + Framework (Level 3)

In a React project:

1. Choose either Bootstrap or Tailwind
2. Create a `ProductCard` component
3. Create a `ProductGrid` component that displays multiple cards
4. Make it responsive (single column on mobile, grid on desktop)
5. Add a "Featured" variant of the card with different styling

16.7.4 Exercise 16.4: Customisation (Level 4)

Take your Exercise 16.3 project and customise the framework:

1. Change the primary colour to a custom brand colour
2. Adjust the border radius across components
3. Modify the default font family
4. Document the customisation process

Write 200 words explaining how customisation changes the "generic" feel.

16.7.5 Exercise 16.5: Framework Recommendation (Level 5)

A startup is building a SaaS product dashboard. They have:

- Two developers (full-stack, not design-focused)
- Tight three-month deadline
- Need for admin panels, data tables, forms, charts
- Plans to hire a designer in six months

Write a 500-word recommendation:

1. Which framework would you recommend?
2. What's the trade-off they're accepting?
3. How should they approach customisation?
4. What happens when the designer arrives?
5. What alternatives did you consider?

16.8 Chapter Summary

- CSS frameworks provide pre-built styles and design systems
- Bootstrap offers component-based development with pre-styled elements
- Tailwind offers utility-based development with maximum flexibility
- Both work well with React
- Framework choice depends on project needs, team skills, and timeline
- Customisation makes generic frameworks feel branded

16.9 Reflection

Before moving on, ensure you can:

☐ Explain the difference between component and utility frameworks
☐ Use Bootstrap to create responsive layouts and components
☐ Use Tailwind to style elements with utility classes
☐ Integrate either framework with React
☐ Customise framework colours, fonts, and spacing
☐ Articulate when to choose each framework

16.10 Your Learning Journal

Record your responses to these prompts:

1. **Preference Exploration**: After trying both Bootstrap and Tailwind, which felt more natural to you? Why might someone prefer the other?
2. **Trade-off Analysis**: Think of a project you might build. Which framework would you choose and what trade-offs would you accept?
3. **AI Conversation Reflection**: What framework concept was hardest to grasp? What question to your AI partner helped clarify it?
4. **Professional Judgment**: A client asks "Why don't we just use custom CSS?" How would you respond?

16.11 Next Steps

You now have tools for rapid, consistent UI development. But professional work also requires practices that ensure quality, collaboration, and maintainability.

In Chapter 17, we'll cover version control with Git, testing basics, and code quality practices. These professional practices separate hobby projects from production-ready work and are expected in any development role.

Chapter 17

Professional Practices

17.1 The Concept First

In Chapter 16, you learned to build interfaces quickly with CSS frameworks. But professional development is about building **systems that keep working and that a team can maintain**.

Consider the difference:

- **Hobbyist**: Gets the feature working, moves on
- **Professional**: Gets the feature working, ensures it stays working, makes it easy for others to understand and modify

Professional practices are **risk management**. Every project eventually faces:

- "It was working yesterday. What changed?"
- "I can't remember why I wrote this code."
- "Someone else needs to fix this while I'm on holiday."
- "The client changed their mind. Can we undo this?"

Professional practices answer these questions before they become crises, so code **keeps working** tomorrow as well as today.

17.2 Understanding Through Craftsmanship

Consider two furniture makers:

The first measures wood, cuts it, assembles pieces. If something doesn't fit, they shave a bit here, add a shim there. The table works, but any later modification requires figuring out all the adjustments again.

The second measures twice, documents the cuts, keeps offcuts labelled, photographs each stage. If something doesn't fit, they can trace exactly where the

deviation started. Years later, they can build a matching chair.

Both produce functional furniture. But the second can:

- Train apprentices using their documentation
- Reproduce work reliably
- Fix problems systematically
- Collaborate with other craftspeople

Software development is the same. Version control is measuring twice. Testing is quality checks before delivery. Documentation is the labelled workshop.

> **Professional Practice Serves Future You**
>
> In six months, you won't remember why you made certain decisions. Professional practices create a trail for "future you" to follow. That's not extra work, it's insurance.

17.3 Discovering Professional Practices with Your AI Partner

17.3.1 Exploration 1: Why Version Control Matters

```
Ask your AI:
Imagine a team of four developers working on the
same website without
version control. What problems would they face?
How would they handle
conflicts when two people change the same file?
```

This should reveal:

- Overwriting each other's work
- No way to revert mistakes
- "Which version is the real one?"
- Fear of making changes
- No history of what changed or why

```
Continue the conversation:
Now explain how Git solves each of these problems.
What mental model
should I have for how Git works?
```

17.3.2 Exploration 2: The Testing Mindset

Testing gives you **confidence** to change code.

```
Ask your AI:
A developer says "I tested it manually, it works."
Why isn't that
enough for professional work? What's the
difference between manual
testing and automated tests?
```

Points to notice:

- Manual testing doesn't scale
- Humans miss things, especially when tired
- Automated tests run every time, consistently
- Tests document expected behaviour
- Regression testing catches "breaking old stuff while adding new"

```
Continue the conversation:
For a simple e-commerce site, what would be the
most valuable things
to test automatically? Where would you start?
```

17.3.3 Exploration 3: Code as Communication

Code is read far more than it's written. Every piece of code is communication with:

- Future you
- Teammates
- Future maintainers

```
Ask your AI:
What makes code "readable"? Show me an example of
unreadable code and
the same functionality written readably. What
changed?
```

This reveals:

- Meaningful variable names
- Logical structure
- Appropriate comments (why, not what)
- Consistent formatting
- Small, focused functions

```
Continue the conversation:
When should I write comments, and when does the
code speak for itself?
Give me guidelines for useful commenting.
```

17.3.4 Exploration 4: Documentation Purpose

```
Ask your AI:
What's the difference between code comments,
README files, and
technical documentation? When do I need each, and
what goes in each?
```

Different documentation serves different audiences:

- **Code comments**: Future developers modifying the code
- **README**: Anyone encountering the project for the first time
- **Technical docs**: Users of your code (API consumers, library users)
- **User docs**: Non-technical end users

17.4 From Concept to Code

17.4.1 Git: Version Control Fundamentals

Git tracks changes to files over time. The mental model:

```
Working Directory → Staging Area → Repository
    (your files)     (ready to commit)  (permanent
    history)
```

Initial setup:

```
# Configure your identity (once per machine)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Starting a project:

```
# Create a new repository
git init

# Or clone an existing one
git clone https://github.com/username/repo.git
```

Basic workflow:

```
# See what's changed
git status

# Stage changes (prepare to commit)
git add filename.js
git add .  # Stage all changes

# Commit (save to history)
git commit -m "Add user login feature"

# View history
git log --oneline
```

Understanding the commands:

Command	What it does
`git status`	Shows changed, staged, and untracked files
`git add`	Moves changes to staging area
`git commit`	Creates permanent snapshot with message
`git log`	Shows commit history
`git diff`	Shows what changed (unstaged changes)

> **Commit Messages Matter**
>
> Good: "Fix login timeout on slow connections" Bad: "Fixed bug" or "Updates"
> The message should explain **why** you made the change, not what files you touched. Future you needs to understand the intent.

Working with branches:

Branches let you work on features without affecting the main code:

```
# Create and switch to a new branch
git checkout -b feature/user-profiles

# Work, commit, work, commit...

# Switch back to main
git checkout main
```

```
# Merge the feature branch
git merge feature/user-profiles
```

```
Ask your AI:
What's the difference between merging and rebasing
in Git? When would
I use each? I want to understand the trade-offs.
```

Working with GitHub:

```
# Connect to remote repository
git remote add origin
https://github.com/username/repo.git

# Push commits to GitHub
git push origin main

# Pull changes from GitHub
git pull origin main
```

17.4.2 Testing: Starting Simple

Testing verifies your code does what you expect. Start simple:

Manual test file (for learning):

```
// math.js
function add(a, b) {
    return a + b;
}

function multiply(a, b) {
    return a * b;
}

// Simple tests (learning only)
console.log("Testing add:");
console.log(add(2, 3) === 5 ? "PASS" : "FAIL");
console.log(add(-1, 1) === 0 ? "PASS" : "FAIL");
console.log(add(0, 0) === 0 ? "PASS" : "FAIL");

console.log("Testing multiply:");
console.log(multiply(2, 3) === 6 ? "PASS" :
```

```
"FAIL");
console.log(multiply(-2, 3) === -6 ? "PASS" :
"FAIL");
```

Using a testing framework (Vitest with React/Vite):

```
npm install -D vitest
```

```
// math.test.js
import { describe, it, expect } from 'vitest';
import { add, multiply } from './math.js';

describe('add', () => {
    it('adds positive numbers', () => {
        expect(add(2, 3)).toBe(5);
    });

    it('handles negative numbers', () => {
        expect(add(-1, 1)).toBe(0);
    });

    it('handles zero', () => {
        expect(add(0, 0)).toBe(0);
    });
});

describe('multiply', () => {
    it('multiplies positive numbers', () => {
        expect(multiply(2, 3)).toBe(6);
    });

    it('handles negative numbers', () => {
        expect(multiply(-2, 3)).toBe(-6);
    });
});
```

Run tests:

```
npx vitest
```

What to test:

Focus on behaviour, not implementation:

```
// Testing a React component's behaviour
import { render, screen } from
'@testing-library/react';
import userEvent from
'@testing-library/user-event';
import Counter from './Counter';

describe('Counter', () => {
    it('starts at zero', () => {
        render(<Counter />);
        expect(screen.getByText('Count:
        0')).toBeInTheDocument();
    });

    it('increments when button clicked', async ()
    => {
        render(<Counter />);
        await
        userEvent.click(screen.getByRole('button'));
        expect(screen.getByText('Count:
        1')).toBeInTheDocument();
    });
});
```

```
Ask your AI:
I've heard of unit tests, integration tests, and
end-to-end tests.
What's the difference? As a beginner, where should
I focus my
testing efforts for maximum value?
```

17.4.3 Code Quality Practices

Meaningful names:

```
// Poor names
const d = new Date();
const x = users.filter(u => u.a > 18);

// Better names
const currentDate = new Date();
const adultUsers = users.filter(user => user.age >
```

```
18);
```

Small, focused functions:

```
// Too much in one function
function processOrder(order) {
    // Validate order (20 lines)
    // Calculate total (15 lines)
    // Apply discount (10 lines)
    // Process payment (25 lines)
    // Send confirmation (15 lines)
}

// Better: separate concerns
function validateOrder(order) { /* ... */ }
function calculateTotal(items) { /* ... */ }
function applyDiscount(total, discount) { /* ...
*/ }
function processPayment(amount, paymentMethod) {
/* ... */ }
function sendConfirmation(order, customer) { /*
... */ }

function processOrder(order) {
    validateOrder(order);
    const total = calculateTotal(order.items);
    const discountedTotal = applyDiscount(total,
    order.discount);
    processPayment(discountedTotal,
    order.paymentMethod);
    sendConfirmation(order, order.customer);
}
```

Comments that explain why:

```
// Poor comment (explains what)
// Loop through users
for (const user of users) {

// Better comment (explains why)
// Process users oldest-first to prioritise
long-term customers
const sortedUsers = users.sort((a, b) =>
```

```
a.joinDate - b.joinDate);
for (const user of sortedUsers) {
```

Consistent formatting:

Use a formatter like Prettier:

```
npm install -D prettier
npx prettier --write "src/**/*.{js,jsx}"
```

Use a linter like ESLint:

```
npm install -D eslint
npx eslint --init
npx eslint src/
```

These tools enforce consistency automatically, no arguments about tabs vs spaces.

17.4.4 Documentation Essentials

README.md (project overview):

````
# Project Name

Brief description of what this project does.

## Getting Started

### Prerequisites

- Node.js 18 or higher
- npm

### Installation

```bash
npm install
````

17.4.5 Running Locally

```
npm run dev
```
```

17.5 Usage

Basic usage examples here.

17.6 Contributing

How others can contribute.

17.7 License

MIT License

````
**Code documentation (JSDoc):**

```javascript
/**
 * Calculates the total price including tax.
 *
 * @param {number} subtotal - The pre-tax total
 * @param {number} taxRate - Tax rate as decimal
 (e.g., 0.1 for 10%)
 * @returns {number} The total including tax
 * @example
 * calculateTotal(100, 0.1) // Returns 110
 */
function calculateTotal(subtotal, taxRate) {
    return subtotal * (1 + taxRate);
}
````

17.8 Building Your Mental Model

17.8.1 The Professional Practice Pyramid

```
                    Code Review        (Team
                quality)

                  Documentation
              (Knowledge sharing)
```
```

```
                  Testing
          (Confidence)

                  Version Control    (Safety
     net)

                  Clean Code
  (Foundation)
```

Each layer builds on the one below. You can't effectively review code that isn't in version control. You can't confidently test code that's poorly structured. Start from the foundation.

17.8.2 The Cost of Skipping Practices

```
Time spent                    Technical Debt
debugging /
fixing


                Early in     Time      Later
                in
                project                project

vs.


Time spent

with good         (Steady, predictable)
practices

                Early in     Time      Later
                in
                project                project
```

Professional practices have upfront cost but prevent much larger debugging costs later.

17.8.3 When Each Practice Matters Most

Practice	Solo Project	Team Project	Long-term Maintenance
Version Control	Essential	Critical	Critical
Testing	Valuable	Essential	Critical
Code Quality	Valuable	Essential	Critical
Documentation	Helpful	Essential	Critical
Code Review	N/A	Essential	Essential

Even solo projects benefit from version control and testing. The "team" includes future you.

17.9 Business Applications

17.9.1 Risk Reduction

Version control supports business continuity:

- Roll back problematic deployments
- Trace when bugs were introduced
- Prove what was deployed when
- Recover from disasters (laptop theft, hardware failure)

For clients: "Your entire codebase is safely stored with complete history. If anything goes wrong, we can restore any previous version."

17.9.2 Quality Assurance

Testing provides business confidence:

- Catch bugs before customers do
- Verify features work after changes
- Reduce manual testing costs
- Enable faster, safer deployments

For clients: "Before any release, automated tests verify the critical functionality. This catches problems before your customers see them."

17.9.3 Collaboration and Onboarding

Good practices enable team growth:

- New developers understand the codebase faster
- Documentation reduces "tribal knowledge" dependency

- Consistent style means anyone can work on any file
- Code review spreads knowledge across the team

For clients: "If a developer leaves, their work is documented and reviewable. You're not dependent on any single person."

17.9.4 Long-term Maintainability

Code lives longer than developers expect:

- "Quick fix" becomes permanent
- "Temporary project" runs for years
- "Solo project" gets a team

Professional practices from the start prevent painful migrations later.

> **Skills You Are Building**
>
> This develops **professional communication and project management** and **evaluating and improving solutions**. Professional practices aren't separate from development, they're how professionals develop. These skills are expected in any development role.

17.10 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

17.10.1 Exercise 17.1: Git Fundamentals (Level 1)

Create a new project with Git:

1. Create a folder with an `index.html` file
2. Initialise a Git repository
3. Stage and commit the file with a meaningful message
4. Make changes to the file
5. View the diff, then commit the changes
6. View the commit history

Document the commands you used and what each did.

17.10.2 Exercise 17.2: Branching Workflow (Level 2)

Extend Exercise 17.1:

1. Create a branch called `feature/styling`
2. Add a `styles.css` file and link it in HTML
3. Commit the changes on the branch
4. Switch back to main
5. Verify the CSS file isn't there
6. Merge the feature branch into main
7. Verify the CSS file now exists on main

Document when you'd use this workflow in a real project.

17.10.3 Exercise 17.3: First Tests (Level 3)

Create a JavaScript module with tests:

1. Create a `utils.js` file with these functions:
   - `capitalise(string)` - capitalises the first letter
   - `truncate(string, length)` - shortens string to length with "…"
   - `slugify(string)` - converts to URL-safe slug
2. Install Vitest
3. Write tests for each function covering:
   - Normal cases
   - Edge cases (empty string, already capitalised)
   - Error cases (invalid input)
4. Run tests and ensure they pass

17.10.4 Exercise 17.4: Testing a Component (Level 4)

Create a React component with tests:

1. Build a `SearchFilter` component that:
   - Shows an input field
   - Displays a list of items
   - Filters items as user types
2. Write tests that verify:
   - Component renders with provided items
   - Typing filters the displayed items
   - Clearing input shows all items
   - Empty state displays appropriately

Focus on testing behaviour from the user's perspective.

17.10.5 Exercise 17.5: Professional Project Setup (Level 5)

Set up a complete project with professional practices:

1. Create a React project with Vite
2. Initialise Git repository
3. Create meaningful `.gitignore`
4. Set up ESLint and Prettier
5. Configure Vitest for testing
6. Write a comprehensive README
7. Create an initial commit with good message
8. Push to GitHub

Write 300 words explaining each decision you made and why these practices matter.

17.11 Chapter Summary

- Version control (Git) tracks changes and enables collaboration
- Testing provides confidence that code works correctly
- Code quality practices make code readable and maintainable
- Documentation serves future developers (including yourself)
- Professional practices have upfront cost but prevent much larger debugging costs later
- These practices are expected in professional development roles

17.12 Reflection

Before moving to the React Integration Project, ensure you can:

☐ Initialise a Git repository and make commits
☐ Create branches and merge them
☐ Write basic automated tests
☐ Explain why testing matters beyond "it works"
☐ Identify ways to make code more readable
☐ Create useful README documentation
☐ Articulate the business value of professional practices

17.13 Your Learning Journal

Record your responses to these prompts:

1. **Practice Value**: Which professional practice do you think will have the most impact on your work? Why?

2. **Real Experience**: Think of a time you lost work or couldn't remember why you made a decision. How would version control or documentation have helped?
3. **AI Conversation Reflection**: What professional practice concept was hardest to grasp? What question to your AI partner helped clarify it?
4. **Team Perspective**: If you were hiring a junior developer, which of these practices would you most want them to understand?

17.14 Next Steps

You now have both the technical skills (HTML, CSS, JavaScript, React, CSS frameworks) and professional practices (version control, testing, documentation) to build production-quality applications.

In the **React Integration Project** (Chapter 18), you'll bring everything together: build a React frontend that consumes WordPress content via API, styled with a CSS framework, using professional version control and documentation practices.

Chapter 18

Project: React Integration

> **Example implementations available online**
>
> The online edition at books.borck.education (https://books.borck.education) includes links to example implementations, starter code, and companion resources for each project.

18.1 Project Overview

The modern frontend chapters taught you component thinking, CSS frameworks, and professional practices. Now you'll bring it together: build a React frontend that consumes content from WordPress via the REST API, demonstrating headless architecture in action.

This project bridges backend content management with modern frontend development. You'll make architectural decisions, implement a complete application, and document your approach, skills that directly translate to professional frontend development roles.

18.2 Learning Outcomes Addressed

- **ULO 1**: Design and implement effective web applications
- **ULO 4**: Select and integrate appropriate technologies
- **ULO 5**: Evaluate emerging technologies and approaches

18.3 Technical Context

In Chapter 13, you learned that headless WordPress separates content management (WordPress) from content presentation (your frontend). This project implements that architecture:

```
              REST API

   WordPress                    React App

   (Backend)         JSON Data         (Frontend)



 - Content                            - Components

 - Media                              - Styling

 - Users                              - Routing
```

Your React application will fetch and display content that's managed in WordPress, the same content model used by major publishers, e-commerce sites, and enterprise applications.

18.4 Project Scenarios

Choose one scenario that builds on your WordPress site from the Business Site Project, or create new content:

18.4.1 Option A: Blog Frontend

Rebuild your WordPress blog as a modern React single-page application:

- Post listing with categories
- Individual post pages
- Author information display
- Search functionality
- Category filtering

18.4.2 Option B: Portfolio Showcase

Display portfolio items or projects from WordPress:

- Grid/list view of projects
- Project detail pages with images
- Category or technology filtering
- Contact form integration

18.4.3 Option C: Business Directory

Create a directory interface for business listings:

- Searchable listing grid
- Individual business pages
- Location or category filtering
- Featured listings section

18.4.4 Option D: Custom Application

Propose your own application that:

- Consumes WordPress REST API content
- Has at least two distinct views (list and detail)
- Includes meaningful filtering or search
- Demonstrates component architecture

18.5 Requirements

18.5.1 Phase 1: Planning and Setup

Before writing code, document your approach:

Application Design Document (300-500 words):

1. **Scenario Overview**: Which scenario did you choose and why?
2. **Component Architecture**: What components will you need? Draw a component tree.
3. **Data Requirements**: What WordPress content will you fetch? What endpoints?
4. **User Interface**: Sketch or describe the main views
5. **Technology Choices**: Which CSS framework? Why?

Component Tree Example:

```
App
  Header
      Logo
      Navigation
  Main
      PostList (or)
          PostCard (×n)
      PostDetail
          PostContent
          AuthorInfo
  Footer
```

18.5.2 Phase 2: React Implementation

Build the application with these requirements:

Core Setup:

- ☐ React project created with Vite
- ☐ Git repository initialised
- ☐ CSS framework installed and configured
- ☐ Project structure organised logically
- ☐ README with setup instructions

Component Architecture:

- ☐ At least 5 distinct components
- ☐ Proper use of props for data passing
- ☐ State management with useState
- ☐ Side effects with useEffect
- ☐ Components are reusable where appropriate

WordPress Integration:

- ☐ Fetch posts from WordPress REST API
- ☐ Handle loading states (spinner, skeleton, or message)
- ☐ Handle error states gracefully
- ☐ Display at least 3 post fields (title, content, excerpt, author, date, etc.)
- ☐ Images loaded from WordPress media

User Interface:

- ☐ List view showing multiple items
- ☐ Detail view showing single item
- ☐ Navigation between views (useState or React Router)
- ☐ Responsive design (works on mobile and desktop)

- ☐ Consistent styling with chosen CSS framework

Professional Practices:

- ☐ Meaningful Git commits throughout development
- ☐ Code formatted consistently (Prettier recommended)
- ☐ No console errors or warnings in final version
- ☐ Comments where logic isn't self-evident

18.5.3 Phase 3: Enhanced Features

Implement at least TWO of the following:

Search/Filter:

- ☐ Search posts by title or content
- ☐ Filter by category or tag
- ☐ Clear filters to show all

Pagination:

- ☐ Load more posts on demand
- ☐ Or implement traditional pagination
- ☐ Handle edge cases (no more posts, first page)

Routing:

- ☐ React Router for URL-based navigation
- ☐ Direct links to individual posts
- ☐ Back navigation works correctly

Additional Content:

- ☐ Display WordPress pages (not just posts)
- ☐ Show author information with avatar
- ☐ Display featured images properly

18.5.4 Phase 4: Documentation and Reflection

Technical Documentation:

Create a README.md that includes:

1. **Project Description**: What the application does
2. **Setup Instructions**: How to run locally
3. **WordPress Configuration**: Required WordPress setup
4. **Architecture Overview**: How components fit together
5. **API Endpoints Used**: Which WordPress endpoints and why

Reflection Document (500-700 words):

1. **Headless Architecture**: What are the advantages and disadvantages of this approach compared to traditional WordPress themes?
2. **Component Decisions**: Describe two components you created. Why did you structure them that way? What data do they receive and display?
3. **State Management**: Where does state live in your application? Why did you put it there?
4. **Challenges and Solutions**: What was the hardest part of this project? How did you solve it?
5. **Future Improvements**: If you had more time, what would you add or change?

18.6 AI Collaboration Guidelines

18.6.1 Effective AI Partnership

Use AI as a thinking partner, not a code generator:

```
Ask your AI:
I'm fetching WordPress posts but the featured
images aren't showing.
Here's my code... Help me understand what's
happening. What am I
missing about how WordPress returns image data?
```

```
Ask your AI:
I have a PostList component and a PostDetail
component. When the user
clicks a post, I need to show the detail. What are
my options for
handling this navigation? Walk me through the
trade-offs.
```

```
Ask your AI:
My component is getting complex with loading,
error, and data states.
Is there a pattern for handling this more cleanly?
Show me options
and explain when each makes sense.
```

18.6.2 AI Collaboration Log

Document your AI interactions:

Challenge	Question Asked	AI Response Summary	How I Applied It
(e.g., Images not loading)	…	…	…

For each significant interaction:

1. What problem were you solving?
2. What did you ask?
3. What did the AI explain?
4. How did you adapt the suggestion to your specific situation?
5. What did you learn that you'll use again?

18.6.3 Attribution Requirements

Be clear about AI involvement:

- Code you wrote yourself vs. code adapted from AI suggestions
- Concepts you learned through AI conversation
- Documentation created with AI assistance

The purpose is evidence that you **understood** what you built.

18.7 Evaluation Criteria

18.7.1 Planning and Design (15%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Application design	Clear architecture, thoughtful component breakdown	Good planning with minor gaps	Basic planning present	Incomplete or unclear
Technology justification	Clear reasoning for all choices	Good justification for most	Basic reasoning provided	No justification

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Component tree	Complete, logical hierarchy	Good structure, minor issues	Basic structure shown	Missing or illogical

18.7.2 React Implementation (35%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Component architecture	Well-structured, reusable, single responsibility	Good structure with minor issues	Functional but not well organised	Poor structure or monolithic
Props and state	Correct use, appropriate lifting	Mostly correct usage	Basic usage, some issues	Incorrect or missing
API integration	Robust fetching, proper error handling	Good integration, minor gaps	Basic fetching works	Broken or incomplete
useEffect usage	Correct dependencies, no unnecessary calls	Mostly correct	Functional but issues	Incorrect or missing

18.7.3 User Interface (25%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Visual design	Professional, consistent, polished	Good design, minor inconsistencies	Functional but basic	Broken or unstyled
Responsive design	Excellent on all screen sizes	Good responsive behaviour	Works but issues	Broken on mobile
Loading/error states	Clear, helpful, consistent	Present and functional	Basic handling	Missing or broken

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Navigation	Intuitive, works correctly	Good navigation, minor issues	Functional but awkward	Broken or missing

18.7.4 Professional Practices (10%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Git usage	Meaningful commits throughout	Good commit history	Some commits	No commits or single commit
Code quality	Clean, consistent, readable	Good quality, minor issues	Functional but messy	Poor quality
README	Complete, clear, helpful	Good documentation	Basic documentation	Missing or incomplete

18.7.5 Reflection and AI Collaboration (15%)

Criteria	Excellent (4)	Good (3)	Adequate (2)	Needs Work (1)
Headless architecture reflection	Deep understanding demonstrated	Good understanding shown	Basic understanding	Missing or superficial
Component decisions	Clear reasoning, shows learning	Good explanations	Basic explanations	Missing or unclear
AI collaboration log	Detailed, shows critical thinking	Good documentation	Basic log present	Missing or superficial
Learning demonstrated	Clear growth and understanding	Good learning shown	Some learning evident	No learning demonstrated

18.8 Submission Checklist

Code:

- ☐ Complete React source code (exclude `node_modules`)
- ☐ `package.json` with all dependencies
- ☐ `.gitignore` properly configured
- ☐ No API keys or secrets in code

Documentation:

- ☐ README.md with setup instructions
- ☐ Application design document
- ☐ Reflection document (500-700 words)
- ☐ AI collaboration log

Demonstration:

- ☐ Screenshots of main views (list, detail, mobile)
- ☐ Screenshot of loading state
- ☐ Screenshot of error state (simulated if necessary)

WordPress:

- ☐ WordPress export file (if content is needed)
- ☐ Or note that app uses public API/test data

18.9 Getting Started

Begin with these conversations:

```
Ask your AI:
I'm building a React app that fetches WordPress
posts. What's the
basic structure I should start with? Walk me
through the component
hierarchy and data flow.
```

```
Ask your AI:
In a React app, I need to show a list of posts and
then show details
when one is clicked. What are my options for
handling this without
React Router? Explain the useState approach.
```

```
Ask your AI:
My WordPress site is at [URL]. What does the REST
API endpoint for
posts look like? How do I get featured images
included?
```

18.10 Common Pitfalls to Avoid

1. **Skipping the plan**: Jumping into code without designing component architecture leads to refactoring
2. **One giant component**: Put everything in App.js instead of breaking into focused components
3. **Ignoring loading states**: App breaks or shows nothing while data loads
4. **No error handling**: App crashes when API fails instead of showing helpful message
5. **Props drilling confusion**: Not understanding where state should live
6. **Not using keys in lists**: Causes warnings and potential bugs
7. **Committing node_modules**: Makes repository huge and unusable
8. **Hardcoded URLs**: API endpoints should be configurable or documented

18.11 Technical Tips

Fetching WordPress posts with featured images:

```
// Include _embed to get featured media data
const response = await fetch(
    'http://your-site.local/wp-json/wp/v2/posts?_embed'
);
const posts = await response.json();

// Access featured image URL
const imageUrl =
post._embedded?.['wp:featuredmedia']?.[0]?.source_url;
```

Project structure suggestion:

```
src/
  components/
      Header.jsx
      PostCard.jsx
      PostList.jsx
```

```
      PostDetail.jsx
      LoadingSpinner.jsx
  App.jsx
  main.jsx
  index.css
```

useState for view navigation:

```
const [selectedPost, setSelectedPost] =
useState(null);

// In render
{selectedPost ? (
    <PostDetail post={selectedPost} onBack={() =>
    setSelectedPost(null)} />
) : (
    <PostList posts={posts}
    onSelectPost={setSelectedPost} />
)}
```

18.12 Professional Context

This project demonstrates skills employers look for:

1. **Frontend architecture**: Structuring React applications logically
2. **API integration**: Working with external data sources
3. **State management**: Handling application data flow
4. **User experience**: Loading states, error handling, responsive design
5. **Professional practices**: Version control, documentation, code quality

Headless architecture is increasingly common in enterprise development. Understanding how to build frontends that consume APIs, whether WordPress, a custom backend, or third-party services, is valuable regardless of your eventual specialisation.

18.13 After Submission

This project, combined with your WordPress Business Site, demonstrates full-stack thinking:

- Backend: WordPress content management
- API: REST interface design

- Frontend: React component architecture
- Professional: Version control, documentation, testing

Consider extending this portfolio piece by:

- Adding more interactive features
- Implementing React Router for proper URLs
- Adding tests with Vitest
- Deploying to a hosting service

These extensions show initiative and technical depth to potential employers.

Part VI

The Professional Developer

Chapter 19

Evaluating Emerging Technologies

19.1 The Concept First

In Chapter 17, you learned practices that make developers effective. But technologies change constantly. The frameworks popular today may be legacy tomorrow. The "best practices" of five years ago may now be anti-patterns.

This creates a challenge: **how do you evaluate new technologies without getting swept up in hype?**

You cannot know every new tool. Instead, develop a **systematic evaluation framework** that works for any technology.

Consider the difference:

- **Chasing trends**: "Everyone's using X, we should too"
- **Systematic evaluation**: "X solves problem Y with trade-offs Z. Given our context, is it appropriate?"

The second approach serves you for your entire career, regardless of which technologies rise and fall.

19.2 Understanding Through Investment Thinking

Adopting a technology is an investment decision. Like financial investments, technology investments have:

- **Upfront costs**: Learning time, integration effort, migration pain
- **Ongoing costs**: Maintenance, updates, ecosystem changes

- **Expected returns**: Productivity gains, capability improvements, competitive advantages
- **Risks**: Technology abandonment, breaking changes, hiring difficulties

Smart investors don't just ask "Will this stock go up?" They ask "Given my goals, timeline, and risk tolerance, is this the right investment?"

Smart technologists don't just ask "Is this technology good?" They ask "Given our problem, team, timeline, and constraints, is this technology appropriate?"

> The Hype Cycle Awareness
>
> New technologies typically follow a pattern: initial hype, peak of inflated expectations, trough of disillusionment, and finally a plateau of productivity. Understanding where a technology sits on this curve helps calibrate your evaluation.

19.3 Discovering Evaluation with Your AI Partner

19.3.1 Exploration 1: Building an Evaluation Framework

```
Ask your AI:
Help me create a comprehensive framework for
evaluating whether a
business should adopt a new web technology. What
categories of
questions should I ask? Walk me through the
thinking process.
```

A good framework might include:

Problem Fit: - What specific problem does this solve? - Do we actually have this problem? - How are we solving it currently? - How painful is the current solution?

Maturity and Stability: - How long has this technology existed? - Who is using it in production? - How frequently do breaking changes occur? - What's the governance model (company, community, foundation)?

Ecosystem and Support: - What's the documentation quality? - How active is the community? - Are there quality learning resources? - Can we hire people who know this?

Integration and Migration: - How does this fit with our existing stack? - What's the migration path? - Can we adopt incrementally? - What's the rollback strategy?

Total Cost: - What's the learning curve? - What are ongoing maintenance requirements? - Are there licensing costs? - What's the opportunity cost of adoption time?

```
Continue the conversation:
Now apply this framework to evaluate whether a
small business
should adopt TypeScript for their JavaScript
projects. Walk me
through each category.
```

19.3.2 Exploration 2: Distinguishing Hype from Value

```
Ask your AI:
How do you distinguish technology hype from
genuine value? Give me
examples from web development history,
technologies that were hyped
but didn't deliver, and technologies that seemed
overhyped but
proved genuinely valuable.
```

This should reveal patterns:

Signs of hype without substance: - Solves problems most projects don't have - Benefits are vague ("makes development better") - Requires rewriting everything to adopt - Community focuses on features, not outcomes

Signs of genuine value: - Addresses real, common pain points - Benefits are concrete and measurable - Can be adopted incrementally - Users talk about problems solved, not features

```
Continue the conversation:
What red flags should I watch for when evaluating
a new JavaScript
framework? What would make you cautious?
```

19.3.3 Exploration 3: Current Technology Landscape

Technologies change, but evaluation skills persist. Let's practice:

```
Ask your AI:
What web technologies are currently gaining
significant adoption?
```

```
For each, explain: (1) what problem it solves, (2)
what trade-offs
it makes, and (3) what type of project would
benefit most from it.
```

As of this book's writing, relevant technologies include:

- **TypeScript**: Type safety for JavaScript
- **Server-side rendering (SSR)**: Performance and SEO
- **Edge computing**: Latency and global distribution
- **Progressive Web Apps (PWAs)**: Native-like web experiences
- **AI-assisted development**: Code generation and assistance

```
Continue the conversation:
For a team of three developers building a customer
portal for a
medium-sized business, which of these technologies
would you
recommend evaluating seriously? Which would you
suggest ignoring
for now? Explain your reasoning.
```

19.3.4 Exploration 4: Learning from History

```
Ask your AI:
Tell me about technologies that were "must learn"
five years ago
but are now less relevant. What can we learn from
this about
evaluating current technologies?
```

This develops healthy scepticism. Technologies that seemed essential often:

- Were replaced by simpler alternatives
- Solved problems that platforms eventually solved
- Had high adoption costs that weren't justified
- Were driven by specific company interests

19.4 From Concept to Code

19.4.1 TypeScript: Adding Types to JavaScript

TypeScript adds static typing to JavaScript. The evaluation:

Problem it solves: - Runtime errors from type mismatches - Difficulty understanding code at scale - Refactoring without confidence - Poor IDE support for large codebases

Trade-offs: - Additional compilation step - Learning curve for type system - More verbose code - Build tooling complexity

Example comparison:

```
// JavaScript
function calculateDiscount(price, percentage) {
    return price * (percentage / 100);
}

// Works at runtime, but wrong results
calculateDiscount("100", "10");  // "100" repeated
0.1 times = NaN-ish behaviour
calculateDiscount(100);          // NaN
(percentage is undefined)
```

```
// TypeScript
function calculateDiscount(price: number,
percentage: number): number {
    return price * (percentage / 100);
}

// Errors caught at compile time
calculateDiscount("100", "10");  // Error:
Argument of type 'string' is not assignable
calculateDiscount(100);          // Error:
Expected 2 arguments, but got 1
```

When TypeScript makes sense: - Teams larger than 2-3 people - Codebases expected to grow significantly - APIs consumed by multiple clients - Long-lived projects

When to skip TypeScript: - Quick prototypes - Solo projects with short lifespans - Teams with no TypeScript experience and tight deadlines

```
Ask your AI:
I'm starting a new React project. It's a dashboard
for internal use
by about 50 employees, built by me and one other
developer, expected
to be maintained for 3-5 years. Should we use
```

```
TypeScript? Walk me
through the decision.
```

19.4.2 Progressive Web Apps (PWAs)

PWAs use web technologies to deliver app-like experiences.

Problem they solve: - App store friction for distribution - Need for offline functionality - Push notification capability - Installation on home screen

Trade-offs: - More complex than standard websites - iOS support limitations - Not all native features available - User education about "installation"

Basic PWA structure:

```
// service-worker.js (simplified)
const CACHE_NAME = 'my-app-v1';
const ASSETS_TO_CACHE = [
    '/',
    '/index.html',
    '/styles.css',
    '/app.js'
];

// Cache assets on install
self.addEventListener('install', event => {
    event.waitUntil(
        caches.open(CACHE_NAME)
            .then(cache =>
            cache.addAll(ASSETS_TO_CACHE))
    );
});

// Serve from cache when offline
self.addEventListener('fetch', event => {
    event.respondWith(
        caches.match(event.request)
            .then(response => response ||
            fetch(event.request))
    );
});
```

```
// manifest.json
{
    "name": "My Application",
    "short_name": "MyApp",
    "start_url": "/",
    "display": "standalone",
    "background_color": "#ffffff",
    "theme_color": "#3498db",
    "icons": [
        {
            "src": "/icon-192.png",
            "sizes": "192x192",
            "type": "image/png"
        }
    ]
}
```

When PWAs make sense: - Content-focused apps needing offline access - Avoiding app store distribution - Cross-platform with single codebase - Re-engagement via notifications

When to use native/hybrid instead: - Heavy device feature requirements - Performance-critical applications - App store presence is valuable - Complex offline data sync needs

19.4.3 Server Components and SSR

Modern frameworks offer server-side rendering and server components.

Problem they solve: - Slow initial page load (JavaScript bundle download) - Poor SEO for dynamic content - Layout shift after hydration - Server-client data duplication

Trade-offs: - Server infrastructure required - More complex mental model - Debugging across client/server boundary - Caching complexity

Mental model:

```
Traditional SPA:
Browser → Request HTML → Get minimal HTML
       → Download large JS bundle
       → JS renders content
       → User sees content (slow)
```

```
SSR:
Browser → Request HTML → Server renders full HTML
       → User sees content immediately
       → JS downloads and "hydrates"
       → Interactivity ready
```

```
Ask your AI:
My React app loads slowly because it fetches data
after the page
loads, showing spinners everywhere. Would
server-side rendering
help? What would change in how I build the app?
```

19.4.4 Evaluating AI-Assisted Development

AI coding assistants are a current example of technology requiring evaluation.

Problem they solve: - Boilerplate code writing - Remembering syntax and APIs - Exploring unfamiliar codebases - Documentation lookup

Trade-offs: - Code quality varies - May perpetuate bad patterns - Over-reliance reduces learning - Security/privacy considerations

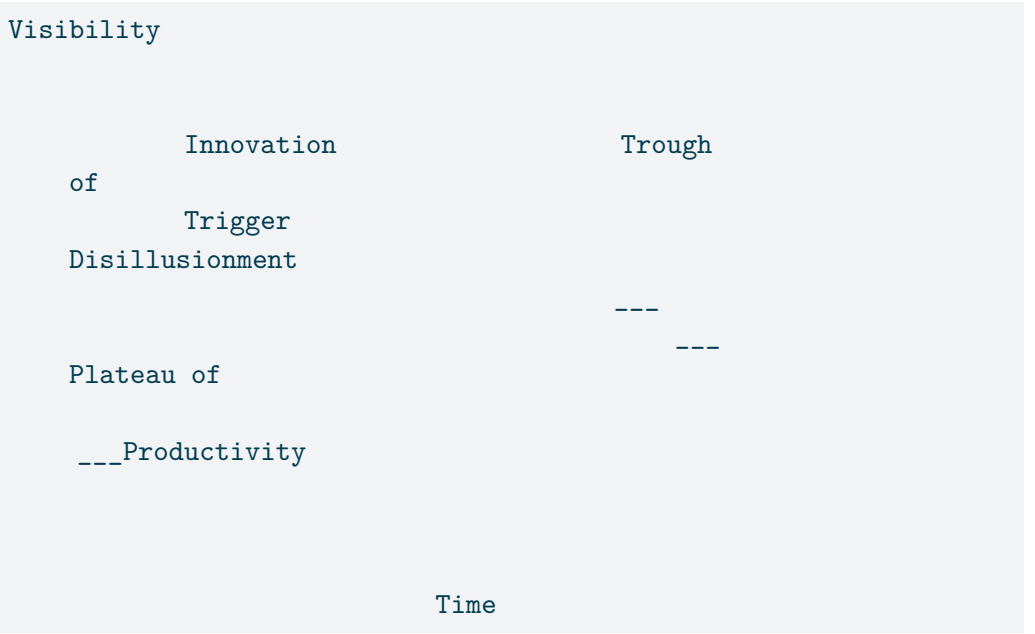
Evaluation questions: - Does AI assistance improve code quality or just speed? - Are developers learning or just accepting suggestions? - What happens when AI is unavailable? - How do we review AI-generated code?

```
Ask your AI:
As an AI, what limitations should I be aware of
when using AI
coding assistants? When should I trust AI
suggestions versus
being sceptical?
```

19.5 Building Your Mental Model

19.5.1 The Technology Adoption Curve

```
                    Peak of
                    Inflated
                    Expectations
```



Technologies at different stages need different evaluation approaches:

- **Innovation trigger**: High risk, high uncertainty, evaluate carefully
- **Peak of expectations**: Hype is highest, scepticism warranted
- **Trough of disillusionment**: Early adopters struggling, realistic assessments emerge
- **Plateau of productivity**: Proven technology, lower risk, realistic expectations

19.5.2 The Adoption Decision Matrix

Project Characteristics	Technology Risk Tolerance
Short-lived, experimental	Higher (can pivot easily)
Long-lived, critical	Lower (need stability)
Small team, familiar stack	Lower (context switching cost)
Growing team, scaling	Higher (need better tools)
Tight deadline	Lower (no time to learn)
Greenfield project	Higher (no migration cost)

19.5.3 Questions That Reveal Reality

When evaluating technology, these questions cut through marketing:

1. **"What will we stop doing?"** - Adoption always has costs

2. **"Who has used this for 2+ years?"** - Early success doesn't mean long-term success
3. **"What's the worst-case migration?"** - If it fails, what's the recovery?
4. **"Who maintains this?"** - Single company? Open community? Foundation?
5. **"What problems does the community complain about?"** - Reveals real issues

19.6 Business Applications

19.6.1 Strategic Technology Planning

Technology evaluation skills enable business strategy:

- **Roadmap development**: When to adopt which technologies
- **Risk assessment**: Understanding adoption risks
- **Competitive analysis**: What technologies enable competitors
- **Talent planning**: What skills will be needed

19.6.2 Client Advisory

Clients often ask about new technologies. Professional response:

Poor: "Yes, blockchain is great, we should use it."

Better: "Let me understand your problem first. Blockchain solves specific problems, immutable record-keeping, decentralised consensus. Do you have those problems? If not, simpler solutions exist."

19.6.3 Career Development

Evaluation skills help you:

- **Prioritise learning**: Focus on technologies with staying power
- **Avoid burnout**: Stop chasing every new framework
- **Build expertise**: Go deep on well-chosen technologies
- **Stay relevant**: Recognise genuine shifts versus fads

> **i** Skills You Are Building
>
> This develops **assessing emerging technologies**. The ability to evaluate technologies objectively, separate from hype and personal preference, is essential for advising businesses on technology decisions.

19.7 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

19.7.1 Exercise 19.1: Framework Application (Level 1)

Apply the evaluation framework to a technology of your choice:

1. Choose a web technology you've heard about but haven't used
2. Research it using the framework categories (problem fit, maturity, ecosystem, integration, cost)
3. Document your findings for each category
4. Write a one-paragraph recommendation

19.7.2 Exercise 19.2: Historical Analysis (Level 2)

Research a technology that was popular 5 years ago but is less used now:

1. What problem did it solve?
2. Why did adoption decline?
3. What replaced it?
4. What could evaluators have noticed earlier?

Write 300 words on lessons learned.

19.7.3 Exercise 19.3: Comparative Evaluation (Level 3)

You need to choose between two competing technologies (e.g., two CSS frameworks, two state management libraries, two build tools):

1. Define your evaluation criteria
2. Research both technologies
3. Score each against your criteria
4. Make a recommendation with clear reasoning
5. Note what would change your recommendation

19.7.4 Exercise 19.4: Context-Dependent Recommendation (Level 4)

A client asks whether they should adopt a specific new technology. Create three different recommendations based on three different client contexts:

1. Early-stage startup with 2 developers, moving fast
2. Established company with 20 developers, maintaining legacy systems
3. Enterprise with strict compliance requirements

For each, explain how context changes the recommendation.

19.7.5 Exercise 19.5: Technology Radar (Level 5)

Create your own "Technology Radar" for web development:

1. Research 8-12 current web technologies
2. Categorise each as: Adopt, Trial, Assess, or Hold
3. For each, write 100-150 words justifying the categorisation
4. Note your assumptions and biases
5. Identify what new information would change your categorisations

This mirrors how technology advisory firms like ThoughtWorks communicate recommendations.

19.8 Chapter Summary

- Evaluation frameworks outlast specific technologies
- Technology adoption is an investment decision with costs and risks
- Context determines whether a technology is appropriate
- The hype cycle helps calibrate expectations
- Questions that reveal trade-offs are more valuable than feature lists
- Professional technologists advise based on client needs, not personal preferences

19.9 Reflection

Before moving on, ensure you can:

☐ Apply a systematic evaluation framework to any technology
☐ Identify signs of hype versus genuine value
☐ Explain how context affects technology recommendations
☐ Describe the hype cycle and its implications
☐ Ask questions that reveal technology trade-offs
☐ Make recommendations appropriate to specific business contexts

19.10 Your Learning Journal

Record your responses to these prompts:

1. **Personal Bias Awareness**: What technologies are you biased toward or against? How might this affect your evaluations?
2. **Hype Recognition**: Think of a technology you were excited about that didn't deliver. What signs did you miss?
3. **AI Conversation Reflection**: What technology did you evaluate with AI help? What did you learn about the evaluation process?
4. **Future Prediction**: What technology do you think will be much more or less important in 5 years? What's your reasoning?

19.11 Next Steps

You now have tools to evaluate any technology that emerges. But technology skills alone don't build a career.

In Chapter 20, we'll focus on your professional development, building a portfolio, positioning yourself in the market, networking effectively, and planning continuous growth.

Chapter 20

Your Development Career

20.1 The Concept First

In Chapter 19, you learned to evaluate technologies systematically. But technical skills alone don't build careers; they create potential. Converting that potential into opportunities requires a different skill set: **professional development**.

Consider two developers with identical technical abilities:

- **Developer A**: Strong skills, no online presence, applies to jobs cold, waits for opportunities
- **Developer B**: Strong skills, portfolio showing work, active in communities, creates opportunities

Developer B will have more options, better offers, and a clearer path forward, because they have made their talent **visible and accessible**.

This chapter isn't about self-promotion or personal branding gimmicks. It's about the practical work of building a professional presence that accurately represents your capabilities and connects you with opportunities that match your goals.

20.2 Understanding Through Career Capital

Think of your career as a form of capital accumulation. You build different types of capital:

Technical Capital: Your skills and knowledge - The languages you know - The frameworks you've used - The problems you've solved

Social Capital: Your professional relationships - People who know your work - Communities you contribute to - References who vouch for you

Reputation Capital: How you're perceived - Your portfolio and public work - Contributions others can see - Your professional presence

Most developers focus only on technical capital. But technical capital without social and reputation capital means opportunities don't find you, you have to chase every one.

> The Visibility Principle
>
> Skills that people can see are worth more than skills they can't. A public project demonstrates capability in ways a resume cannot. Documentation you've written, code you've shared, and problems you've solved publicly all create evidence of competence.

20.3 Discovering Career Building with Your AI Partner

20.3.1 Exploration 1: Portfolio Strategy

```
Ask your AI:
I'm building a web development portfolio. What
types of projects
should I include? How do I make projects stand out
when thousands
of developers have similar portfolios?
```

Points to notice:

Quality over quantity: Three excellent projects beat ten mediocre ones

Show thinking, not just code: - Why you made certain decisions - Challenges you overcame - What you'd do differently

Variety that tells a story: - Different technologies demonstrating range - Increasing complexity showing growth - Business problems solved, not just technical exercises

```
Continue the conversation:
I have the projects from this course, the
portfolio site, WordPress
business site, and React integration. How do I
present these
professionally? What should I add or emphasise?
```

20.3.2 Exploration 2: Professional Positioning

You can't be excellent at everything. Positioning means being known for something specific.

```
Ask your AI:
What does "positioning" mean for a developer's
career? How do I
choose what to be known for when I'm just starting
out?
```

Positioning focuses your message:

- **Generalist trap**: "I do everything" means "Hire me for anything" which often means "Hire someone else who specialises"
- **Specialist value**: "I build accessible React applications" is memorable and referable
- **Evolution**: Your positioning can change as you grow

```
Continue the conversation:
Based on what I've learned in this course
(HTML/CSS, JavaScript,
WordPress, React), what are realistic positioning
options for
someone entering the job market?
```

20.3.3 Exploration 3: Networking Without Networking

Many developers dislike traditional networking. But professional relationships matter.

```
Ask your AI:
I'm introverted and don't enjoy networking events.
How can I
build professional relationships authentically as
a developer?
What works better than collecting business cards?
```

Developer-friendly relationship building:

- **Contribute to open source**: Work alongside others, earn respect through code
- **Answer questions**: Stack Overflow, Reddit, Discord communities
- **Write about learning**: Blog posts help others and demonstrate expertise
- **Attend (or speak at) meetups**: Even quietly attending builds familiarity

```
Continue the conversation:
What's the minimum viable networking for a junior
developer?
What activities have the highest return for the
time invested?
```

20.3.4 Exploration 4: Continuous Learning

Technology changes. Your learning must continue.

```
Ask your AI:
How do I keep learning after formal education
ends? How do I avoid
tutorial hell, watching courses without building
real skills?
```

Effective continuous learning:

- **Learn by doing**: Projects beat tutorials
- **Just-in-time learning**: Learn what you need for current work
- **Depth over breadth**: Master one thing rather than sampling many
- **Teach to learn**: Explaining forces understanding

```
Continue the conversation:
How do I decide what to learn next? There are so
many technologies.
How do I prioritise without chasing every trend?
```

20.4 From Concept to Code

20.4.1 Building Your Portfolio

Your portfolio demonstrates capability. Structure it intentionally.

Portfolio website essentials:

```
Portfolio Site
   About
       Brief professional introduction
       What you're looking for
       Contact information
   Projects
       Featured project 1 (most impressive)
```

```
      Featured project 2
      Featured project 3
   (Each with: description, technologies,
challenges, links)
  Skills
      Technologies with honest proficiency
levels
  Blog/Writing (optional but valuable)
       Technical posts showing your thinking
```

Project case study template:

```
## Project Name

**Overview**: One paragraph explaining what this
is and why it matters.

**The Challenge**: What problem did this solve?
What constraints existed?

**My Approach**: Key decisions and why you made
them.

**Technologies Used**: List with brief explanation
of why each was chosen.

**Key Features**:
- Feature 1 and its purpose
- Feature 2 and its purpose
- Feature 3 and its purpose

**Challenges Overcome**: Specific problems you
solved (this shows learning).

**Results**: If applicable, metrics or outcomes.

**What I Learned**: Reflection that shows growth
mindset.

**Links**: Live demo, GitHub repository, related
blog post.
```

Example project description:

```
## Business Dashboard (React + WordPress)

**Overview**: A React single-page application that
displays business
data from a WordPress backend, demonstrating
headless architecture.

**The Challenge**: Create a modern, fast frontend
while allowing
non-technical staff to manage content through
WordPress.

**My Approach**: Chose React for component
reusability and WordPress
REST API for content management. Used Tailwind for
rapid styling.
Prioritised loading states and error handling for
reliability.

**Technologies**: React, WordPress REST API,
Tailwind CSS, Vite

**Key Features**:
- Dynamic content fetched from WordPress
- Responsive design (mobile-first)
- Loading skeletons for perceived performance
- Filter and search functionality

**Challenges Overcome**: WordPress featured images
required the
`_embed` parameter, debugging this taught me to
read API documentation
more carefully and test endpoints directly.

**What I Learned**: Headless architecture
separates concerns
effectively. The frontend can evolve independently
of the CMS.
I'd add TypeScript next time for better
maintainability.
```

```
**Links**: [Live Demo] [GitHub] [Blog Post:
Headless WordPress Lessons]
```

20.4.2 GitHub Profile Optimisation

Your GitHub profile is often viewed before your resume.

Profile README (create a repository named after your username):

```
# Hi, I'm [Your Name]

I build web applications with a focus on [your
positioning].

## Currently Working On
-  [Current project]
-  Learning [current focus]

## Recent Projects
- Project 1 - Brief description
- Project 2 - Brief description
- Project 3 - Brief description

## Technologies
![JavaScript](badge) ![React](badge)
![WordPress](badge)

## Get in Touch
- Portfolio: [link]
- LinkedIn: [link]
- Email: [email]
```

Repository presentation:

- Clear README for every project
- Live demo links where applicable
- Descriptive commit messages (people read these)
- Consistent activity (small, regular commits beat sporadic large ones)

20.4.3 Writing for Professional Development

Writing demonstrates expertise and aids learning.

Blog post formats that work:

1. **Tutorial**: "How I Built X with Y"
    - Teach something you learned
    - Include code and explanation
    - Address common pitfalls
2. **Problem/Solution**: "How I Fixed X"
    - Describe the problem clearly
    - Explain your debugging process
    - Share the solution with context
3. **Comparison**: "X vs Y: When to Use Each"
    - Fair evaluation of alternatives
    - Specific use cases for each
    - Your recommendation with reasoning
4. **Learning Journey**: "What I Learned Building X"
    - Honest reflection
    - Mistakes and lessons
    - Advice for others

Writing tips:

- Write for your past self (what did you need to know?)
- Include code examples (developers scan for code)
- Be concise (respect readers' time)
- Proofread (errors undermine credibility)

```
Ask your AI:
I want to write a blog post about something I
learned while building
my React project. Help me brainstorm topics that
would be useful to
other beginners and demonstrate my understanding.
```

20.4.4 LinkedIn for Developers

LinkedIn matters for job searching, even if you don't love it.

Profile optimisation:

- **Headline**: Not just "Student" but "Web Developer | React, WordPress, JavaScript"
- **Summary**: Brief story of your path and what you're looking for
- **Experience**: Include projects, not just jobs
- **Skills**: List specific technologies
- **Recommendations**: Request from collaborators, instructors, colleagues

Activity that helps:

- Share your blog posts
- Comment thoughtfully on industry discussions
- Celebrate completions (courses, projects, certifications)
- Engage with content from companies you're interested in

20.4.5 The Job Search Process

When you're ready to apply:

Application strategy:

1. **Target, don't spray**: Quality applications beat quantity
2. **Research the company**: Reference specific things in applications
3. **Customise materials**: Generic applications get generic rejections
4. **Follow up appropriately**: Once, after a week
5. **Track everything**: Spreadsheet with company, role, date, status

Technical interviews:

- Practice coding problems (but don't only practice problems)
- Be ready to discuss your projects in depth
- Ask thoughtful questions about their work
- Follow up with thank-you notes

Portfolio presentations:

When asked to present work:

1. Start with the problem, not the technology
2. Explain your decision-making process
3. Be honest about challenges and what you'd do differently
4. Connect technical choices to business outcomes
5. Prepare for questions about alternatives you considered

20.5 Building Your Mental Model

20.5.1 Career Capital Accumulation



```
            (Skills)        (Visible
                            work)



                  Social Capital
                  (Relationships)



                    Your Effort
```

All three types of capital reinforce each other. Skills let you build visible work. Visible work attracts relationships. Relationships lead to opportunities that build more skills.

20.5.2 The Career Flywheel

```
Build Skills → Create Projects → Share Work →
Build Relationships



           Get Opportunities
```

Each element powers the next. Starting anywhere gets the wheel turning.

20.5.3 Positioning Quadrant

```
                   Specialist



  Frontend    "React      "Full-     Full-Stack
               Dev"        Stack
                           Dev"
```

```
              "WordPress   "Agency
                Dev"        Dev"



                    Generalist
```

Different positions have different advantages:

- **Specialist**: Higher rates, clearer referrals, deeper expertise
- **Generalist**: More flexibility, broader opportunities, varied work
- **Neither is better**, choose based on your preferences and market

20.6 Business Applications

20.6.1 Understanding the Hiring Perspective

Employers evaluating candidates consider:

- **Risk**: Will this person succeed or create problems?
- **Evidence**: What demonstrates capability?
- **Fit**: Will they work well with the team?
- **Growth**: Can they improve and adapt?

Your portfolio and presence reduce perceived risk by providing evidence. Everything you make public helps employers answer "Can this person do the job?"

20.6.2 The Value of Visible Contributions

Open source contributions, blog posts, and public projects:

- Demonstrate skills without requiring trust
- Show communication ability (code is communication)
- Indicate engagement with the community
- Provide conversation starters in interviews

Even small contributions count. A typo fix in documentation shows you engage with the ecosystem.

20.6.3 Long-term Career Planning

Think beyond the first job:

- **Year 1-2**: Build foundation, try different areas
- **Year 3-5**: Develop specialisation, build reputation

- **Year 5+**: Leadership opportunities, consulting, teaching

Each stage requires different strategies. Early career is about learning and visibility. Later career is about influence and impact.

> **Skills You Are Building**
>
> This develops **professional communication and project management**. Career development isn't separate from technical work, it's how you translate technical capability into professional opportunity. These skills matter regardless of which technologies you ultimately work with.

20.7 Practice Exercises

> **Exercise Levels**
>
> - **Level 1**: Direct application
> - **Level 2**: Minor modifications
> - **Level 3**: Combining concepts
> - **Level 4**: Problem-solving
> - **Level 5**: Open-ended design

20.7.1 Exercise 20.1: Portfolio Audit (Level 1)

Evaluate your current portfolio presence:

1. Google your name, what comes up?
2. Review your GitHub profile, what does it communicate?
3. Check your LinkedIn, is it complete and current?
4. List three things you could improve immediately

20.7.2 Exercise 20.2: Project Case Study (Level 2)

Write a case study for one of your projects using the template provided:

1. Choose your strongest project
2. Complete all sections of the template
3. Get feedback from a peer
4. Revise based on feedback

20.7.3 Exercise 20.3: Professional Positioning (Level 3)

Develop your professional positioning:

1. List your strongest skills
2. Identify which combinations are distinctive
3. Research job postings that match your skills
4. Write a positioning statement (one sentence that describes what you do)
5. Get feedback, is it clear? Memorable? Accurate?

20.7.4 Exercise 20.4: Content Creation (Level 4)

Create professional content:

1. Write a technical blog post about something you learned
2. Publish it (dev.to, Medium, personal blog, or LinkedIn)
3. Share it on one social platform
4. Reflect: What was harder than expected? What did you learn from writing?

20.7.5 Exercise 20.5: Career Strategy (Level 5)

Develop a 12-month career strategy:

1. Define where you want to be in one year
2. Identify the gaps between current state and goal
3. Create a plan with monthly milestones
4. Include: skills to develop, projects to build, connections to make, content to create
5. Write 500 words explaining your strategy and reasoning

20.8 Chapter Summary

- Technical skills create potential; professional presence creates opportunities
- Portfolios should show thinking, not just code
- Positioning helps you be memorable and referable
- Relationships develop through contribution, not just networking
- Continuous learning requires intentional practice, not just consumption
- Career development is a long game, invest consistently

20.9 Reflection

Before moving on, ensure you can:

☐ Explain what makes a strong portfolio project
☐ Write a compelling project case study
☐ Describe your professional positioning
☐ Identify at least three ways to build professional relationships

- ☐ Plan continuous learning without chasing every trend
- ☐ Understand what employers look for beyond technical skills

20.10 Your Learning Journal

Record your responses to these prompts:

1. **Portfolio Assessment**: What's the strongest thing in your current portfolio? What's the biggest gap?
2. **Positioning Exploration**: What do you want to be known for? How does this align with your interests and the market?
3. **AI Conversation Reflection**: What career question did you explore with AI? What insight was most valuable?
4. **Networking Comfort**: What forms of professional relationship building feel authentic to you? What would you avoid?

20.11 Next Steps

You now have frameworks for building your professional presence and planning your career development.

In Chapter 21, we'll conclude by examining what it means to be a developer in the AI era. How do AI tools change the profession? What skills remain essential? How do you prepare for a future where AI capabilities continue to expand?

Chapter 21

The AI-Era Developer

21.1 The Concept First

Throughout this book, you've learned web development with AI as your partner. You've used AI to explore concepts, debug problems, generate code, and build understanding. Now ask: **What does it mean to be a developer in an era where AI can write code?**

Job security is part of that question, but the larger part is where your value lies and which skills matter.

Consider what you've actually done in this course:

- You **decided** what to build
- You **evaluated** whether solutions were appropriate
- You **understood** why code worked (or didn't)
- You **communicated** with stakeholders (real or imagined)
- You **judged** trade-offs between approaches
- You **adapted** suggestions to your specific context

AI assisted with all of this. But AI didn't do any of this **for** you. The judgment, understanding, and decision-making were yours.

AI amplifies human capability rather than replacing human judgment.

21.2 Understanding Through Partnership Evolution

Think about how your relationship with AI has evolved through this book.

Early in your journey (the Web Foundations chapters): - AI explained concepts - You asked "how does this work?" - AI provided examples - You learned fundamentals

As you grew (the platform and component chapters): - You asked "which approach is better?" - AI provided options with trade-offs - You made decisions based on context - AI helped implement your decisions

Now (the professional developer chapters): - You ask "is this appropriate for this situation?" - AI provides analysis - You evaluate and judge - The partnership is collaborative

This evolution reflects how professionals actually work with AI. Beginners need more guidance. Experts need thinking partners. The AI adapts to where you are.

> The Question Quality Principle
>
> As you develop expertise, the quality of questions you ask AI improves. Beginner questions: "How do I centre a div?" Expert questions: "Given these constraints, what are the trade-offs between these three architectural approaches?" Better questions yield better AI partnership.

21.3 Discovering Your AI-Era Role with Your AI Partner

21.3.1 Exploration 1: What Won't Be Automated

Let's think critically about AI's limits:

```
Ask your AI:
What aspects of software development are unlikely
to be fully
automated by AI in the foreseeable future? Be
specific about why
each aspect resists automation.
```

Areas that remain human:

Problem definition: AI can solve problems; it struggles to identify which problems matter. "We need better user engagement" is a human judgment. AI can't determine what "better" means for your specific business.

Stakeholder communication: Clients often don't know what they want. Extracting requirements through conversation, reading non-verbal cues, and building trust remain human skills.

Ethical judgment: "Can we build this?" differs from "Should we build this?" AI can identify potential issues but can't make values-based decisions for you.

Context switching: Developers constantly shift between technical depth, business strategy, user empathy, and team dynamics. This fluid context-awareness is

distinctly human.

Accountability: When something goes wrong, humans are accountable. This responsibility shapes how we approach work in ways AI doesn't replicate.

```
Continue the conversation:
For each area you mentioned, give me a concrete
example from web
development where human judgment was essential.
```

21.3.2 Exploration 2: Your Unique Value

```
Ask your AI:
I'm a developer who works with AI tools. What's my
unique value?
What can I contribute that AI cannot? Be honest,
don't just
reassure me.
```

Honest assessment of human value:

- **Caring about outcomes**: AI doesn't care if your project succeeds. You do.
- **Understanding consequences**: You understand that slow-loading pages frustrate real humans.
- **Navigating ambiguity**: Real requirements are messy. "Make it better" requires human interpretation.
- **Building relationships**: Clients hire people, not AI. Trust is human.
- **Learning from failure**: You remember what didn't work and why. This wisdom guides future decisions.

```
Continue the conversation:
What skills should I deliberately develop to
remain valuable as
AI capabilities expand? What's the best investment
of my learning
time?
```

21.3.3 Exploration 3: Effective AI Partnership

```
Ask your AI:
How can developers work with AI most effectively?
What practices
lead to good outcomes? What mistakes should I
```

```
avoid?
```

Effective AI partnership practices:

Verify, don't trust blindly: AI makes confident-sounding errors. Always test generated code.

Understand what AI gives you: Don't use code you can't explain. If AI generates something, understand it before committing.

Iterate conversationally: One-shot prompts rarely produce optimal results. Refine through dialogue.

Provide context: AI works better with more context. Share constraints, goals, and existing code.

Stay critical: Just because AI suggests something doesn't make it right. Evaluate every suggestion.

```
Continue the conversation:
What are warning signs that I'm over-relying on
AI? How do I
maintain my own skills while using AI assistance?
```

21.3.4 Exploration 4: The Future Partnership

```
Ask your AI:
How might human-AI collaboration in software
development evolve
over the next 5-10 years? What should I prepare
for?
```

Possible evolutions:

- **More capable code generation**: AI will write more complex code more reliably
- **Better context awareness**: AI will understand your codebase more deeply
- **Integration with development workflows**: AI embedded in all tools
- **New specialisations**: AI trainers, prompt engineers, AI-human coordinators

What remains constant:

- Need for human judgment about what to build
- Requirement to understand what code does
- Importance of stakeholder communication
- Responsibility for outcomes

21.4 From Concept to Practice

21.4.1 Patterns for AI Collaboration

Throughout this course, you've developed patterns for working with AI:

Pattern 1: Exploration Before Implementation

```
Poor: "Write me a shopping cart component"

Better: "I need to implement a shopping cart. What
approaches exist?
What are the trade-offs between storing cart state
in localStorage
versus managing it in React state versus using a
backend? Help me
think through this before we write code."
```

Explore options before committing. AI is excellent at laying out alternatives.

Pattern 2: Explain, Then Generate

```
Poor: "Fix this bug" [pastes code]

Better: "This component should filter products by
category when the
dropdown changes. Instead, it shows all products
regardless of
selection. Walk me through what the code is doing,
then help me
identify where the logic is wrong."
```

Understanding before fixing leads to better solutions and learning.

Pattern 3: Incremental Building

```
Poor: "Build a complete e-commerce site"

Better:
1. "Let's start with the product data model. What
fields do I need?"
2. "Now help me create a ProductCard component to
display one product"
3. "Add filtering by category to the product list"
4. [Continue incrementally]
```

Complex systems are built incrementally. AI helps with each step.

Pattern 4: Context-Rich Requests

```
Poor: "Make the button look better"

Better: "This button is the primary call-to-action
for signup. It's
in a hero section with a dark background image.
Currently it's styled
with Tailwind as `bg-blue-500 text-white px-4
py-2`. The button feels
too small and doesn't stand out enough. Suggest
improvements that
maintain accessibility."
```

Pattern 5: Critical Review

When AI generates code:

1. **Read it:** don't copy blindly
2. **Understand it:** can you explain each line?
3. **Test it:** does it actually work?
4. **Evaluate it:** is this the right approach?
5. **Adapt it:** modify it for your specific needs

21.4.2 Maintaining Your Skills

AI assistance can atrophy skills if you're not careful.

Practice without AI regularly:

- Solve small problems manually first
- Use AI to check your work, not do your work
- Implement features without AI, then compare approaches

Understand fundamentals deeply:

- Know why code works, not just that it works
- Understand algorithms, not just implementations
- Learn concepts that transfer across technologies

Keep learning independently:

- Read documentation, not just AI summaries
- Work through problems yourself
- Build mental models that outlast any tool

21.4.3 Quality Control for AI-Generated Code

AI code needs review. Develop a checklist:

Functionality:

- ☐ Does it actually do what you asked?
- ☐ Edge cases handled?
- ☐ Error cases handled?

Code Quality:

- ☐ Readable and maintainable?
- ☐ Consistent with project style?
- ☐ Appropriate abstraction level?

Security:

- ☐ Input validation present?
- ☐ No hardcoded secrets?
- ☐ Appropriate error exposure?

Performance:

- ☐ Obvious inefficiencies?
- ☐ Unnecessary computations?
- ☐ Appropriate data structures?

21.4.4 Ethical Considerations

Working with AI raises ethical questions:

Attribution and honesty:

- Be clear about AI involvement in your work
- Don't claim AI-generated work as purely your own
- Understand your organisation's AI policies

Quality responsibility:

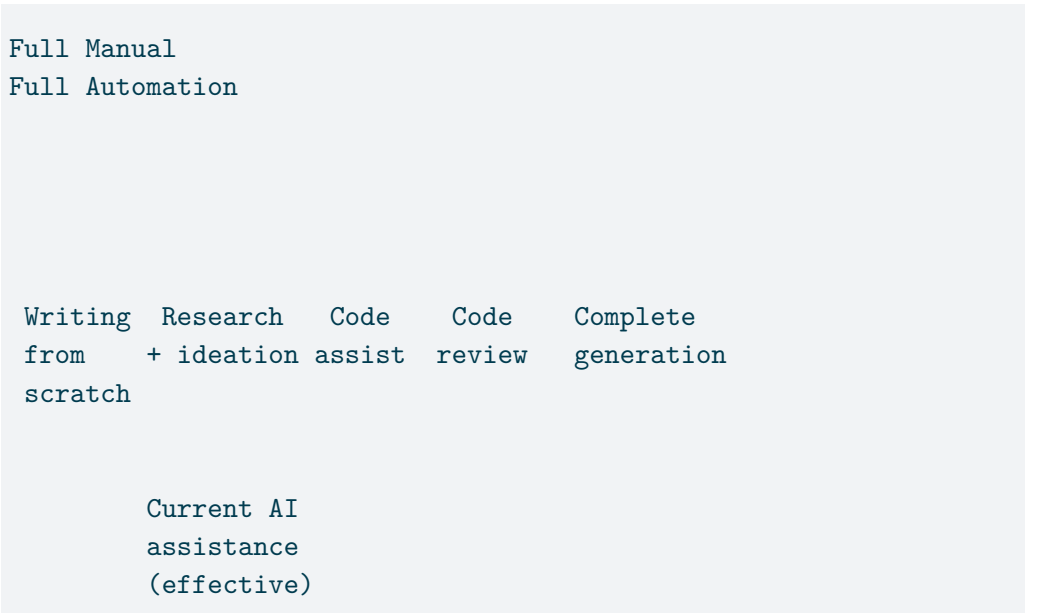
- You're responsible for code you commit, regardless of source
- "AI wrote it" isn't an excuse for bugs or security issues
- Review AI code as carefully as you'd review any code

Learning integrity:

- Using AI to learn is good
- Using AI to bypass learning undermines yourself
- The goal is capability, not just completion

21.5 Building Your Mental Model

21.5.1 The Developer-AI Collaboration Spectrum



Current AI is most valuable in the middle, enhancing your work, not replacing it. Full automation remains unreliable for complex, context-dependent work.

21.5.2 The Skill Evolution Model

Traditional Developer	AI-Era Developer
Writing code creation	Directing code
Memorising syntax concepts	Understanding
Solo problem-solving reasoning	Collaborative
Deep specialisation	Broad orchestration + selective depth

Skills that GAIN value:

- Problem definition
- Communication
- Judgment and evaluation

```
• Learning ability
• Ethical reasoning

Skills that CHANGE form:
• Coding → directing + reviewing
• Research → prompting + verifying
• Debugging → explaining + guiding
```

21.5.3 The Partnership Maturity Model

```
Level 1: User
- AI does tasks for you
- You accept outputs uncritically
- Dependency without understanding

Level 2: Consumer
- AI assists your work
- You evaluate outputs
- Selective use

Level 3: Collaborator
- AI thinks with you
- You guide direction
- True partnership

Level 4: Director
- AI amplifies your judgment
- You orchestrate capabilities
- AI is your tool, not your crutch
```

Progress through these levels by maintaining your independent capability while using AI appropriately.

21.6 Business Applications

21.6.1 Productivity and Quality

AI-assisted development can improve both speed and quality, if used well:

- **Speed**: Faster prototyping, less boilerplate
- **Quality**: More time for design and review
- **Learning**: Faster skill acquisition

- **Exploration**: Try more approaches

Time saved on rote tasks should be invested in higher-value activities (design, testing, user research), not just producing more code faster.

21.6.2 Team Dynamics

AI changes how teams work:

- **Review processes**: Now include AI-generated code review
- **Skill distribution**: AI democratises some capabilities
- **Communication**: More emphasis on clear requirements
- **Training**: Onboarding includes AI tool proficiency

21.6.3 Client Communication

Explaining AI involvement to clients:

Honest framing: "We use AI tools to enhance our productivity. All AI-generated code is reviewed and tested by our developers. The AI assists; humans remain responsible for quality."

Value focus: "AI helps us explore more options and iterate faster. This means you get better solutions, not just faster delivery."

21.6.4 Career Positioning

In an AI-augmented field:

- **Differentiate on judgment**: Anyone can generate code; not everyone can evaluate it
- **Emphasise communication**: Translating between humans and systems gains value
- **Build domain expertise**: AI is generic; domain knowledge is specific
- **Demonstrate responsibility**: Show you can be trusted with AI tools

> **i** Skills You Are Building
>
> This develops all the core professional skills, particularly **evaluating and improving solutions** and **assessing emerging technologies**. Working effectively with AI is now a core professional skill, requiring the judgment, communication, and evaluation abilities developed throughout this book.

21.7 Practice Exercises

Exercise Levels

- **Level 1**: Direct application
- **Level 2**: Minor modifications
- **Level 3**: Combining concepts
- **Level 4**: Problem-solving
- **Level 5**: Open-ended design

21.7.1 Exercise 21.1: AI Interaction Analysis (Level 1)

Review your AI conversations from throughout this book:

1. Find three conversations where AI was most helpful
2. Find one conversation where AI led you astray
3. For each, identify what made the interaction effective or ineffective
4. Write patterns you'll use going forward

21.7.2 Exercise 21.2: Manual Implementation (Level 2)

Build a small component **without** AI assistance:

1. Choose something you previously built with AI help
2. Implement it from scratch using only documentation
3. Compare: What was harder? What did you understand better?
4. Reflect on the value of both approaches

21.7.3 Exercise 21.3: AI Code Review (Level 3)

Practice reviewing AI-generated code:

1. Ask AI to generate a moderately complex component
2. Review it using the checklist provided
3. Identify at least three improvements
4. Discuss the improvements with AI
5. Implement the final version

21.7.4 Exercise 21.4: Partnership Optimisation (Level 4)

Develop your personal AI collaboration framework:

1. Document your most effective prompting patterns
2. Create a personal checklist for AI-generated code
3. Define boundaries (when you will/won't use AI)

4. Write guidelines for a team member new to AI tools

21.7.5 Exercise 21.5: Future Preparation (Level 5)

Create a 12-month professional development plan that accounts for AI:

1. Identify skills that will gain value with AI assistance
2. Identify skills you need to maintain independently
3. Plan specific learning activities for each
4. Define how you'll measure your progress
5. Write 500 words explaining your strategy

21.8 Chapter Summary

- AI amplifies human capability; it doesn't replace human judgment
- Effective AI partnership requires understanding, evaluation, and adaptation
- Maintaining independent skills prevents over-reliance
- The value shifts to problem definition, communication, and judgment
- Ethical use requires honesty, quality responsibility, and integrity
- Career success depends on how well you collaborate with AI, not whether

21.9 Final Reflection

As you complete this book, reflect on your journey:

Technical Growth:

☐ Can you build web applications with HTML, CSS, and JavaScript?
☐ Can you work with WordPress as both a CMS and an API?
☐ Can you create React components and manage state?
☐ Can you evaluate and adopt CSS frameworks appropriately?

Professional Development:

☐ Can you make and defend technology decisions?
☐ Can you communicate technical concepts to non-technical stakeholders?
☐ Can you use version control and professional practices?
☐ Can you evaluate emerging technologies systematically?

AI Partnership:

☐ Can you collaborate effectively with AI tools?
☐ Do you maintain your own understanding and skills?
☐ Can you evaluate AI output critically?
☐ Are you positioned to adapt as AI capabilities evolve?

21.10 Your Final Learning Journal Entry

Record your thoughts on completing this course:

1. **Your Journey**: What surprised you most about learning web development? What was harder than expected? Easier?
2. **AI Partnership Evolution**: How did your relationship with AI tools change from the beginning to now? What patterns work best for you?
3. **Business Thinking**: How do you now approach technology decisions differently than when you started?
4. **Identity**: How do you see yourself as a developer? What kind of professional do you want to become?
5. **Next Steps**: What will you build next? What will you learn? How will you continue growing?

21.11 Looking Forward

This book ends, but your development journey continues.

You now have:

- **Technical foundations** across the modern web stack
- **Business thinking** for technology decisions
- **AI collaboration skills** for enhanced productivity
- **Professional practices** for quality work
- **Career frameworks** for ongoing growth

You also have **the ability to learn**. Technologies will change. Frameworks will evolve. AI capabilities will expand. But your foundation, understanding how to think about problems, evaluate solutions, communicate with stakeholders, and collaborate with tools, remains valuable.

The developers who thrive in the AI era won't be those who resist AI or those who depend on AI. They'll be those who partner with AI thoughtfully, maintaining their own judgment, understanding, and skills while using AI to amplify their capability.

Now build something, learn from your mistakes, collaborate with AI, and contribute to the community.

Welcome to your career as a web professional.

Part VII

Appendices

How to Use This Book

This book is a reference, not a reading schedule for every module. The unit sets the order. The book supports the work.

The 2027 edition adds three practice chapters at the front of the build:

- Chapter 2, **Specify Before You Build**, turns a client brief into a contract with acceptance criteria.
- Chapter 3, **Checks That Run**, turns those criteria into evidence that the site still works.
- Chapter 4, **The Agent-Ready Repo and Handover**, turns the build into a repository that another person can run and maintain.

Read those three when you are building. Read the concept chapter you need when you need it.

If you need to	Read
Understand the client and write the spec	Chapter 2, then Chapter 5
Audit the site at 375px	Chapters 5 and 6
Write a check and read the output	Chapter 3
Integrate live data and handle failure	Chapter 8
Choose a content platform	Chapters 10 and 11
Extend or audit a WordPress site	Chapters 11 and 12
Present data from an API	Chapters 8 and 13
Build a component front end	Chapter 15
Polish, hand over and defend the build	Chapters 16 and 17
Evaluate a technology and recommend it	Chapters 19, 20 and 21

The 2027 course map appendix maps each module to the chapters that support it. When the course map and this book disagree, the course map wins.

This edition supports ISYS3004 Business Web and Mobile Technologies, S1 2027.

2027 Course Map

This appendix maps the 2027 modules to the chapters in this book. The unit materials are the source of truth. When the course map and this appendix disagree, the course map wins.

The module order is fixed. Calendar breaks do not change the numbering.

Module	Core question	Book chapters	Assessment artefact
01 Ship by Friday	What is a website, and how does it exist?	Chapter 1, AI as Partner; Chapter 4, The Agent-Ready Repo and Handover; Chapter 5, Structure	Personal site; client site v0; `SPEC.md` v0
02 The 375px Problem	Why does it break on a phone?	Chapter 5, Structure; Chapter 6, Presentation	375px audit and three fixes
03 Say What You Mean	What does the client actually need?	Chapter 2, Specify Before You Build; Chapter 5, Structure	`SPEC.md` v1 and replication test
04 Checks It Can't Charm	How do you know it works?	Chapter 3, Checks That Run; Chapter 17, Professional Practices	Checks passing and one new check
05 Keys in the Window	Where does the data come from, and who can see it?	Chapter 8, Connection	API integration, secrets and the unhappy path

Module	Core question	Book chapters	Assessment artefact
06 The Taste Test	Which build would the client pay for?	Chapter 6, Presentation; Chapter 17, Professional Practices	Client site final and viva
07 Who Chose React?	Who maintains this on Monday?	Chapter 10, Why CMS?; Chapter 11, WordPress as Platform; Chapter 13, Headless Architecture	Platform spike and decision record
08 The Inherited Site	What did the previous developer leave?	Chapter 11, WordPress as Platform; Chapter 12, Extending Platforms	Sprint review and inherited-site audit
09 Data Behind the Glass	Who can see the data?	Chapter 8, Connection; Chapter 13, Headless Architecture	REST data integration
10 Pocket-Sized	Does the client need an app?	Chapter 19, Evaluating Emerging Technologies	Installable app
11 The Last 20%	What is the hard part?	Chapter 16, Rapid Development; Chapter 17, Professional Practices	Final app and viva
12 Tech Radar	What would you recommend, and why?	Chapter 19, Evaluating Emerging Technologies; Chapter 20, Your Development Career; Chapter 21, The AI-Era Developer	Audit practice and mock viva

Where the practice chapters sit

The practice spine is designed to be read in order:

1. Chapter 2, **Specify Before You Build**.
2. Chapter 3, **Checks That Run**.
3. Chapter 4, **The Agent-Ready Repo and Handover**.

The concept chapters can be read in any order after that. The module map above shows which ones a module uses.

References

About the Author



Michael Borck is a software developer and educator. He developed the Intentional Prompting methodology to help programmers keep agency and build understanding while using AI tools.

Michael believes that the future of programming lies not in delegating to AI, but in conversing with it, treating AI as a collaborative partner that enhances human capability rather than replacing human understanding.

When not writing about AI collaboration, Michael works on practical applications of these principles across software development, education, and creative projects. He creates educational software and resources, and explores the 80/20 principle in learning and productivity.

Connect

- GitHub (https://github.com/michael-borck): code, books and projects
- books.borck.education (https://books.borck.education): the free book collection
- 8020workshop.com (https://8020workshop.com): passion projects and workshops
- LinkedIn (https://linkedin.com/in/michaelborck)

Other Books in This Series

Foundational Methodology:

- *Conversation, Not Delegation: Your Expertise + AI's Breadth = Amplified Thinking*
- *Converse Python, Partner AI: The Python Edition*

Python Track:

- *Think Python, Direct AI: Computational Thinking for Beginners*
- *Code Python, Consult AI: Python Fundamentals for the AI Era*
- *Ship Python, Orchestrate AI: Professional Python in the AI Era*

For Educators:

- *Partner, Don't Police: AI in the Business Classroom*